

Capital University of Science and Technology Islamabad

Lab Manual

Web Application Development

(CS3191)

2ndst Edition

Last Updated: Summer 2026

DEPARTMENT OF COMPUTER SCIENCE



Supervision and Coordination

Mr. Salman Ahmed

Senior Lecturer

Faculty of Computing



Written by

Mr. Afnan Zahoor

Lab Instructor

Faculty of Computing

Mr. Zohaib Ahmed

Lab Instructor

Faculty of Computing

Table of Content:

LAB 01: INTRODUCTION & DEVELOPMENT ENVIRONMENT	1
LAB 02: HTML BASICS	13
LAB 03: CASCADING STYLE SHEETS (CSS)	38
LAB 04: BOOTSTRAP CSS	70
LAB 05: JAVASCRIPT BASICS.....	86
LAB 06: JAVASCRIPT FUNCTIONS & ARRAYS.....	112
LAB 07: DOM & EVENTS.....	128
LAB 08: REACT BASICS	141
LAB 09: REACT FUNDAMENTALS — USEEFFECT, CONDITIONAL RENDERING, AND REACT ROUTER	152
LAB 10: ASYNC JS & FETCH API — PROMISES, ASYNC/AWAIT, AND THE FETCH API	168
LAB 11: NODE.JS BASICS — NODE RUNTIME, NPM/PACKAGE.JSON, AND A SIMPLE HTTP SERVER.....	179
LAB 12: EXPRESS.JS REST API — SERVER SETUP, ROUTING, AND BASIC CRUD ROUTES.....	190
LAB 13: SQL & SEQUELIZE — SQL CRUD OPERATIONS AND CONNECTING EXPRESS TO SQL USING SEQUELIZE	198
LAB 14: FULL-STACK INTEGRATION & DEPLOYMENT — CONNECTING FRONTEND, BACKEND, AND DATABASE	208

Lab Schedule

Lab #	Contents
Lab 1	Introduction & Dev Environment Lab: Setting up code editor and browser dev tools. Git & GitHub basics. Hosting a simple static site (e.g., GitHub Pages).
Lab 2	HTML Basics Lab: HTML boilerplate, basic tags, headings, paragraphs, lists and attributes.
Lab 3	HTML & CSS Lab: HTML forms and tables combined with CSS selectors, colors, fonts and the Box Model.
Lab 4	Responsive Design Lab: Flexbox layout, media queries, and Bootstrap grid/components.
Lab 5	JavaScript Basics Lab: Variables, data types, operators, conditionals and loops.
Lab 6	JavaScript Functions & Arrays Lab: Functions, arrow functions, arrays/objects and array methods.
Lab 7	DOM & Events Lab: Selecting/modifying DOM elements and handling events and form/input events.
Lab 8	React Basics Lab: Project setup, JSX, components, props and state (useState).
Mid Term	Exams
Lab 9	React Fundamentals Lab: useEffect, conditional rendering, and React Router.
Lab 10	Async JS & Fetch API Lab: Promises, async/await, and the Fetch API for HTTP requests.
Lab 11	Node.js Basics Lab: Node runtime, npm/package.json, and a simple HTTP server.
Lab 12	Express.js REST API Lab: Express server setup, routing, and basic CRUD routes.
Lab 13	SQL & Sequelize Lab: SQL CRUD operations and connecting Express to SQL using Sequelize.
Lab 14	Full-Stack Integration & Deployment Lab: Connecting frontend, backend and database.

Lab 01: Introduction & Development Environment

Background

Web application development brings together many small, focused tools: a code editor for writing HTML, CSS and JavaScript; a browser and its built-in developer tools for inspecting and debugging what actually runs; and a version control system for tracking changes and collaborating safely. This lab sets up the environment used throughout the rest of this course — Visual Studio Code, Chrome DevTools, and Git and GitHub — and finishes by publishing a simple static website live on the internet using GitHub Pages.

Setting Up Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight source-code editor from Microsoft, and is the editor used throughout this course. It supports every language covered in this lab manual — HTML, CSS, JavaScript, and later Node.js — through a large ecosystem of extensions.

1. Go to <https://code.visualstudio.com> and download the installer for your operating system (Windows, macOS, or Linux).
2. Run the installer, accepting the default options. On Windows, make sure the “Add to PATH” option is checked, since this allows VS Code to be opened from a terminal using the code command.
3. Launch VS Code once installation finishes.

Once VS Code is open, create a project folder for this course and open it: File → Open Folder..., then create a new file named index.html inside it (File → New File, or the New File icon in the Explorer sidebar). Typing an exclamation mark (!) and pressing Tab on the first line of a new, empty .html file triggers VS Code's Emmet abbreviation, which expands into a complete HTML boilerplate automatically:

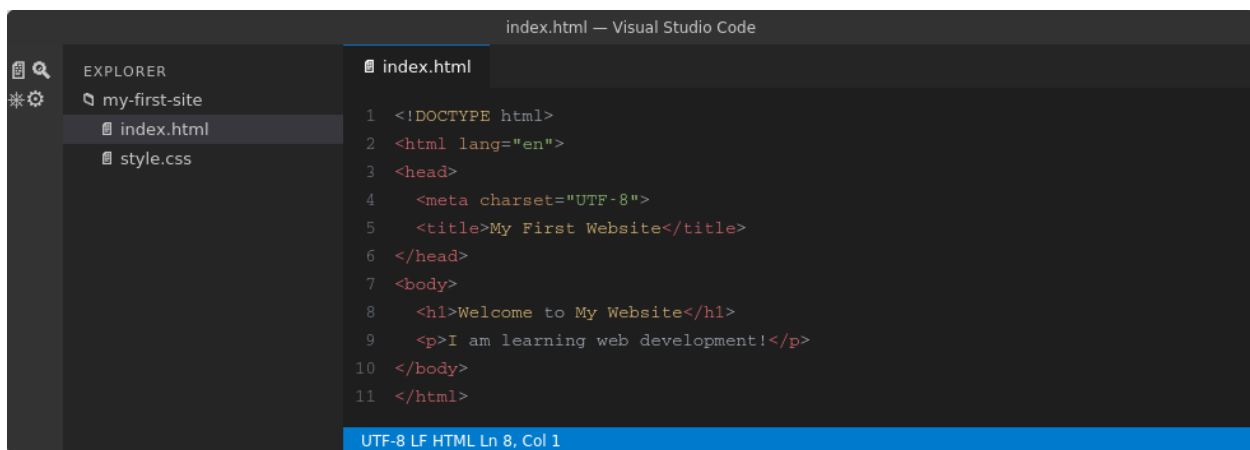


Figure 1: Setting Up Visual Studio Code

The main parts of the interface are:

- **Activity Bar** — the narrow strip of icons on the far left, used to switch between the file Explorer, Search, Source Control, and Extensions views.
- **Explorer (sidebar)** — shows the folders and files in the currently open project.

- **Editor** — the main area where file contents are shown and edited, with tabs across the top for every open file.
- **Status bar** — the blue bar along the bottom, showing the file's encoding, line ending style, language mode, and (once Live Server is installed) the local server port.

Installing the Live Server Extension

Live Server is a VS Code extension that runs a small local web server and automatically reloads the browser every time a file is saved, which removes the need to manually refresh the browser after every change — a significant time-saver over the course of this lab manual.

1. Click the Extensions icon in the Activity Bar (or press Ctrl+Shift+X).
2. Search for “Live Server” (by Ritwick Dey) and click Install.
3. Once installed, right-click anywhere inside index.html and choose “Open with Live Server”, or click “Go Live” in the status bar.

This opens the default browser at an address such as 127.0.0.1:5500, showing the live, auto-refreshing output of the HTML file:

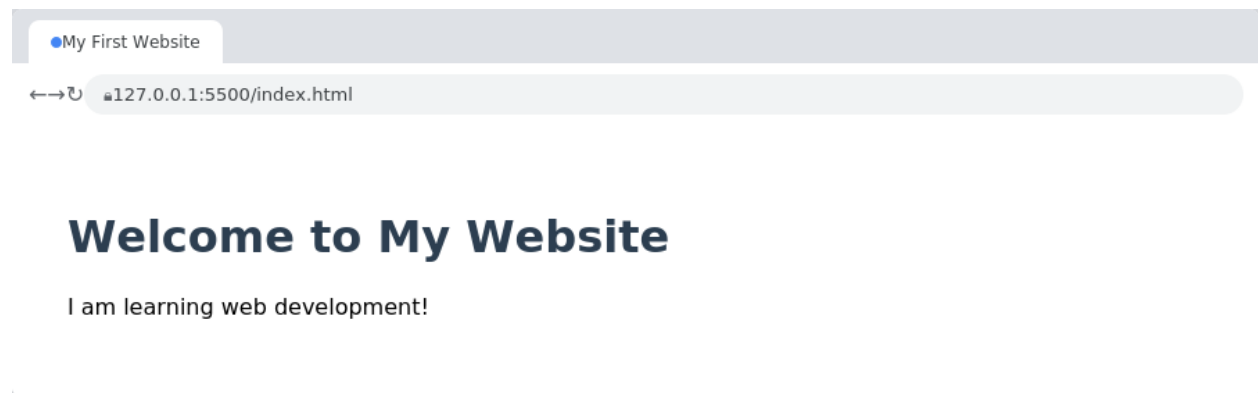


Figure 2: Installing the Live Server Extension

- **Prettier** — an extension that automatically formats HTML, CSS and JavaScript code consistently on save; useful throughout this course.
- **Auto Rename Tag** — automatically renames the matching closing tag whenever an opening HTML tag is renamed, and vice versa.

Exploring Browser Developer Tools

Every modern browser includes a set of developer tools (DevTools) for inspecting a running page: its HTML structure, its applied CSS, any JavaScript output or errors, and the network requests it makes. In Chrome or Edge, DevTools is opened with F12, Ctrl+Shift+I (Cmd+Option+I on macOS), or by right-clicking any element on the page and choosing “Inspect”.

The Elements Panel:

The Elements panel shows the page's HTML as a live, expandable tree, and the CSS rules currently applied to whichever element is selected:

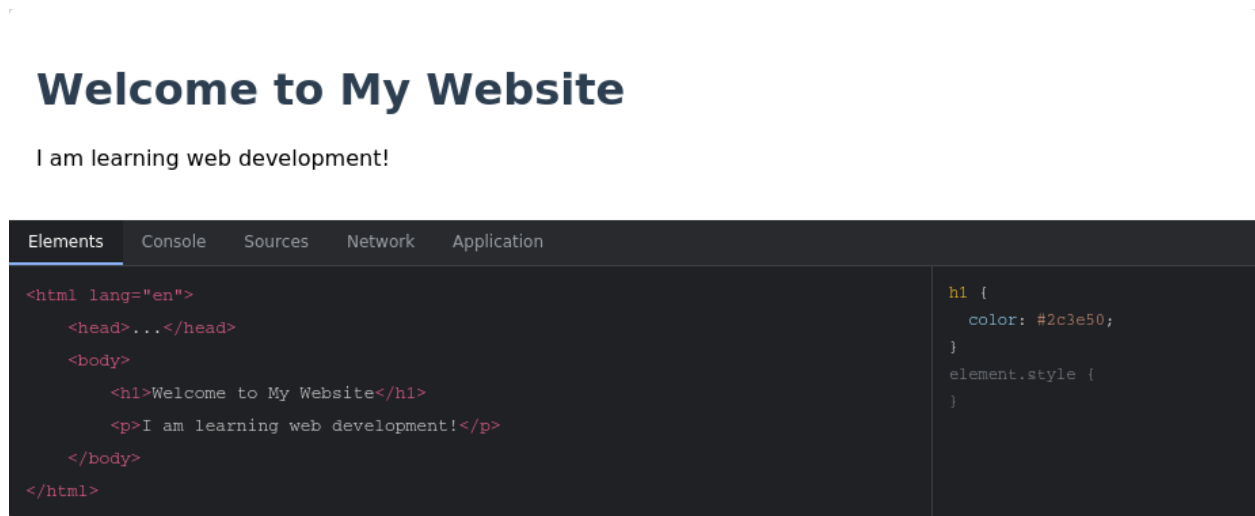


Figure 3: The Elements Panel:

Edits made directly in the Elements panel — changing text, adding a CSS property, toggling a class — update the page immediately, which makes it an extremely fast way to experiment with a layout or a style before editing the actual source file. These changes are not saved anywhere, however; reloading the page discards them, so any change that should be kept still needs to be copied back into VS Code.

The Console Panel

The Console panel shows messages logged by JavaScript running on the page, using `console.log()`, and reports any errors the page's JavaScript encountered:

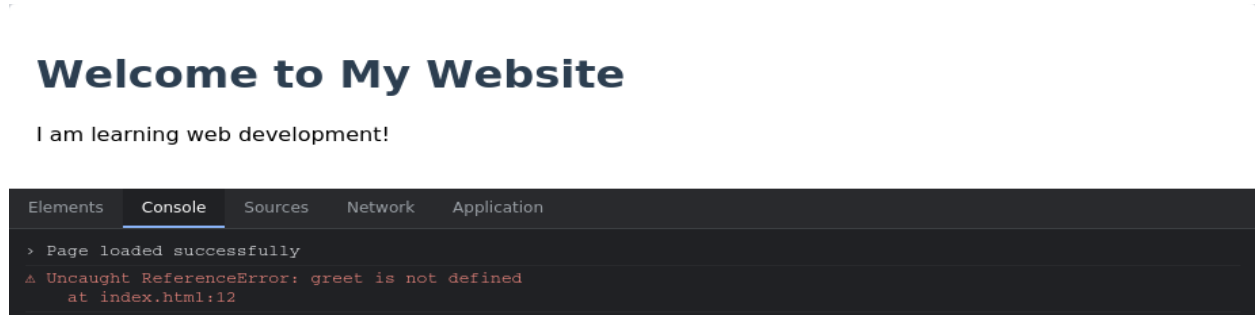


Figure 4: The Console Panel

The Console is also an interactive JavaScript prompt: any expression typed directly into it is executed immediately in the context of the current page, which makes it a convenient place to try out a snippet of JavaScript, or to inspect the value of a variable, without editing any file.

- **Sources** — shows every file the page loaded and allows JavaScript to be paused and stepped through with breakpoints; covered in more detail in Lab 7.
- **Network** — lists every request the page makes (images, stylesheets, API calls) along with its status code and timing; covered in more detail in Lab 10, when this course starts making requests to a REST API.
- **Application** — inspects data stored by the page itself, such as cookies and local storage.

Version Control with Git

Git is a distributed version control system: it records snapshots of a project over time, letting changes be reviewed, reverted, or merged safely, and is the foundation that GitHub, used later in this lab, is built on.

Installing Git

1. Go to <https://git-scm.com/downloads> and download the installer for your operating system.
2. Run the installer, accepting the default options.
3. Open a terminal (VS Code has a built-in terminal, opened with `Ctrl+``) and confirm the installation by running `git --version`.

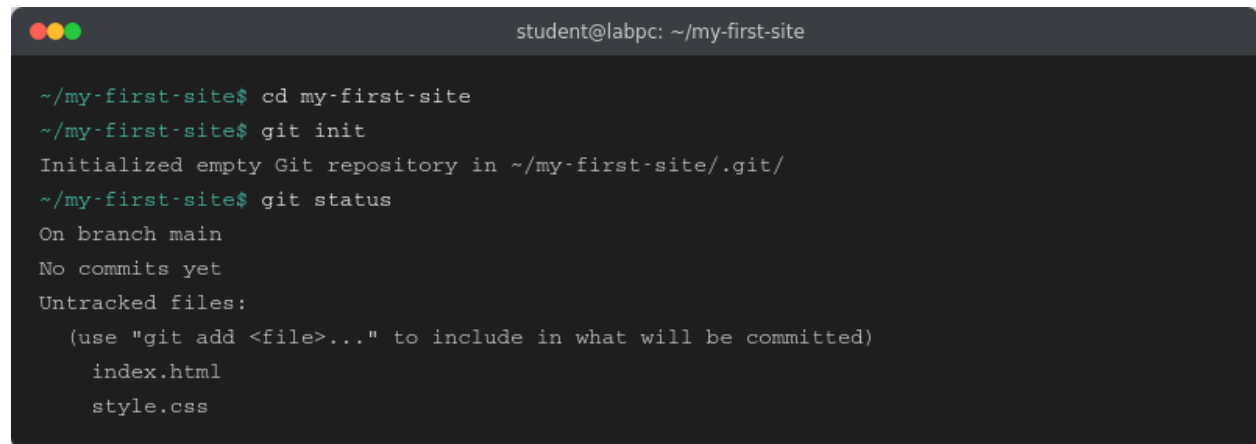
Before using Git for the first time, set a name and email address, which are attached to every commit made on this machine:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

Tracking a Project with Git

Inside the my-first-site project folder created earlier, turn the folder into a Git repository and check its status:

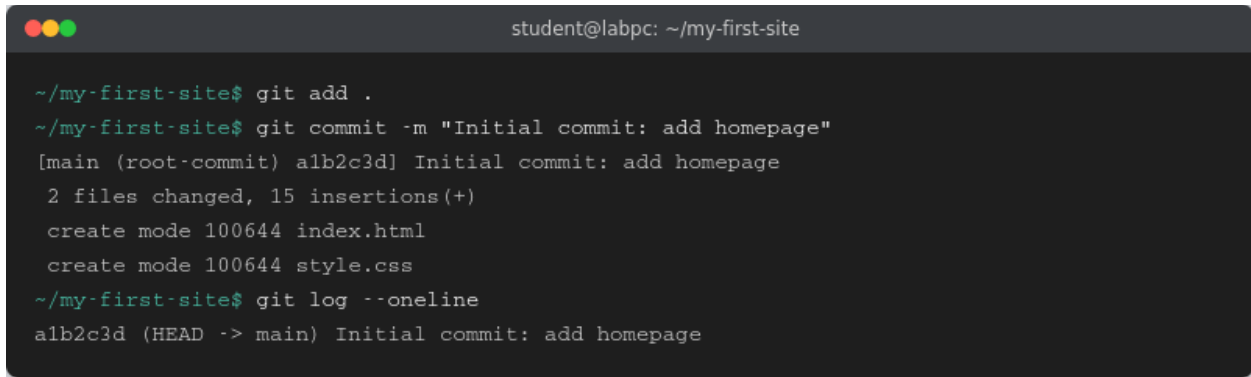
A terminal window with a dark background and light green text. The title bar shows 'student@labpc: ~/my-first-site'. The terminal output shows the following commands and their results:

```
~/my-first-site$ cd my-first-site
~/my-first-site$ git init
Initialized empty Git repository in ~/my-first-site/.git/
~/my-first-site$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
    style.css
```

Figure 5: Tracking a Project with Git

- **git init** creates a hidden `.git` folder that Git uses to track the project's history; this only needs to be done once per project.
- **git status** reports which files have changed since the last commit, and which files are not yet tracked by Git at all.

Untracked files are added to Git's staging area with `git add`, and then permanently recorded with `git commit`:



```

student@labpc: ~/my-first-site

~/my-first-site$ git add .
~/my-first-site$ git commit -m "Initial commit: add homepage"
[main (root-commit) a1b2c3d] Initial commit: add homepage
 2 files changed, 15 insertions(+)
 create mode 100644 index.html
 create mode 100644 style.css
~/my-first-site$ git log --oneline
a1b2c3d (HEAD -> main) Initial commit: add homepage

```

Figure 6: Tracking a Project with Git

- **git add .** stages every changed file in the current folder; a specific file can be staged instead with `git add <filename>`.
- **git commit -m "message"** permanently records the staged changes as a new snapshot, labelled with a short, descriptive message.
- **git log --oneline** lists the commit history, one line per commit, each with a short commit hash such as `a1b2c3d`.

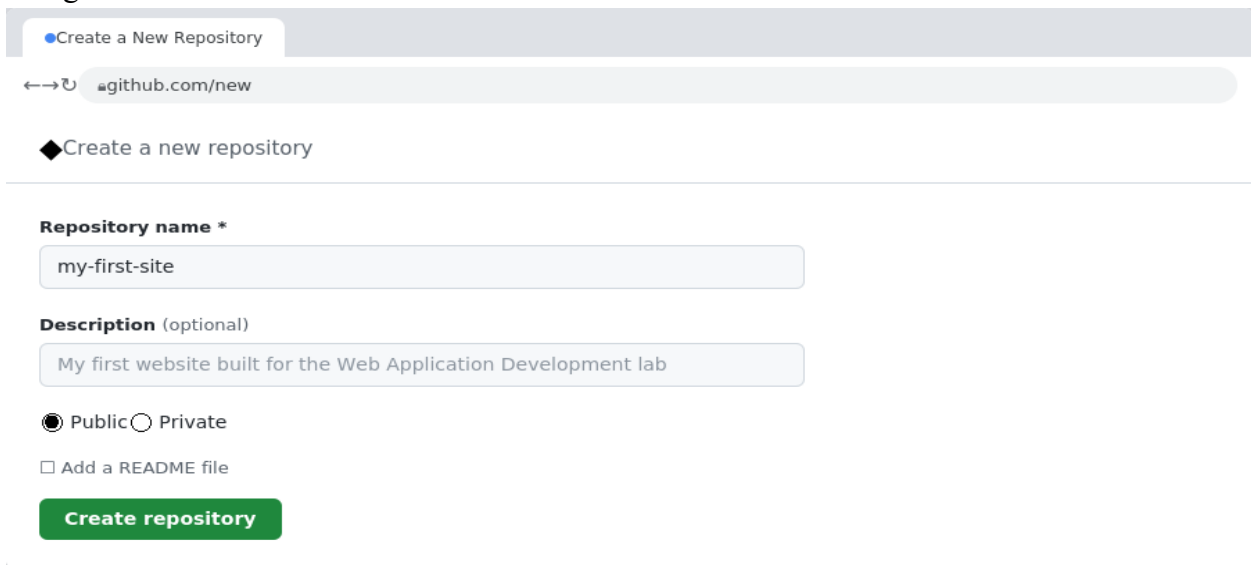
A typical Git workflow, repeated many times over the life of a project, is simply: edit files, `git add`, `git commit` — sometimes described as edit, stage, snapshot.

Hosting Code on GitHub

GitHub is a cloud platform for hosting Git repositories. It adds a web interface, collaboration tools such as pull requests and issues, and — as this lab shows — the ability to publish a static site directly from a repository.

Creating a GitHub Account and Repository

1. Go to <https://github.com> and sign up for a free account, if you do not already have one.
2. Click the + icon in the top-right corner and choose “New repository”, or go directly to github.com/new.



• Create a New Repository

←→ github.com/new

◆ Create a new repository

Repository name *

my-first-site

Description (optional)

My first website built for the Web Application Development lab

☒ Public ☐ Private

☐ Add a README file

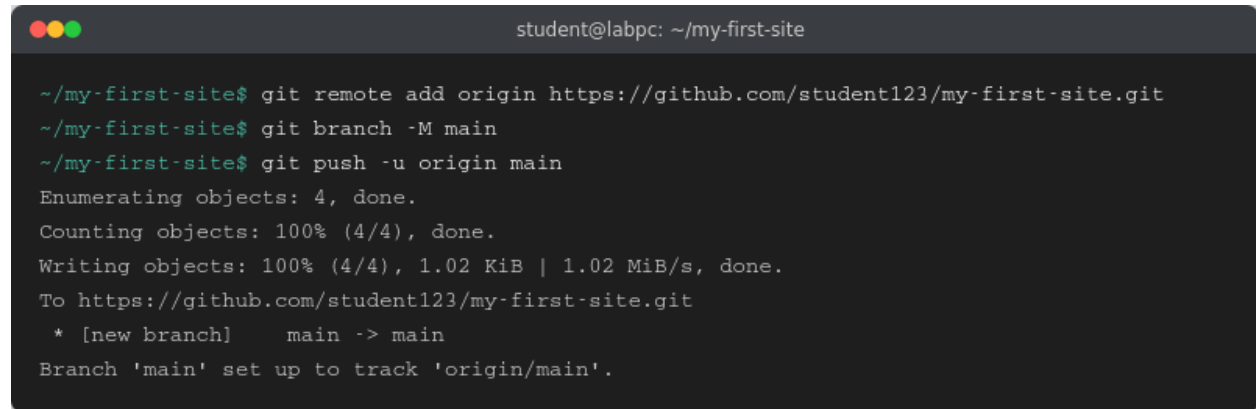
Create repository

Figure 7: Creating a GitHub Account and Repository

Give the repository the same name as the local project folder (my-first-site), leave it Public, and click Create repository — do not initialize it with a README, since the local project already has files that will be pushed to it.

Connecting a Local Repository to GitHub

GitHub shows a remote URL immediately after the repository is created. This URL is registered with the local repository as origin, and the local commit history is then pushed to it:



```
student@labpc: ~/my-first-site

~/my-first-site$ git remote add origin https://github.com/student123/my-first-site.git
~/my-first-site$ git branch -M main
~/my-first-site$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.02 KiB | 1.02 MiB/s, done.
To https://github.com/student123/my-first-site.git
 * [new branch]    main -> main
Branch 'main' set up to track 'origin/main'.
```

Figure 8: Connecting a Local Repository to GitHub

- **git remote add origin <url>** registers the GitHub repository's URL under the short name origin, so it does not need to be typed out again.
- **git branch -M main** ensures the local branch is named main, matching GitHub's default.
- **git push -u origin main** uploads every commit on the main branch to GitHub; the -u flag remembers this pairing, so a plain git push is enough from now on.

Refreshing the repository's page on GitHub now shows every file from the local project, along with the commit message used when each one was last changed:

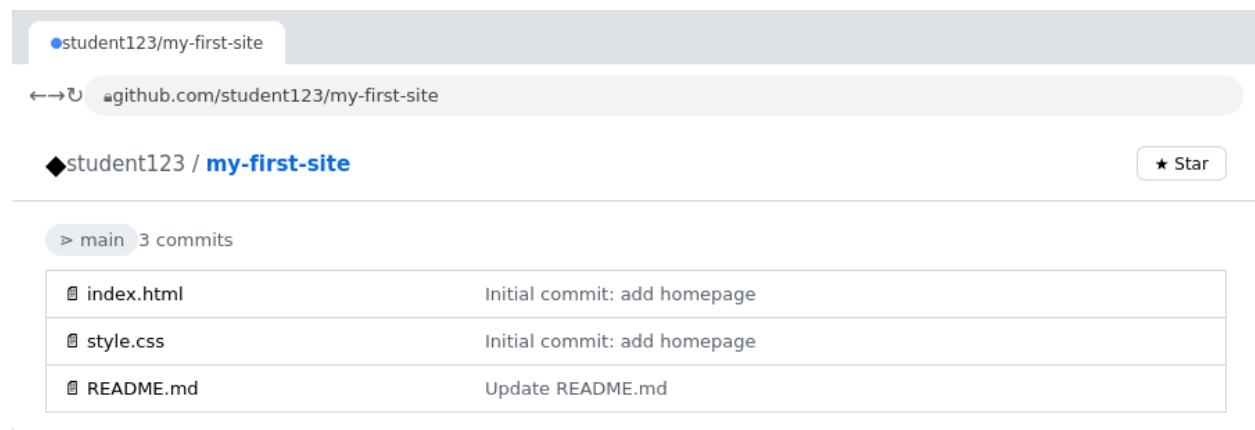


Figure 9: Connecting a Local Repository to GitHub

Cloning an Existing Repository

The reverse operation — downloading a copy of a repository that already exists on GitHub, for example on a different computer — uses git clone:

```
git clone https://github.com/student123/my-first-site.git
```

This creates a new folder containing the full project and its entire commit history, already connected to origin, ready to continue working on immediately.

Publishing with GitHub Pages

GitHub Pages serves the contents of a repository as a live website, directly from GitHub, at no cost — ideal for the static sites built early in this course, and for the finished full-stack project later on.

1. On the repository's GitHub page, go to Settings, then Pages in the left-hand menu.
2. Under Source, choose “Deploy from a branch”.
3. Under Branch, select main and / (root), then click Save.

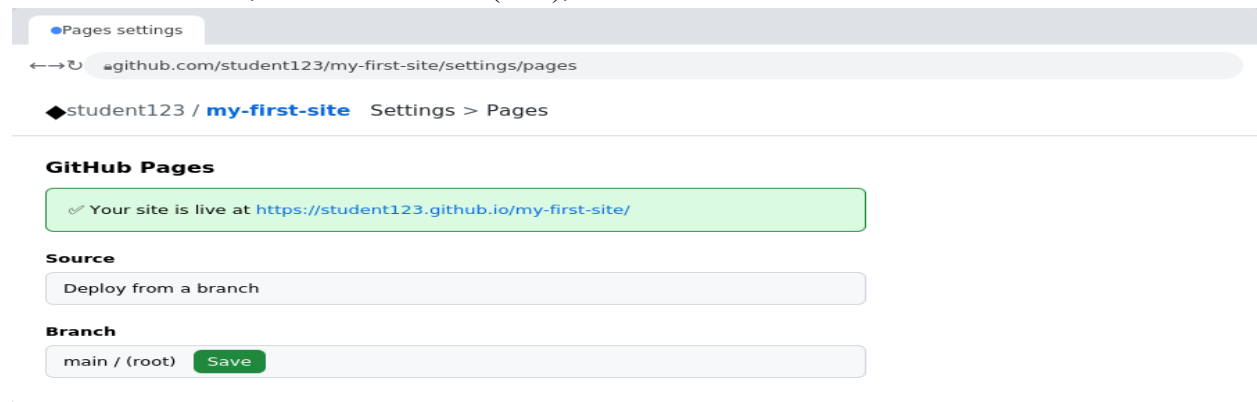


Figure 10: Publishing with GitHub Pages

After a minute or two, GitHub Pages finishes building the site and shows a green confirmation banner with the live URL. Visiting that URL shows exactly the same index.html page, now hosted publicly on the internet rather than only on the local machine:

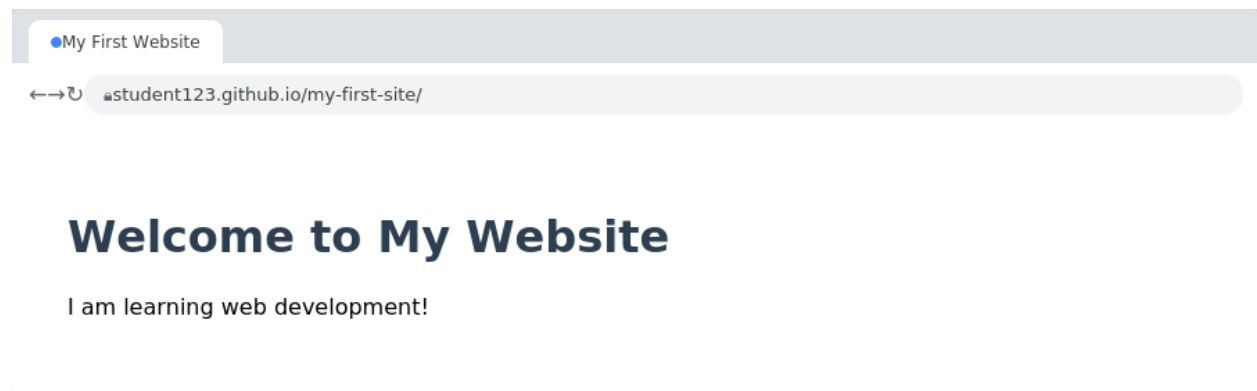


Figure 11: Publishing with GitHub Pages

From now on, every time a change is committed and pushed to the main branch, GitHub Pages automatically rebuilds and republishes the site within a minute or two, with no extra steps required.

Publishing with ProFreeHost:

ProFreeHost allows students to publish a static website on the internet using free web hosting. Unlike GitHub Pages, where the website is published directly from a GitHub repository, ProFreeHost requires the website files to be uploaded to the hosting account. In this lab, students will upload the index.html file created in Visual Studio Code and access it through the website address provided by ProFreeHost.

1. Open the [ProFreeHost](https://profreehost.com) website and create an account(check email after creating the account and press the activate now button)or sign in if you already have an account.

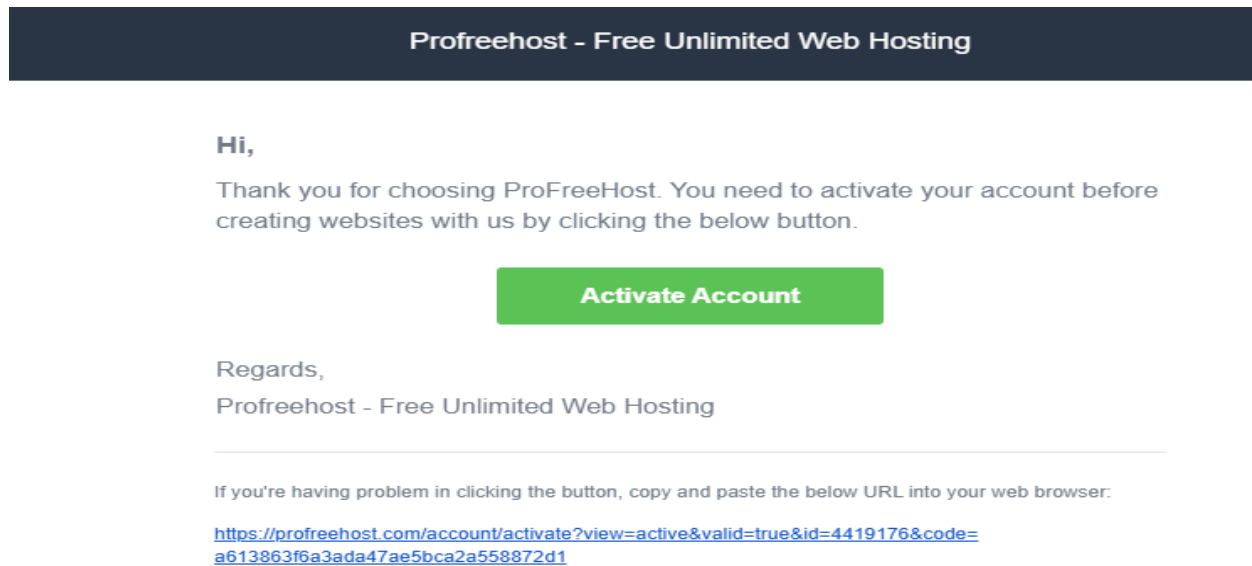


Figure 12: Publishing with ProFreeHost:

2. After signing in, create a **new hosting account**. During this process, ProFreeHost will ask you to choose or create a website address. Complete the required information and create the hosting account (Click the green "Check Availability" button to confirm "Domain" isn't already taken).

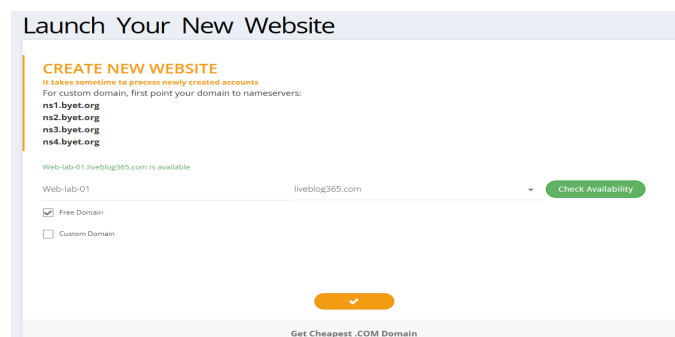


Figure 13: Publishing with ProFreeHost:

3. After the hosting account is created, open its **Control Panel**. The Control Panel contains the tools required to manage your hosted website.
4. In the Control Panel, locate and open **File Manager**. The File Manager allows you to upload and manage the files that will be displayed on your website.

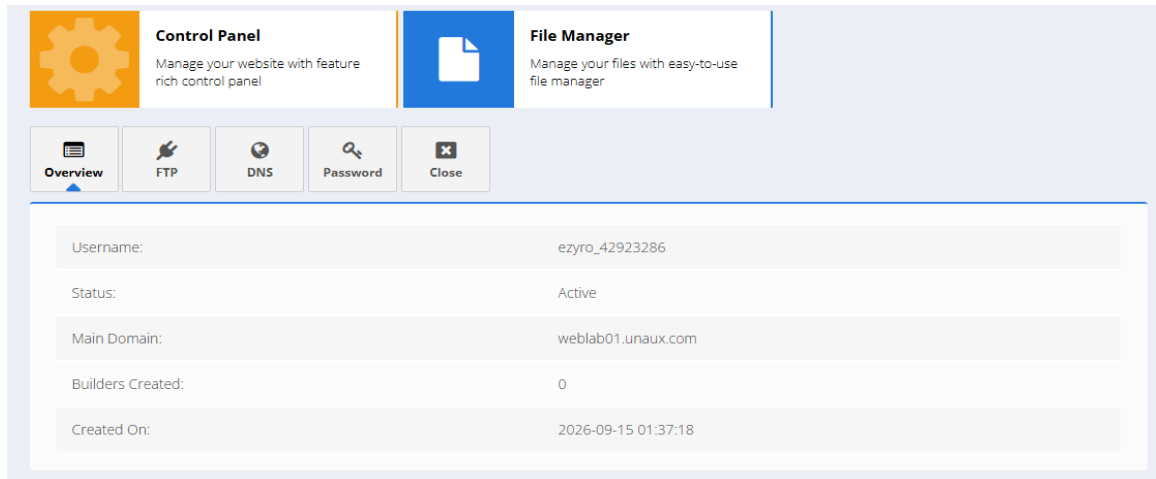


Figure 14: Publishing with ProFreeHost:

5. Inside the File Manager, open the **public website directory**. Depending on the hosting setup, this directory may be named htdocs, public_html, or another directory specified by the hosting service. Files placed in this directory are made available through your website address.
6. Upload the index.html file from your web-lab-01 project into the public website directory.
7. Make sure that index.html is located directly inside the public website directory. The index.html file is used as the main page of the website when visitors open the website address.

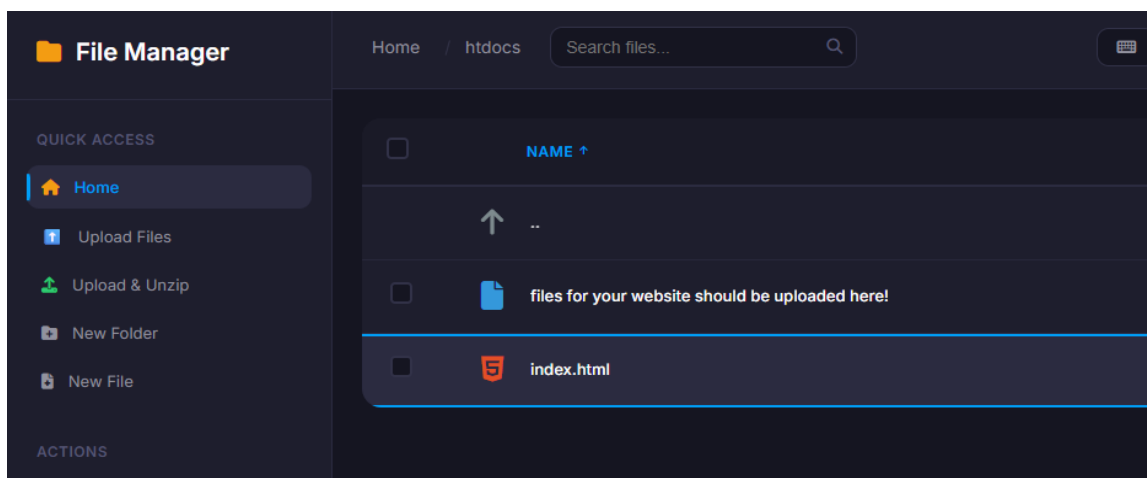


Figure 15: Publishing with ProFreeHost:

8. Open the website address provided by ProFreeHost in a new browser tab.



Welcome to My Website

I am learning web development!

Figure 16: Publishing with ProFreeHost:

9. Verify that the webpage displayed online is the same webpage that you created and tested locally in Visual Studio Code.

The complete deployment process is:

Create Website Locally



Create ProFreeHost Account



Create Hosting Account



Open Control Panel



Open File Manager



Open Public Website Directory



Upload index.html



Open Website URL



Verify the Website

After successful deployment, the website can be accessed through the ProFreeHost URL from a browser. This demonstrates how a locally developed static website can be transferred to a web server and made publicly accessible on the internet.

Lab Objectives

The students will be able to:

- Install and configure Visual Studio Code and the Live Server extension for local web development.
- Use Chrome DevTools to inspect a page's HTML and CSS, and to read logged messages and errors from the Console.
- Explain the purpose of Git and use git init, add, commit, status and log to track a project's history.
- Create a GitHub repository, and use git remote, push, pull and clone to synchronize a local project with it.
- Publish a static website live on the internet using GitHub Pages.

Walkthrough Task

Task: Publish a Personal Introduction Page

Create a new folder named intro-page containing a single index.html file with a short personal introduction — your name, a heading, and two or three sentences about yourself and why you are taking this course. View it locally using Live Server, then initialize a Git repository, make an initial commit, create a matching repository on GitHub, push your code, and enable GitHub Pages so the page is reachable at a public URL.

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Install Visual Studio Code and the Live Server extension on your own laptop, or confirm they are already installed, ready for the lab session.

Home Task 02

Create a free GitHub account, if you do not already have one.

Home Task 03

Read the short guide “Hello World” in the GitHub Docs (<https://docs.github.com>) and note the difference between a repository, a commit, and a branch, in your own words.

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in a folder specified by your lab instructor.

Practice Task 1

Deliberately introduce a JavaScript error into a page (for example, call a function that does not exist) and use the DevTools Console to locate and describe the exact error message and line number reported.

Practice Task 2

Make three separate commits to the intro-page repository from the walkthrough task — for example, changing the heading, then the wording, then adding a new paragraph — each with its own descriptive commit message, and use `git log --oneline` to display the resulting history.

Practice Task 3

On a partner's machine, or in a second folder on your own machine, use `git clone` to download the intro-page repository, make a small change, and push it back to GitHub.

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The page must display correctly locally and once published on GitHub Pages.	30%	The Live Server output and the published URL will both be checked.
Git usage	The repository must show a genuine, meaningful commit history rather than a single commit.	30%	Review of <code>git log --oneline</code> and commit messages.
GitHub Pages deployment	The site must be correctly published and reachable at its GitHub Pages URL.	25%	The submitted URL will be opened and verified.
Presentation	The submitted URL and terminal output should be captured and clearly labelled with the student's name.	15%	Review of the submitted screenshot and repository link.

Lab 02: HTML Basics

Introduction

HTML is the language in which most websites are written. HTML is used to create web pages and make them functional. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Basic HTML Structure

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

The **<!DOCTYPE>** declaration is not an HTML tag; it is an instruction to the web browser about what version of HTML the page is written in. Here **<!DOCTYPE html>** represents we are using HTML 5.

Tip: Always add the **<!DOCTYPE html>** declaration to your HTML documents, so that the browser knows what type of document to expect. The opening **<html>** tag indicates that anything between it and a closing

</html> tag is HTML code. The **<head>** element is a container for metadata (data about data) and is placed between the **<html>** tag and the **<body>** tag. The **<title>** element defines the title of the document, and is required in all HTML documents.

The **<body>** tag indicates that anything between it and the closing **</body>** tag should be shown inside the main browser window. The words between **<h1>** and **</h1>** element are main heading, similarly words between **<p>** and **</p>** represents a paragraph.

In short

- The **<!DOCTYPE html>** declaration defines this document to be HTML5
- The **<html>** element is the root element of an HTML page
- The **<head>** element contains meta information about the document
- The **<title>** element specifies a title for the document
- The **<body>** element contains the visible page content
- The **<h1>** element defines a large heading
- The **<p>** element defines a paragraph

HTML Tags

HTML tags are element names surrounded by angle brackets:

<tagname>content goes here...**</tagname>**

- HTML tags normally come **in pairs** like `<p>` and `</p>`
- The first tag in a pair is the **start tag**, the second tag is the **end tag**

This is the Main Heading This text might be an introduction to the rest of the page. And if the page is a long one it might be split up into several sub-headings. This is a Sub-Heading Many long articles have sub-headings so to help you follow the structure of what is being written. There may even be sub-sub-headings (or lower-level headings). Another Sub-Heading Here you can see another sub-heading.	<pre> <html> <body> <h1>This is the Main Heading</h1> <p>This text might be an introduction to the rest of the page. And if the page is a long one it might be split up into several sub-headings.</p> <h2>This is a Sub-Heading</h2> <p>Many long articles have sub-headings so to help you follow the structure of what is being written. There may even be sub-sub-headings (or lower-level headings).</p> <h2>Another Sub-Heading</h2> <p>Here you can see another sub-heading.</p> </body> </html> </pre>
--	--

- The end tag is written like the start tag, but with a **forward slash** inserted before the tag name

HTML Attributes

Attributes provide additional information about HTML elements.

- All HTML elements can have **attributes**
- Attributes provide **additional information** about an element
- Attributes are always specified in **the start tag**
- Attributes usually come in name/value pairs like: **name="value"**

Example:

```
<a href="https://www.w3schools.com">This is a link</a>
```

Similarly

```

```

HTML images are defined with the `<img>` tag. The filename of the image source is specified in the **src attribute**.

HTML Elements

An HTML element is defined by a start tag, some content, and an end tag.

Example:

```
<body>...</body>
```

```
<h1>...</h1>
```

```
<p>...</p>
```

Let's understand that through practical example:

Figure 17: HTML Elements

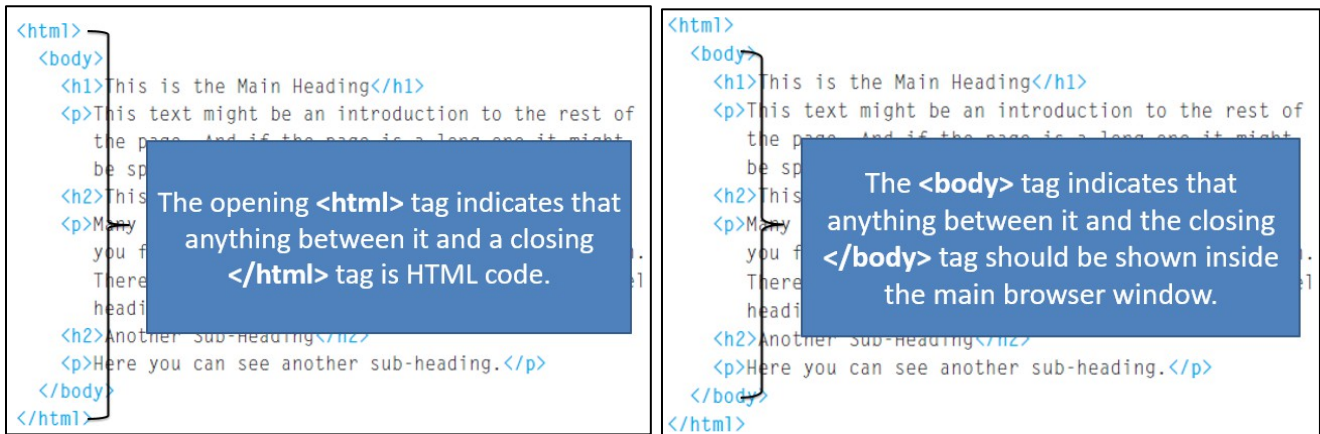


Figure 18: HTML Elements

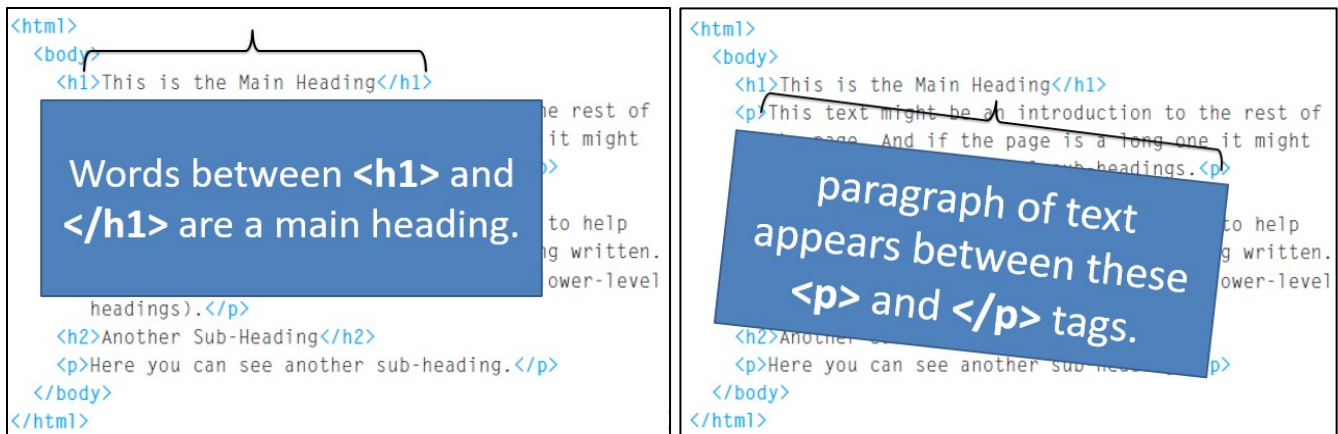


Figure 19: HTML Elements

Attributes

Attributes provide additional information about the contents of an element. They appear on the opening tag of the element and are made up of two parts: a name and a value, separated by an equals sign.



Figure 20: Attributes

For Example:

```
<body>
  <h2 title="inside h2">Web Application Development</h2>
  <h4 title="inside h4 ">Welcome to the course</h4>
  <h4 style="color:blue">Why it appears blue</h4>
</body>
```

Figure 21: Attributes

Output:

Web Application Development

Welcome to the course

Why it appears blue

Figure 22: Attributes

Basic HTML Tags

HTML Headings

HTML has six "levels" of headings. Headings are defined with the **<h1>** to **<h6>** tags.

<h1> defines the most important heading. **<h6>** defines the least important heading.

Code:

```
<html>
<head>
<title>This is the Title of the Page</title>
</head>
<body>
<h1>This is a Main Heading</h1>
<h2>This is a Level 2 Heading</h2>
<h3>This is a Level 3 Heading</h3>
<h4>This is a Level 4 Heading</h4>
<h5>This is a Level 5 Heading</h5>
<h6>This is a Level 6 Heading</h6>
</body>
</html>
```

Output:

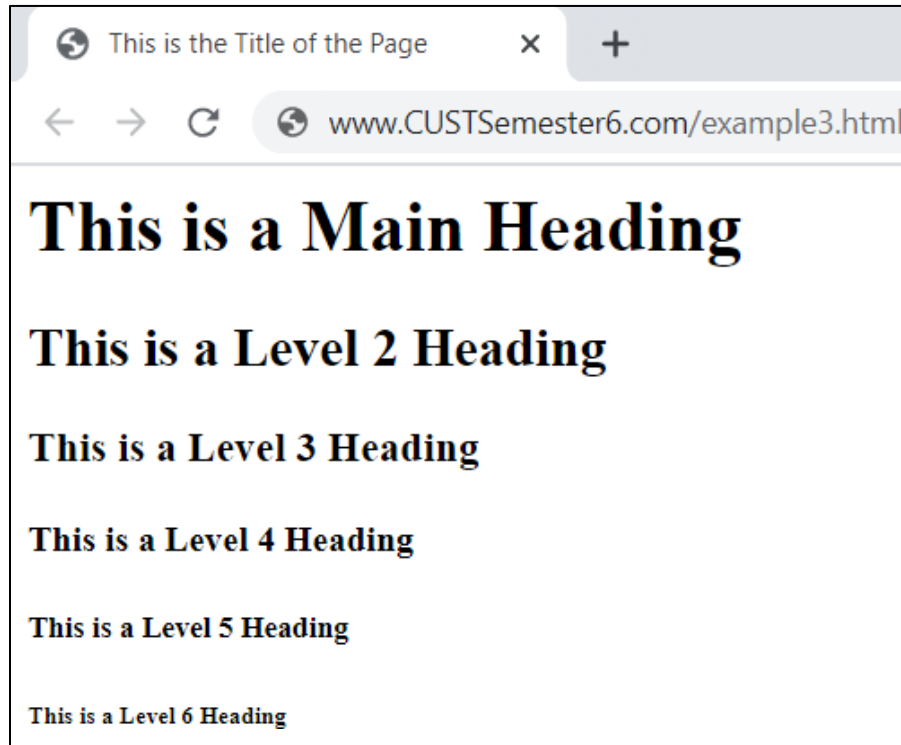


Figure 23: HTML Headings

HTML Paragraphs

A paragraph consists of one or more sentences that form a kind of dialogue. The HTML `<p>` element defines a **paragraph**:

Code:

```
<html>
<head>
<title>This is the Title of the Page</title>
</head>
<body>
<p>this is a paragraph and
when you press enter, does it moves to
new line?</p>
<p>yet another paragraph</p>
</body>
</html>
```

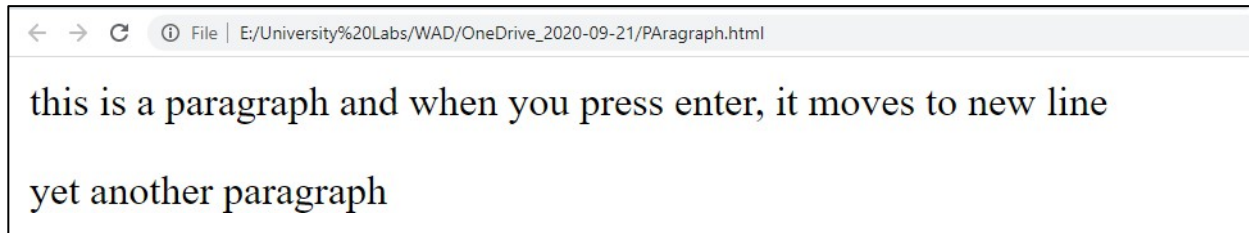
Output:

Figure 24: HTML Paragraphs

HTML Formatting Elements

HTML uses elements like **** and **<i>** for formatting output, like **bold** or *italic* text. Formatting elements were designed to display special types of text:

****- Bold text

****- Important text

<i>- Italic text

****- Emphasized text

<mark>- Marked text

<small>- Small text

****- Deleted text

<ins>- Inserted text

<sub>- Subscript text

<sup>- Superscript text

```

<b> - Bold text</b>
<strong> - Important text</strong>
<i> - Italic text</i>
<em> - Emphasized text</em>
<mark> - Marked text</mark>
<small> - Small text</small>
<del> - Deleted text</del>
<ins> - Inserted text</ins>
<sub> - Subscript text</sub>
<sup> - Superscript text</sup>
  
```

Output:

- **Bold text** - **Important text** - *Italic text* - *Emphasized text* - **Marked text** - Small text - Deleted text - Inserted text - Subscript text - Superscript text

Figure 25

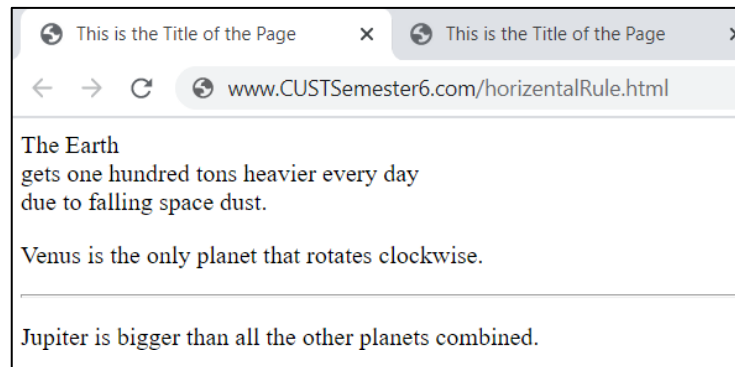
Line Breaks & Horizontal Rules

Code:

```
<html>
<head>
<title>This is the Title of the Page</title>
</head>
<body>
<p>The Earth<br />gets one hundred tons heavier
every day<br />due to falling space dust.</p>

<p>Venus is the only planet that rotates
clockwise.</p>
<hr />
<p>Jupiter is bigger than all the other planets
combined.</p>
</body>
</html>
```

Output:



HTML Quotations

The HTML `<q>` element defines a short quotation.

```
<p>Browsers usually insert quotation marks around the q element.</p>

<p>Mr. Fawad says: <q>You will learn if, you involve yourself in
learning</q></p>
```

Figure 26: HTML Quotations

Output:

```
Browsers usually insert quotation marks around the q element.
Mr. Fawad says: "You will learn if, you involve yourself in learning"
```

Figure 27: Output:

The HTML **<blockquote>** element defines a section that is quoted from another source.

```
<p>Browsers usually indent blockquote elements.</p>

<blockquote cite="https://soeonline.american.edu/">
You can teach a student a lesson for a day, but if you can teach him to
learn by creating curiosity, he will continue the learning process as
long as he lives.
</blockquote>
```

Output:

Browsers usually indent blockquote elements.

You can teach a student a lesson for a day, but if you can teach him to learn by creating curiosity, he will continue the learning process as long as he lives.

HTML Comments

Just like other languages, HTML also provides tags to add comments

You can add comments to your HTML source by using the following syntax:

```
<!-- This is a comment -->
<p>This is a paragraph.</p>
<!-- Comments are not displayed in the browser -->
```

Note: Comments are not displayed by the browser, but they can help document your HTML source code.

HTML Links

HTML links are hyperlinks. You can click on a link and jump to another document. When you move the mouse over a link, the mouse arrow will turn into a little hand.

Note: A link does not have to be text. It can be an image or any other HTML element.

```
<p><a href="https://www.w3schools.com/html/">Visit our HTML tutorial</a></p>
```

Figure 28: HTML Links

Output:

[Visit our HTML tutorial](https://www.w3schools.com/html/)

HTML Images

In HTML, images are defined with the **** tag. The **** tag is empty, it contains attributes only, and does not have a closing tag. The **src** attribute specifies the URL (web address) of the image:

```
<h2>HTML Image</h2>

```

Figure 29: HTML Images

Output:

The alt attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the src attribute, or if the user uses a screen reader).

The value of the **alt** attribute should describe the image. More helpful resource -> https://www.w3schools.com/html/html_images_picture.asp

HTML Style Attribute

Setting the style of an HTML element, can be done with the **style** attribute. The HTML **style** attribute has the following **syntax**:

<tagname style="property:value;">

The **property** is a CSS property. The **value** is a CSS value.

Note: You will learn more about CSS later in this lab, here we will discuss 2-3 examples only.

<pre><body style="background-color:powderblue;"> <h1>This is a heading</h1> <p>This is a paragraph.</p> </body></pre>	This is a heading This is a paragraph.
---	---

The CSS **background-color** property defines the background color for an HTML element. Similarly, The CSS

color property defines the text color for an HTML element:

<pre><h1 style="color:blue;">This is a heading</h1> <p style="color:red;">This is a paragraph.</p></pre>	This is a heading This is a paragraph.
--	---

HTML Table:

An HTML table is defined with the `<table>` tag. Each table row is defined with the `<tr>` tag. A table header is defined with the `<th>` tag. By default, table headings are bold and centered. A table data/cell is defined with the `<td>` tag.

```
<table style="width:100%" border=1>
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
  <th>Age</th>
</tr>
<tr>
  <td>Fawad</td>
  <td>Bin Tariq</td>
  <td>23</td>
</tr>
<tr>
  <td>Ghulam</td>
  <td>Mustafa</td>
  <td>28</td>
</tr>
<tr>
  <td>Muhammad</td>
  <td>Usman</td>
  <td>27</td>
</tr>
</table>
```

Firstname	Lastname	Age
Fawad	Bin Tariq	23
Ghulam	Mustafa	28
Muhammad	Usman	27

Figure 28 Output

Figure 30: HTML Table

HTML Lists

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag. The list items will be marked with bullets (small black circles) by default:

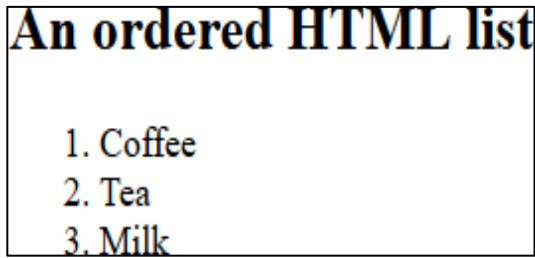
An unordered HTML list

- Coffee
- Tea
- Milk

```
<h2>An unordered HTML list</h2>

<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag. The list items will be marked with numbers by default:



```
<h2>An ordered HTML list</h2>
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

HTML Forms

The HTML **<form>** element defines a form that is used to collect user input:

```
<form>
.
  form elements
.
</form>
```

An HTML form contains **form elements**. Form elements are different types of input elements, like: text fields, checkboxes, radio buttons, submit buttons, and more.

HTML Forms (Text Fields)

<input type="text"> defines a single-line input field for **text input**

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname">
</form>
```

Figure 31: HTML Forms (Text Fields)

This is how it will look like in a browser:

First name:

Last name:

Output:

Notice the use of the **<label>** element in the example above.

The **for** attribute of the **<label>** tag should be equal to the **id** attribute of the **<input>** element to bind them together.

HTML Forms (Radio Buttons)

<input type="radio"> defines a **radio button**. Radio buttons let a user select ONE of a limited number of choices

```
<form>
  <input type="radio" id="male" name="gender" value="male">
  <label for="male">Male</label><br>
  <input type="radio" id="female" name="gender" value="female">
  <label for="female">Female</label><br>
  <input type="radio" id="other" name="gender" value="other">
  <label for="other">Other</label>
</form>
```

Figure 32: HTML Forms (Radio Buttons)

This is how the HTML code above will be displayed in a browser:

☒ Male
☐ Female
☐ Other
HTML Forms (The Submit Button)

<input type="submit"> defines a button for **submitting** the form data to a **form-handler**. The form-handler is specified in the form's **action** attribute.

```

<form action="/next_page">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname"><br><br>
  <input type="submit" value="Submit">|
</form>

```

This is how the HTML code above will be displayed in a browser:

First name:
Last name:
<input type="submit" value="Submit"/>

HTML Forms (Select Element)

The `<select>` element defines a **drop-down list**:

```

<form action="/action_page.php">
  <label for="cars">Choose a car:</label>
  <select id="cars" name="cars">
    <option value="toyota">Toyota</option>
    <option value="honda">Honda</option>
    <option value="suzuki">Suzuki</option>
    <option value="audi">Audi</option>
  </select>
  <input type="submit" value="Submit">
</form>

```

This is how the HTML code above will be displayed in a browser:

Choose a car:

Toyota	▼
--------	---

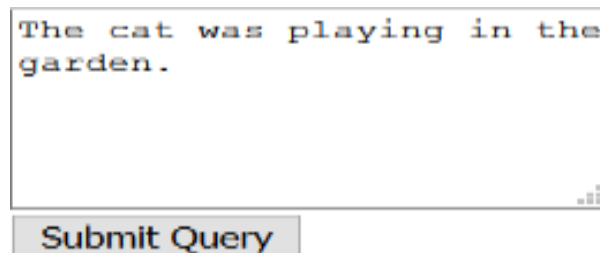
HTML Forms (Text Area)

The `<textarea>` element defines a multi-line input field (**a text area**):

```
<form action="/action_page.php">
  <textarea name="message" style="width:200px; height:200px;">The cat was playing in the
  garden.</textarea>
  <br>
  <input type="submit">
</form>
```

Figure 33: HTML Forms (Text Area)

This is how the HTML code above will be displayed in a browser:



HTML Forms (fieldset and legend)

The `<fieldset>` element is used to group related data in a form. The `<legend>` element defines a caption for the `<fieldset>` element.

```
<form action="/action_page.php">
  <fieldset>
    <legend>Personal Info:</legend>
    <label for="fname">First name:</label><br>
    <input type="text" id="fname" name="fname" placeholder="Fawad"><br>
    <label for="lname">Last name:</label><br>
    <input type="text" id="lname" name="lname" placeholder="Bin Tariq"><br><br>
    <input type="submit" value="Submit">
  </fieldset>
</form>
```

Figure 34: HTML Forms (fieldset and legend)

This is how the HTML code above will be displayed in a browser:

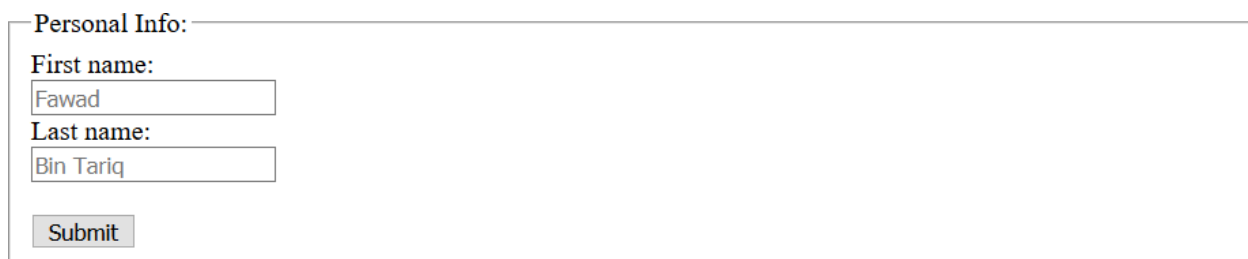


Figure 35: HTML Forms (fieldset and legend)

HTML Forms (Input Types)

Here are the different input types you can use in HTML:

- `<input type="button">`
- `<input type="checkbox">`
- `<input type="color">`
- `<input type="date">`
- `<input type="email">`
- `<input type="file">`
- `<input type="hidden">`
- `<input type="image">`
- `<input type="month">`
- `<input type="number">`
- `<input type="password">`
- `<input type="radio">`
- `<input type="reset">`
- `<input type="search">`
- `<input type="submit">`
- `<input type="tel">`
- `<input type="text">`
- `<input type="time">`
- `<input type="url">`

HTML Forms (Input Attributes)

Value attribute: The input **value** attribute specifies an initial value for an input field.

Readonly attribute: The input **readonly** attribute specifies that an input field is read-only. **Disabled attribute:** The input **disabled** attribute specifies that an input field should be disabled.

Size attribute: The input **size** attribute specifies the visible width, in characters, of an input field. The default value for **size** is 20.

Maxlength attribute: The input **maxlength** attribute specifies the maximum number of characters allowed in an input field.

Min and max attribute: The input **min** and **max** attributes specify the minimum and maximum values for an input field.

Multiple attribute: The input **multiple** attribute specifies that the user is allowed to enter more than one value in an input field.

Pattern attribute: The input **pattern** attribute specifies a regular expression that the input field's value is checked against, when the form is submitted.

Placeholder attribute: The input **placeholder** attribute specifies short a hint that describes the expected value of an input field (a sample value or a short description of the expected format).

Required attribute: The input **required** attribute specifies that an input field must be filled out before submitting the form.

```

<form action="/action_page.php">
  <input type="text" value="Fawad"><br>
  <input type="text" value="Fawad" readonly><br>
  <input type="text" value="Fawad" disabled><br>
  <input type="text" value="Fawad" size="4"><br>
  <input type="text" value="Fawad" maxlength="10"><br>
  <input type="date" min="1995-12-30" max="2020-12-30"><br>
  <input type="file" multiple><br>
  <input type="text" name="country_code" pattern="[A-Za-z]{3}"><br>
  <input type="tel" id="phone" name="phone"
placeholder="0312-1234567"
pattern="[0-9]{4}-[0-9]{7}"><br/>
  <input type="text" name="username" required>
  <input type="submit" value="Submit">
</form>

```

Can be viewed in browser as:

Fawad
Fawad
Fawad
Fawad
Fawad
dd / mm / yyyy
Browse... No files selected.
0312-1234567
Submit

Homework before Lab

This homework will help students to study the concepts of topic before the start of lab.

Problem Solution Modeling

After studying the introduction and concept map sections you should be ready to provide the solution of following problems.

Practices from home

Task-1

Create a following page in html

Seasons

Spring

This can be a really long description of spring, taking up an entire paragraph's worth of space.

There can even be several paragraphs.

Summer

You can also have one or more paragraphs describing summer.

Procedures & Tools

This section provides information about tools and programming procedures used for the lab.

Tools

Students must ensure that Visual studio with web utility is installed on their computer.

Editing, Compiling and Running programs in Visual Studio

[Expected time = 10mins]

You are required to use Microsoft Visual Studio 2019 environment for web application development. You should be able to write html code in visual studio, HTML code in .html files and using C# perform compilation of code. Beside that you are also required to be proficient in performing line by line debugging and break point-based debugging to remove the errors from your code.

Environment Setup

HTML page can be created on multiple text editor including:

- Notepad
- Notepad++
- Sublime
- Visual Studio Code
- Visual Studio
- And much more...

We will be using Visual Studio in this lab:

Step 1: Open visual studio and create new project.

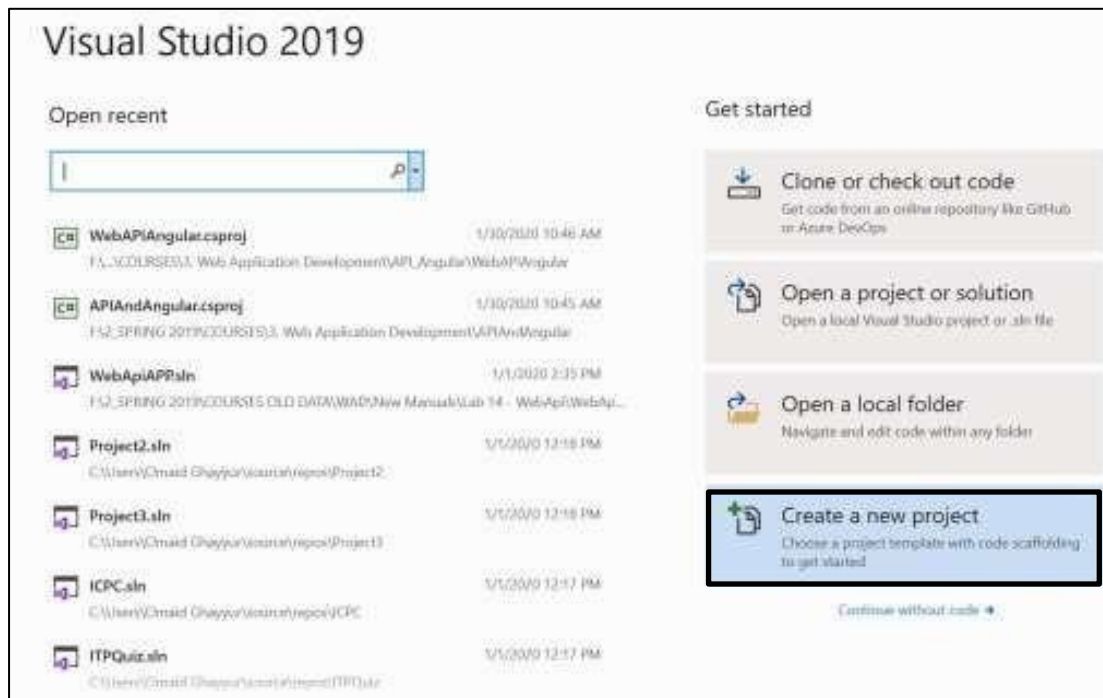


Figure 36: Environment Setup

Step 2: Select ASP.NET Web Application (.Net Framework) using C# language.

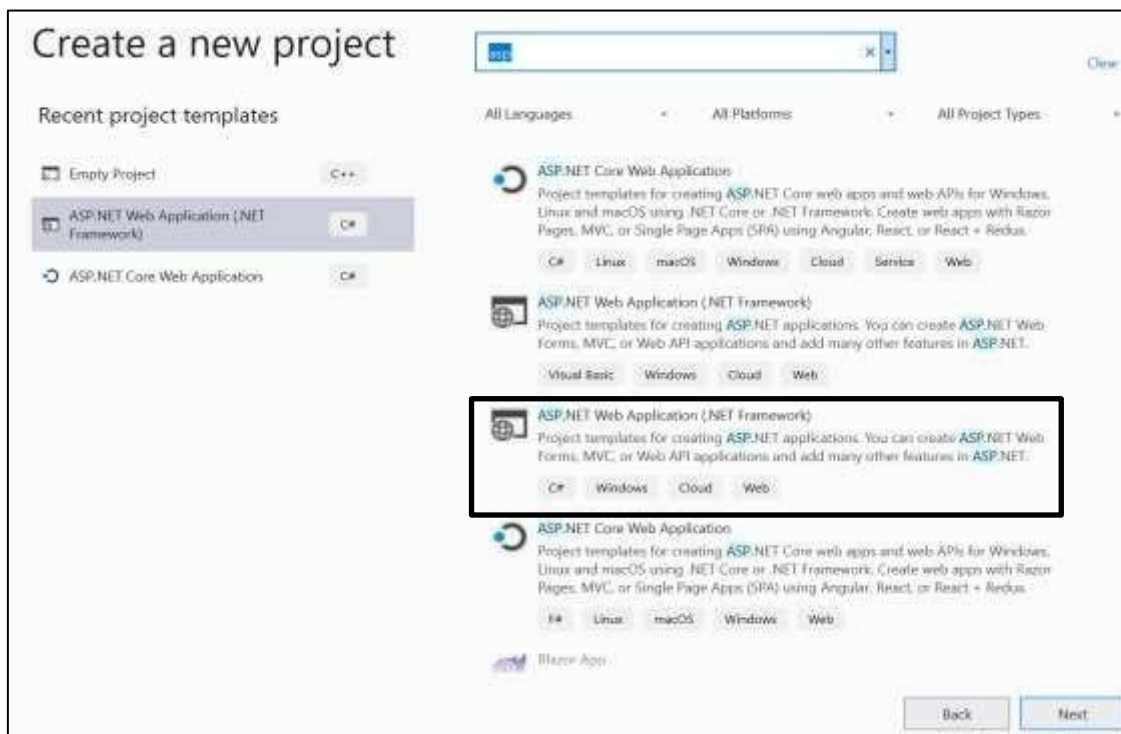


Figure 37: Environment Setup

Give name to the web application:

Configure your new project

ASP.NET Web Application (.NET Framework) Visual Basic Windows Cloud Web

Project name
WebApplication2

Location
C:\Users\Omaid Ghayur\source\repos

Solution
Create new solution

Solution name ⓘ
WebApplication2

☐ Place solution and project in the same directory

Framework
.NET Framework 4.7.2

Back Create

Figure 38: Environment Setup

Select Empty project:

Create a new ASP.NET Web Application

Empty
An empty project template for creating ASP.NET applications. This template does not have any content in it.

Web Forms
A project template for creating ASP.NET Web Forms applications. ASP.NET Web Forms lets you build dynamic websites using a familiar drag-and-drop, event-driven model. A design surface and hundreds of controls and components let you rapidly build sophisticated, powerful UI-driven sites with data access.

MVC
A project template for creating ASP.NET MVC applications. ASP.NET MVC allows you to build applications using the Model-View-Controller architecture. ASP.NET MVC includes many features that enable fast, test-driven development for creating applications that use the latest standards.

Web API
A project template for creating RESTful HTTP services that can reach a broad range of clients including browsers and mobile devices.

Single Page Application
A project template for creating rich client-side JavaScript-driven HTML5 applications using ASP.NET Web API. Single Page Applications provide a rich user experience which includes client-side interactions using HTML5, CSS3, and JavaScript.

Authentication
No Authentication
[Change](#)

Add folders & core references

- ☐ Web Forms
- ☐ MVC
- ☐ Web API

Advanced

- ☒ Configure for HTTPS
- ☐ Docker support
(Requires Docker Desktop)
- ☐ Also create a project for unit tests
[HTML5 static tests](#)

Back Create

Figure 39: Environment Setup

Right click the project and click add new item.

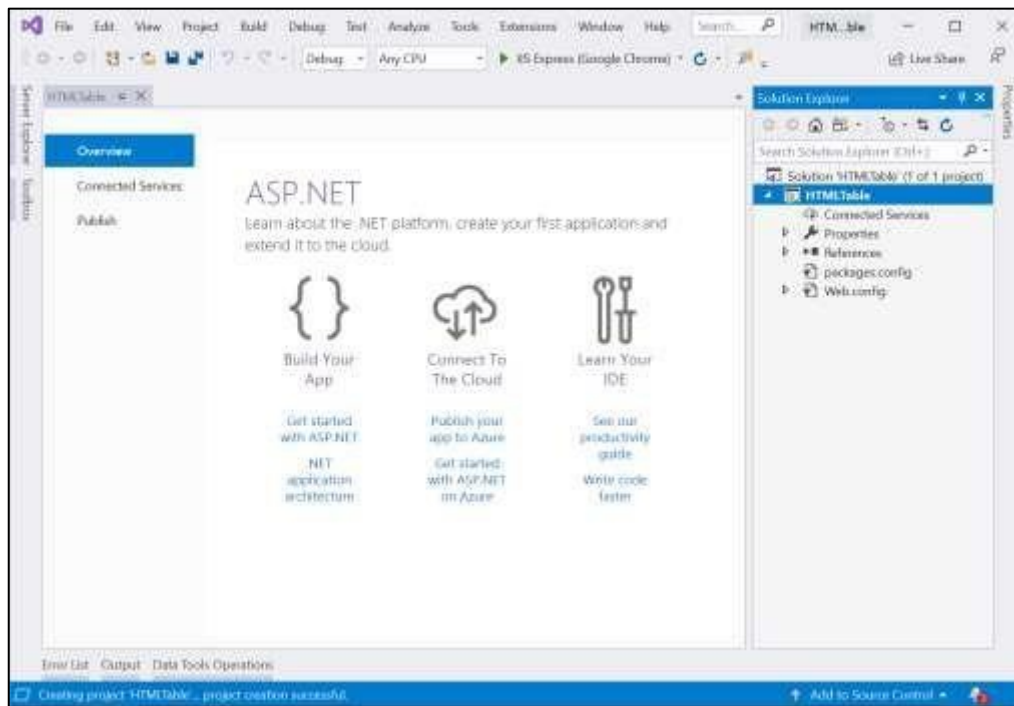


Figure 40: Environment Setup

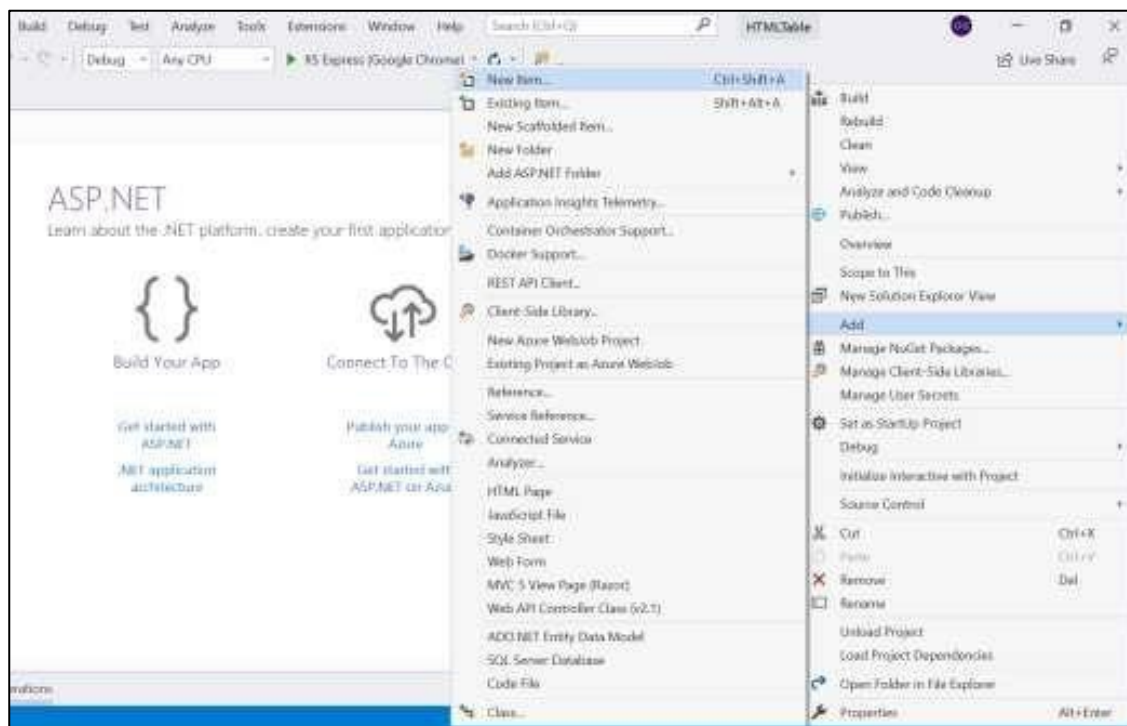


Figure 41: Environment Setup

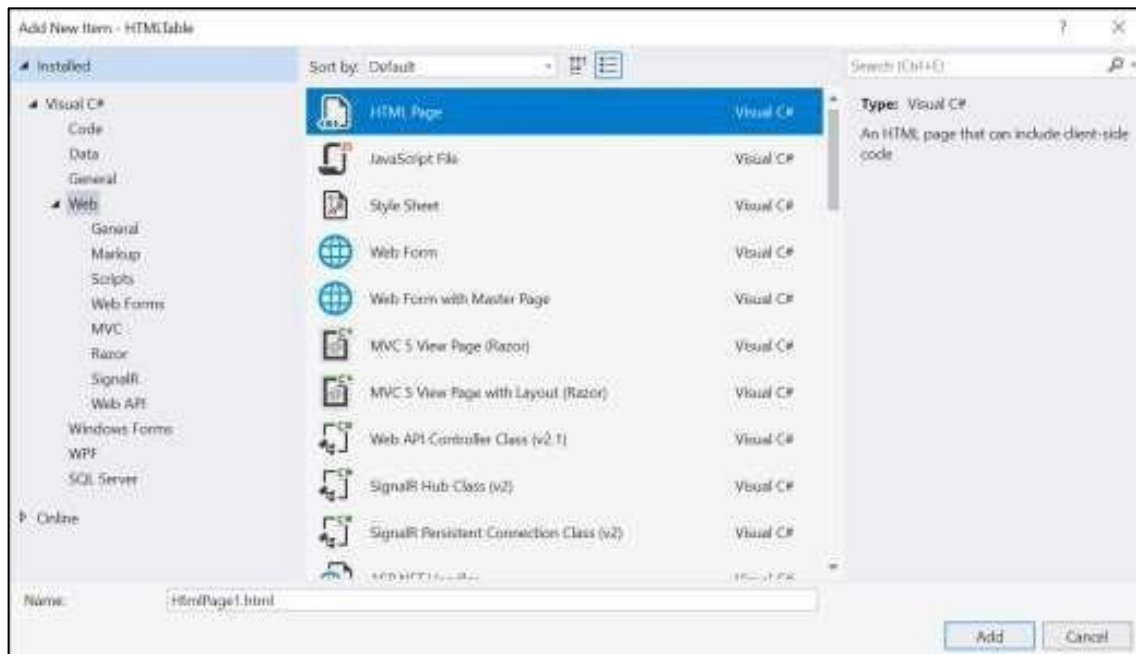


Figure 42: Environment Setup

Walk through Task 1

Let's try to build a web page that structure will look like this:



Figure 43: Walk through Task 1 [Expected time = 40 mins]

- Header section
- Left section
- Central/Main Section
- Right Section
- Footer

For the header section, I have created a box (div) with heading in it.



Figure 44: Walk through Task 1 [Expected time = 40 mins]

Above design can be achieved from the following code:

```
<div style="background-color: #f1f1f1;padding: 20px;text-align: center;">
  <h1>Header</h1>
</div>
```

Figure 45: Walk through Task 1 [Expected time = 40 mins]

For the inner sections, I am going to create a main div, and placing 3 boxes(divs) in it.



Figure 46: Walk through Task 1 [Expected time = 40 mins]

The above design can be achieved from the following lines of code.

```
<div style="height:530px;">
  <div style="float:left;width:20%;background-color: #2196F3;height: 100%;"></div>
  <div style="float:left;width: 60%;background-color: burlywood;height: 100%;"></div>
  <div style="float:left;width:20%;background-color: #4CAF50;height: 100%;"></div>
</div>
```

Figure 47: Walk through Task 1 [Expected time = 40 mins]

And finally the footer section:



Figure 48: Walk through Task 1 [Expected time = 40 mins]

And this can be achieved from the following line:

```
<div style="background-color: darkblue; padding: 20px; text-align: center;color: white;">
  <p>Footer</p>
</div>
```

Figure 49: Walk through Task 1 [Expected time = 40 mins]

Now let's look at the whole design with code:



Figure 50: Walk through Task 1 [Expected time = 40 mins]

And the code looks like:

```
<!DOCTYPE html>
<html>
<head>
  <title>Contact Us</title>
</head>
<body>
  <div style="background-color: #f1f1f1;padding: 20px;text-align: center;">
    <h1>Header</h1>
  </div>
  <div style="height:530px;">
    <div style="float:left;width:20%;background-color: #2196F3;height: 100%;"></div>
    <div style="float:left;width: 60%;background-color: burlywood;height: 100%;"></div>
    <div style="float:left;width:20%;background-color: #4CAF50;height: 100%;"></div>
  </div>
  <div style="background-color: darkblue; padding: 20px; text-align: center;color: white;">
    <p>Footer</p>
  </div>
</body>
</html>
```

Figure 51: Walk through Task 1 [Expected time = 40 mins]

Practice Tasks

This section will provide information about the practice tasks which are required to be performed in lab session. Design solutions of problems discussed in each task and place solution code in a folder specified by your lab instructor.

Practice Task 01 [Expected Time 10 mins]

Implement the following task as shown in the picture.



Practice Task 02 [Expected Time 30 mins]

Design a simple form that consists of following fields:

1. 2 Input fields for your “First Name” & “Last Name”.
2. Read only field that contains your city and make it required.
3. Input field for zip code and restrict field length to 4.
4. Field for taking your date of birth
5. Input field with type “submit”

Practice Task 03 [Expected Time 20 mins]

Design html table with same data as shown below:

Table 3: Evaluation of the Lab 1

Sr. No.	Task No	Description	Marks
1	1	Problem Modeling	20
2	2	Procedures and Tools	10
3	3	Practice tasks and Testing	35

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The HTML application/page must run without errors and produce the required output according to the task requirements.	30%	The HTML page will be opened in a web browser and the output will be checked against the given requirements.
Correct Use of HTML Structure and Tags	HTML elements must be structured correctly using appropriate tags such as headings, paragraphs, links, images, lists, tables, and other required elements.	25%	Review of the HTML source code and browser output to verify correct document structure and use of HTML tags.
Correct Use of Attributes and Forms	HTML attributes must be applied correctly, and forms must contain the required controls and input elements.	20%	The source code and browser output will be inspected to verify correct attributes and form elements.
Code Quality	HTML code should be clean, consistently indented, logically organized, readable, and free of unnecessary code.	15%	Review of code formatting, indentation, structure, and organization.
Presentation	The completed HTML page should be clearly demonstrated, properly organized, and labelled with the student's name where required.	10%	Review of the submitted webpage and demonstration of the final browser output.

Lab 03: Cascading Style Sheets (CSS)

Introduction

Cascading Style Sheets (CSS) is a language that describes the style of an HTML document. CSS describes how HTML elements should be displayed.

Example:

You can specify that the background of the page is cream, all paragraphs should appear in gray using the Arial typeface, or that all level one headings should be in a blue, italic, Times typeface.

CSS Rule

A CSS rule set consists of a selector and a declaration block:

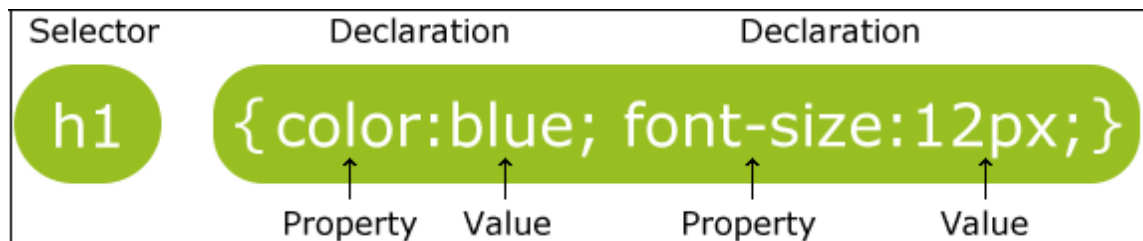


Figure 52: CSS Rule

Example:



Figure 53: Example:

Selectors indicate which **element** (in this case p is an element) the rule applies to. The same rule can apply to more than one element if you separate the element names with commas.

Declarations indicate how the elements referred to in the selector should be styled. In a rule, there can be more than 1 element associated with declaration.

Example **h1, h2, h3** in following screen are associated with declaration.

For Example:



Figure 54: For Example:

Properties

Properties indicate the aspects of the element you want to change. For example, color, font, width, height and border.



Figure 55: Properties

Values specify the settings you want to use for the chosen **properties**. For example, if you want to specify a **color** property then the **value** is the color you want the text in these elements to be as shown in figure above.

CSS Example

```

<!DOCTYPE html>
<html>
<body>
  <h1 style="color:blue;text-align:center;">
    This is a heading
  </h1>
  <p style="color:red;">This is a paragraph.</p>
</body>
</html>

```

Figure 56: CSS Example

Output:

This is a heading

This is a paragraph.

Relevant Lecture Readings

- a) *Revise Lab No. 2 available at \\fs\lectures\$ folder.*
- b) *From web: <https://www.w3schools.com/css/default.asp>*

Activity Time boxing

Table 1: Activity Time Boxing

Task No.	Activity Name	Activity time	Total Time
Task 1	Concept Map	60 minutes	60 minutes
Task 2	Understanding and Creation of project	10 minutes	10 minutes
Task 3	Walk Through Task	40 minutes	40 minutes
Task 4	Practice tasks	60 minutes for all Tasks	60 minutes

Objectives of the experiment

- To get a basic understanding of CSS.
- To get to know how to design basic Web Page.

Concept Map [60 mins]

This concept map will help students to understand the main concepts of topic covered in lab.

Environment Setup

HTML/CSS pages can be created on multiple text editor including:

- Notepad
- Notepad++
- Sublime
- Visual Studio Code
- Visual Studio
- And much more...

We will be using Visual Studio in this lab.

Following are sequence of steps to be followed in order to create project in visual studio 2019.

Step 1: Open visual studio and create new project.

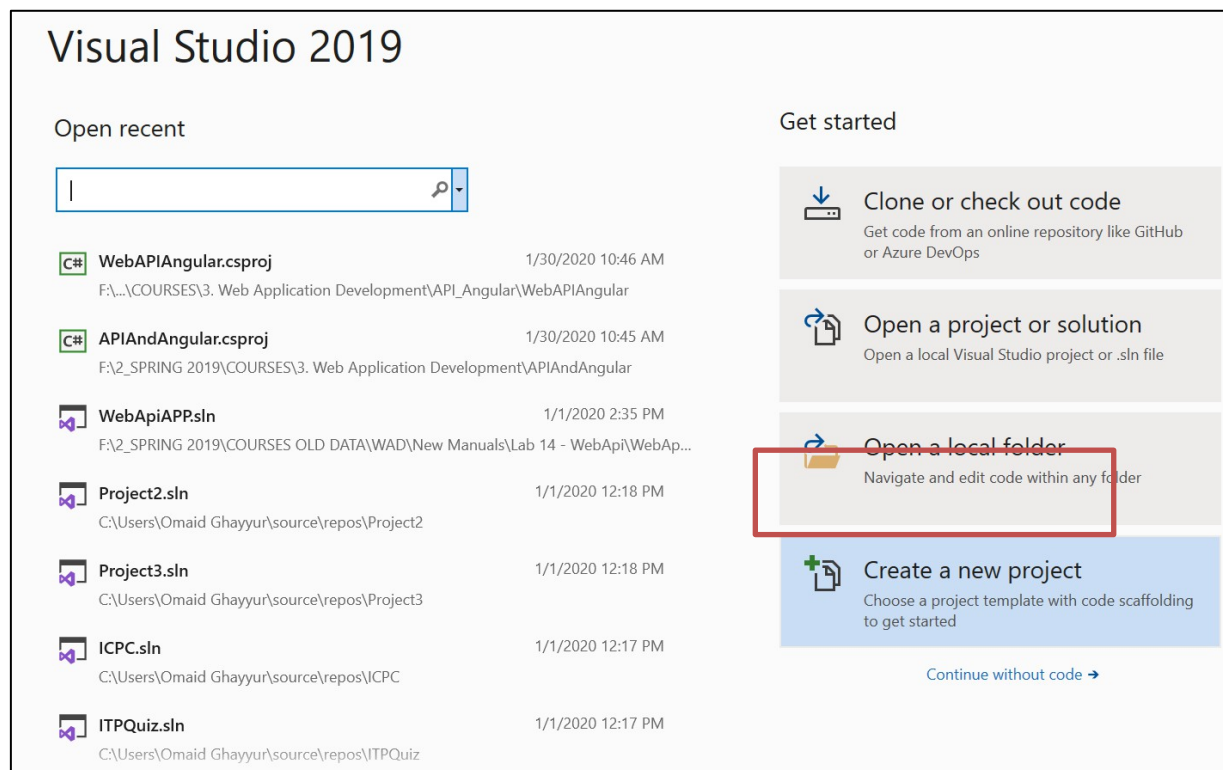


Figure 57: Environment Setup

Step 2: Select ASP.NET Web Application (.Net Framework) using C# language.

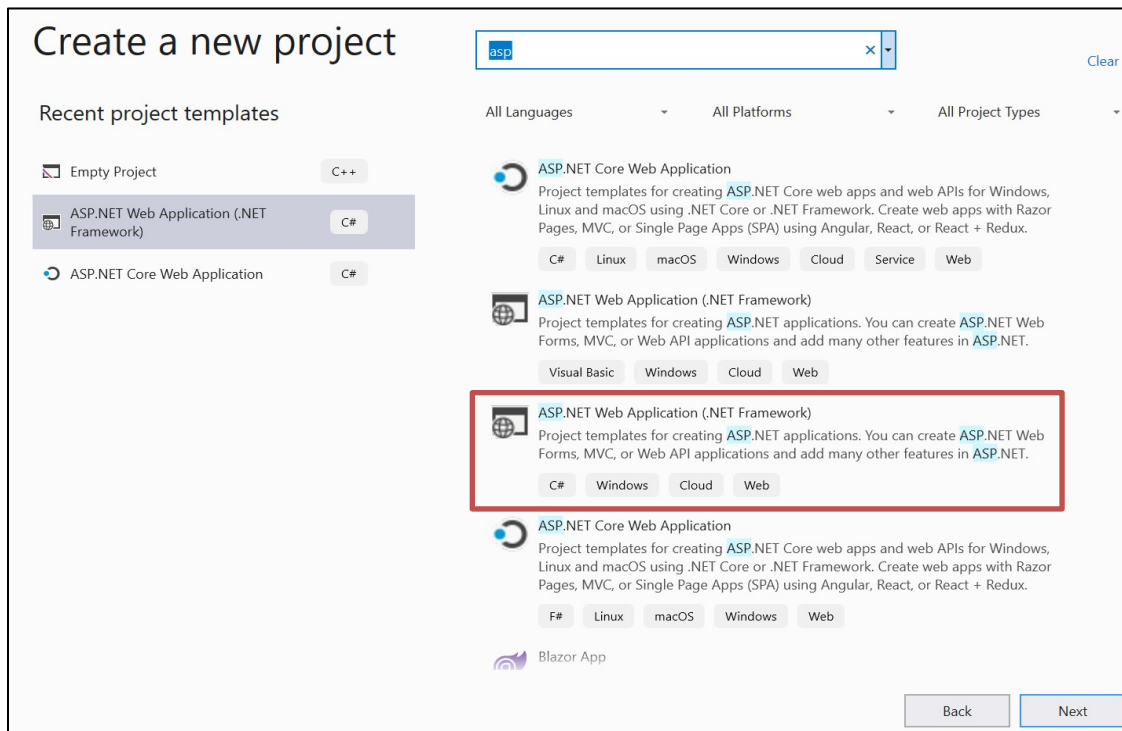


Figure 58: Environment Setup

Give name to the web application:

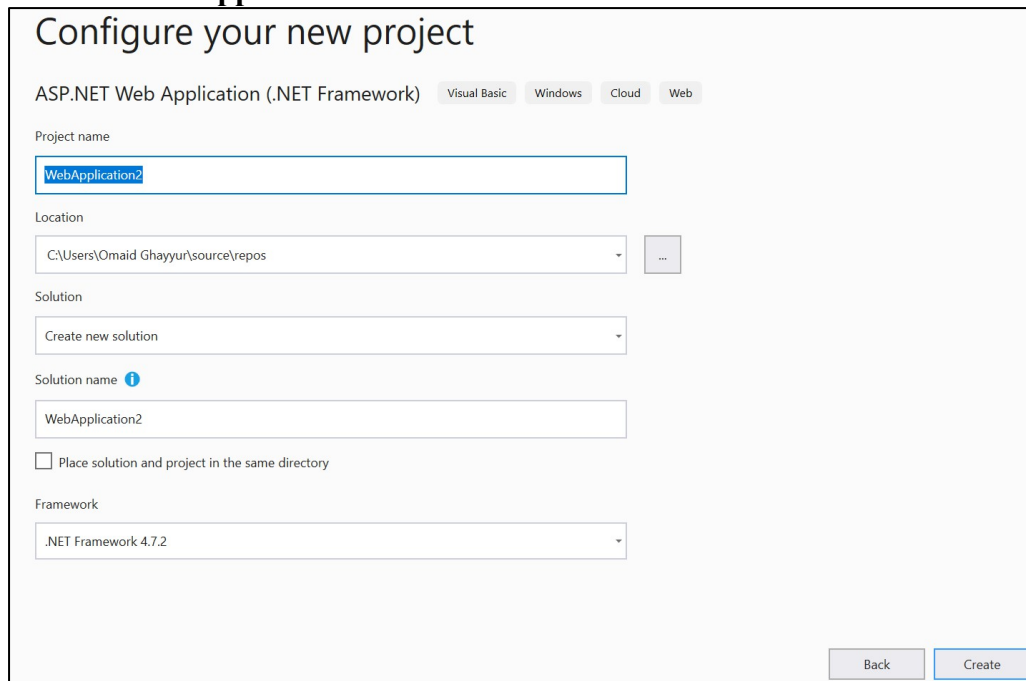
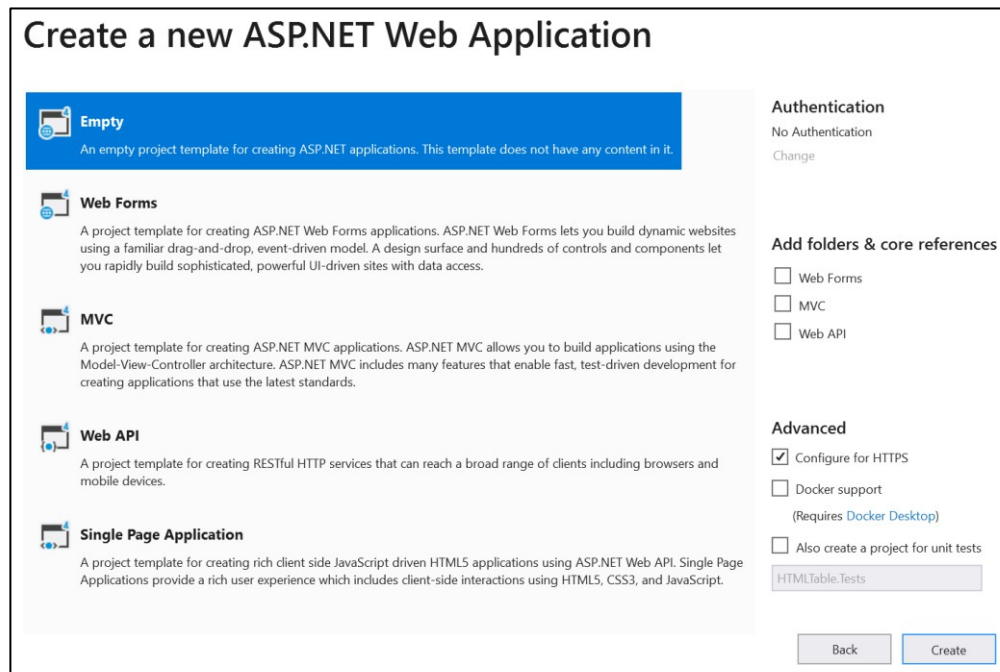


Figure 59: Environment Setup

Select Empty project:



Right click the project and click add new item.

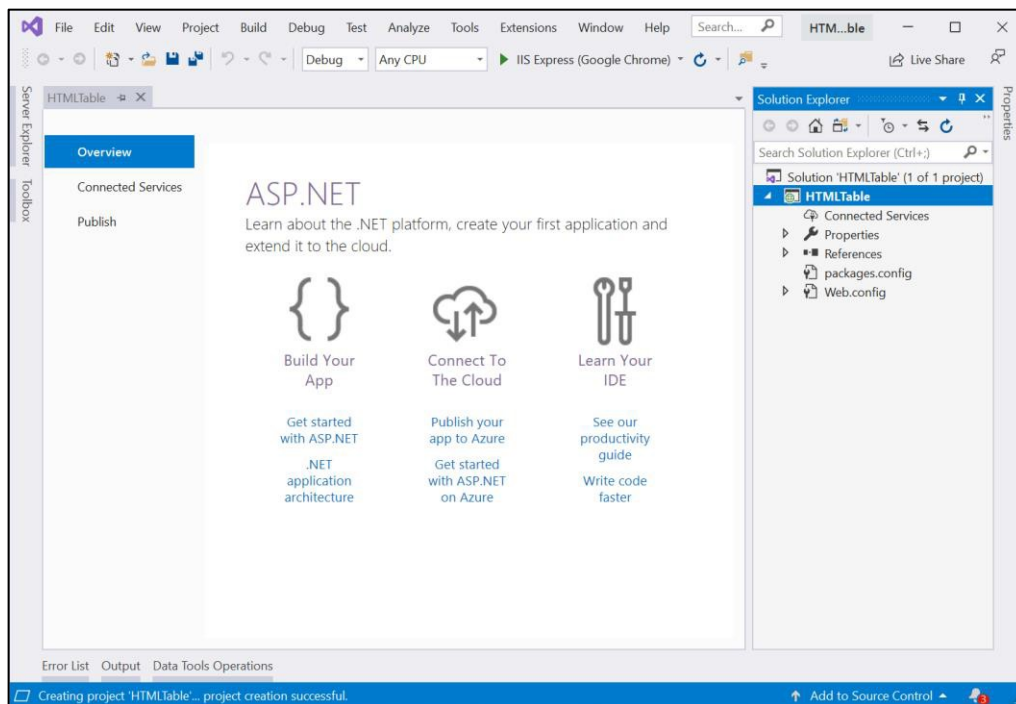
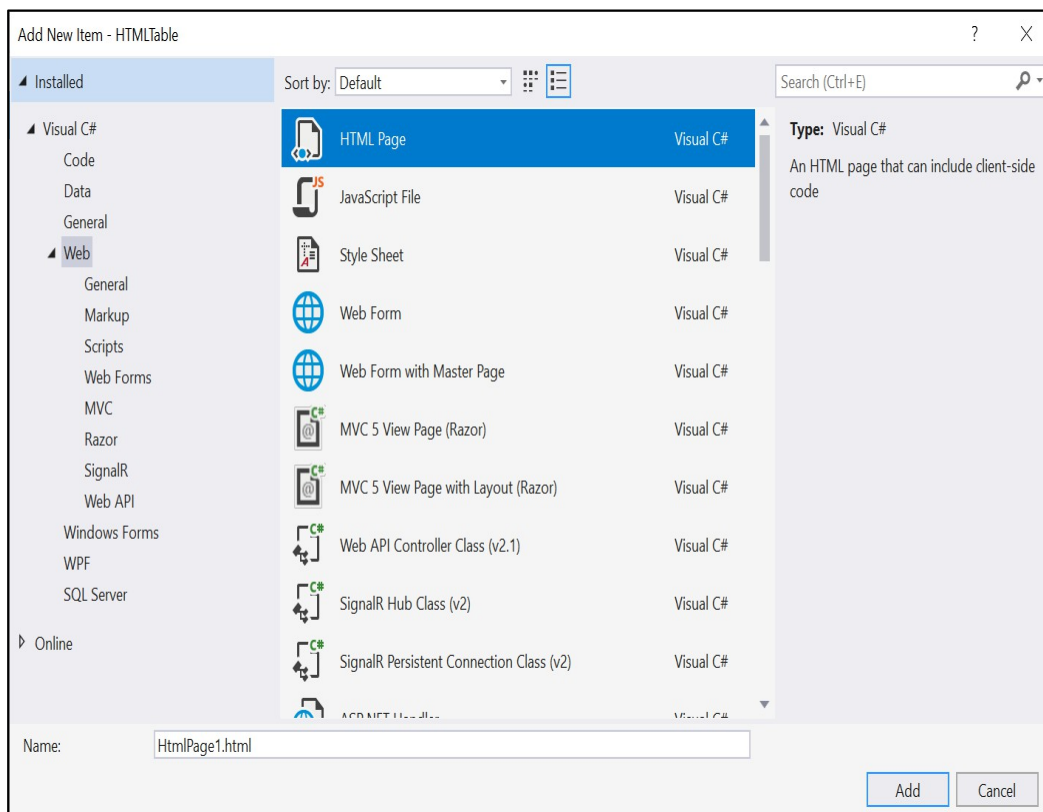
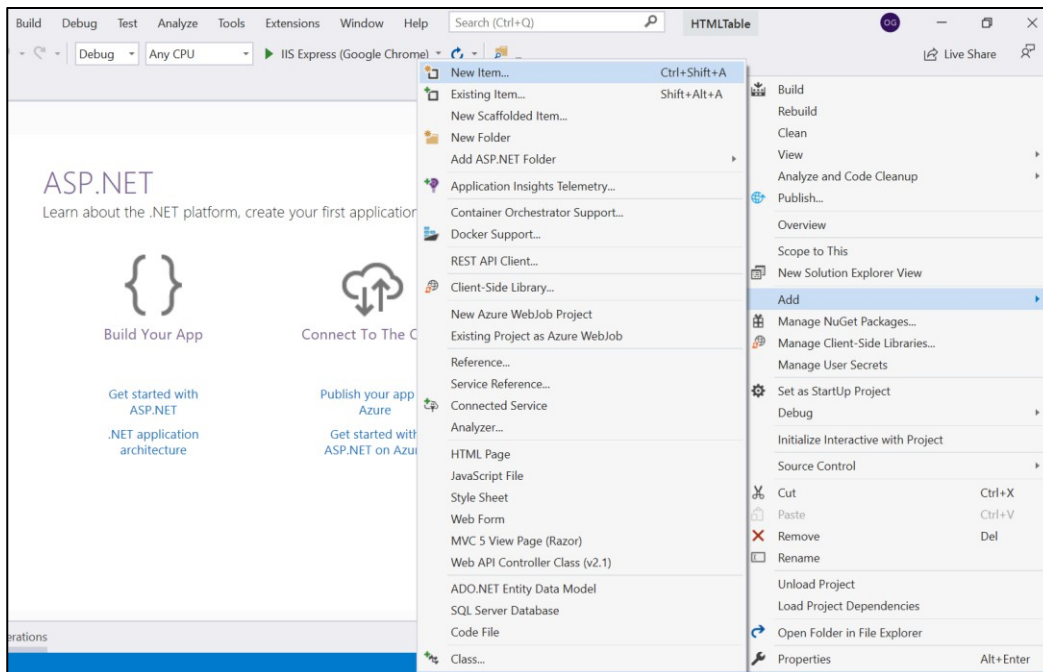


Figure 60: Environment Setup



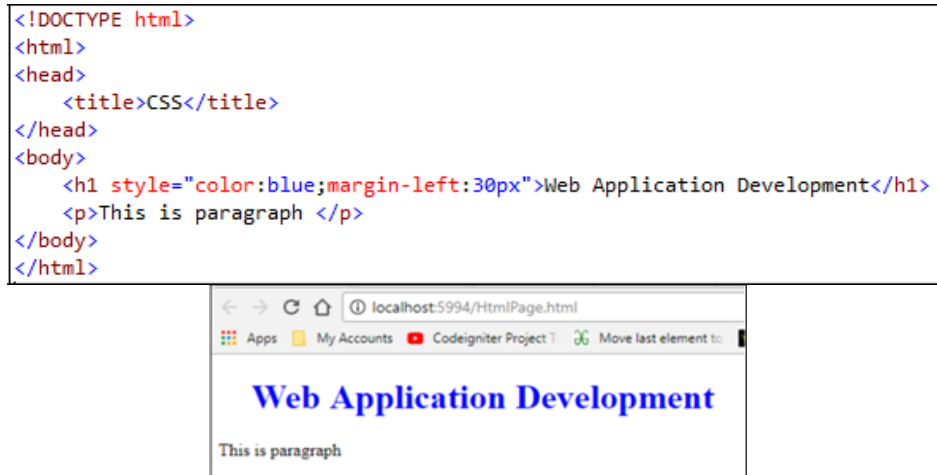
How to Insert CSS

There are three ways of inserting a style sheet:

- Inline style
- External style sheet
- Internal style sheet

Inline Styles

An inline style may be used to apply a unique style for a single element.



Internal Style

An internal style sheet may be used if one single page has a unique style. Internal styles are defined within the <style> element, inside the head section of an HTML page. Example is shown in figure below

```
<html>
  <head>
    <title>CSS</title>
    <style>
      body{
        background-color:linen;
      }

      h1{
        color:maroon;
        margin-left:40px;
      }
    </style>
  </head>
  <body>
    <h1>Web Application Development</h1>
    <p>This is paragraph</p>
  </body>
</html>
```

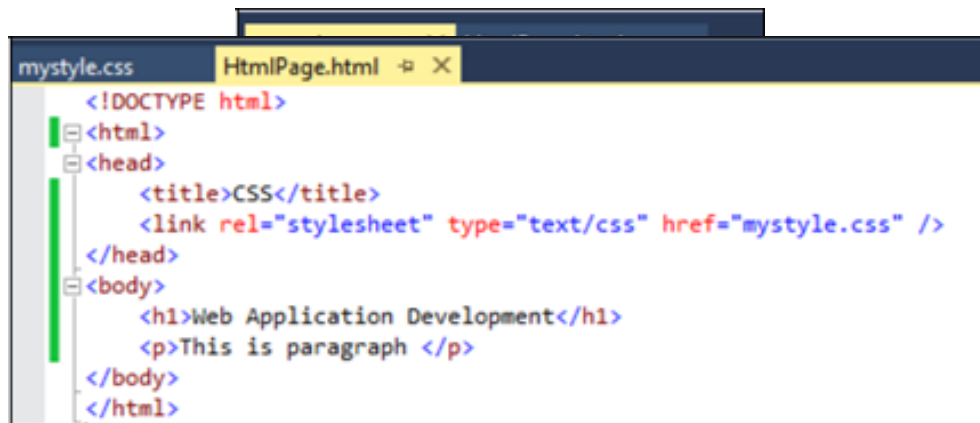
Output:**External Style**

With an external style sheet, you can change the look of an entire website by changing just one file. Each page must include a reference to the external style sheet file inside the `<link>` element. The `<link>` element goes inside the head section. An external style sheet can be written in any text editor. The file should not contain any html tags. The style sheet file must be saved with a `.css` extension.

You need to refer this file in your html document.

Example:

```
<link href="mystyle.css" type="text/css" rel="stylesheet" />
```





CSS Selectors

CSS selectors allow you to select and manipulate HTML elements. CSS selectors are used to "find" (or select) HTML elements based on their id, class, type, attribute, and more.

Simple Selectors

The element selector selects elements based on the element name, id, class. Example is shown below:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS</title>
  <style>
    p {
      text-align: center;
      color: red;
    }
  </style>
</head>
<body>
  <p>This is paragraph one. </p>
</body>
</html>
```

Output:

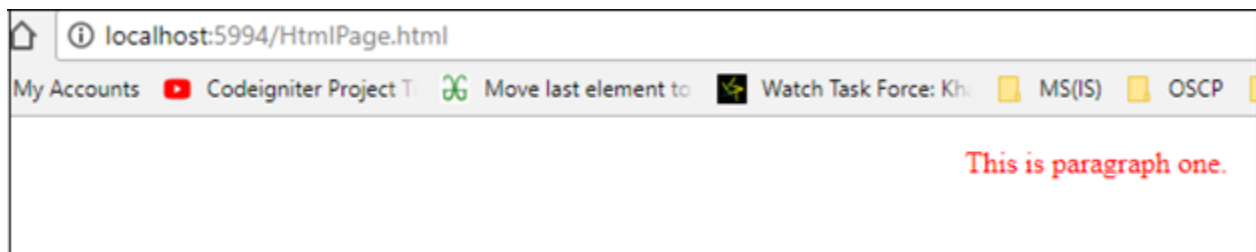


Figure 61: Output:

The Id Selector (#)

The id selector uses the id attribute of an HTML element to select a specific element. The id of an element is unique within a page, so the id selector is used to select one unique element. To select an element with a specific id, write a hash (#) character, followed by

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS</title>
  <style>
    #para2 {
      text-align: center;
      color: red;
    }
  </style>
</head>
<body>
  <p>This is paragraph one. </p>
  <p id="para2">This is paragraph two. </p>
</body>
</html>
```

the id of the element.

Output:

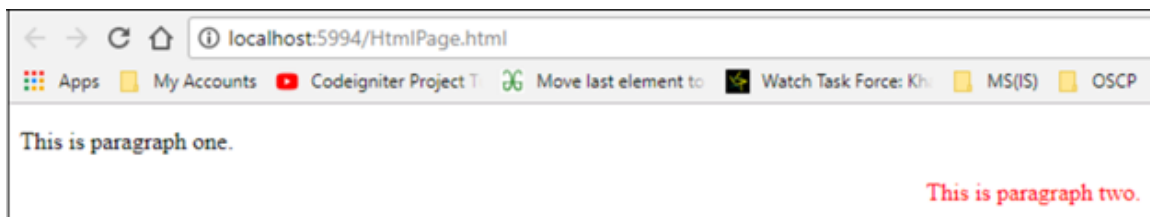


Figure 62: Output:

The Class Selector (.)

The class selector selects elements with a specific class attribute. To select elements with a specific class, write a period character, followed by the name of the class.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .center {
      text-align: center;
      color: red;
    }
  </style>
</head>
<body>
  <h1 class="center">Red and center-aligned heading</h1>
  <p class="center">Red and center-aligned paragraph.</p>
</body>
</html>
```

Output:



Figure 63: Output

Grouping Selectors

If you have elements with the same style definitions, you can group them together.

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS</title>
  <style>
    h1, p {
      text-align: center;
      color: red;
    }
  </style>
</head>
<body>
  <h1>Web Application Development</h1>
  <p>This is paragraph </p>
</body>
</html>
```



Figure 64: Grouping Selectors

Universal Selector (*)

The universal selector (*) selects all HTML elements on the page.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    * {
      text-align: center;
      color: blue;
    }
  </style>
</head>
<body>
  <h1>Hello world!</h1>
  <p>Every element on the page will be affected by the style.</p>
  <p id="para1">Me too!</p>
  <p>And me!</p>
</body>
</html>
```

Output:



Figure 65: Universal Selector (*)

Combinators Selector

Select elements based on a specific relationship between them.

Example:

- descendant selector (space)
- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)

Descendant Selector:

The descendant selector matches all elements that are descendants of a specified element.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div p {
  background-color: yellow;
}
</style>
</head>
<body>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>

<p>Paragraph 4. Not in a div.</p>
<p>Paragraph 5. Not in a div.</p>

</body>
</html>
```

Figure 66: Example:

Output:

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div.

Paragraph 4. Not in a div.

Paragraph 5. Not in a div.

Child selector:

The child selector selects all elements that are the children of a specified element.

```
<!DOCTYPE html>
<html>
<head>
<style>
div > p {
    background-color: yellow;
}
</style>
</head>
<body>

<div>
    <p>Paragraph 1 in the div.</p>
    <p>Paragraph 2 in the div.</p>
    <section><p>Paragraph 3 in the div.</p></section>
    <!-- not Child but Descendant -->
    <p>Paragraph 4 in the div.</p>
</div>

<p>Paragraph 5. Not in a div.</p>
<p>Paragraph 6. Not in a div.</p>

</body>
</html>
```

Figure 67: Child selector:

Output:

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div.

Paragraph 4 in the div.

Paragraph 5. Not in a div.

Paragraph 6. Not in a div.

Adjacent Sibling Selector:

- The adjacent sibling selector selects all elements that are the adjacent siblings of a specified element.
- Sibling elements must have the same parent element, and "adjacent" means "immediately following".

```
<!DOCTYPE html>
<html>
<head>
<style>
div + p {
  background-color: yellow;
}
</style>
</head>
<body>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
</div>

<p>Paragraph 3. Not in a div.</p>
<p>Paragraph 4. Not in a div.</p>

</body>
</html>
```

Figure 68: Adjacent Sibling Selector:

Output:

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3. Not in a div.

Paragraph 4. Not in a div.

General Sibling Selector:

The general sibling selector selects all elements that are siblings of a specified element.

```
<head>
<style>
div ~ p {
    background-color: yellow;
}
</style>
</head>
<body>

<p>Paragraph 1.</p>

<div>
    <p>Paragraph 2.</p>
</div>

<p>Paragraph 3.</p>
<code>Some code.</code>
<p>Paragraph 4.</p>

</body>
```

Output:

```
Paragraph 1.
Paragraph 2.
Paragraph 3.
Some code.
Paragraph 4.
```

Summary of Combinator Selector

Selector	Example	Example description
<i>element element</i>	div p	Selects all <p> elements inside <div> elements
<i>element>element</i>	div > p	Selects all <p> elements where the parent is a <div> element
<i>element+element</i>	div + p	Selects all <p> elements that are placed immediately after <div> elements
<i>element1~element2</i>	p ~ ul	Selects every element that are preceded by a <p> element

Pseudo-class selectors

Select elements based on a certain state. A pseudo-class is used to define a special state of an element.

For example, it can be used to:

- Style an element when a mouse hover over it
- Style visited and unvisited links differently
- Style an element when it gets focus

Syntax:

```
selector : pseudo-class {
  property:value;
}
```

Anchor Pseudo-classes

- **a:link:** Describes properties associated with hyperlink
- **a:visited:** Color of hyperlink once it is visited
- **a:hover:** Color of hyperlink once user hovers over it
- **a:active:** Color of hyperlink once clicked

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* unvisited link */
  a:link {
    color: red;
  }

  /* visited link */
  a:visited {
    color: green;
  }

  /* mouse over link */
  a:hover {
    color: hotpink;
  }

  /* selected link */
  a:active {
    color: blue;
  }
</style>
</head>
<body>
  <p><b><a href="home.html" target="_blank">This is a link</a></b></p>
</body>
</html>
```

Figure 69

Example (Tooltip Hover):

```

<!DOCTYPE html>
<html>
<head>
<style>
p {
  display: none;
  background-color: yellow;
  padding: 20px;
}

div:hover p {
  display: block;
}
</style>
</head>
<body>

<div>Hover over me to show the p element
  <p>Here I am!</p>
</div>

</body>
</html>

```

Figure 70: Example (Tooltip Hover):

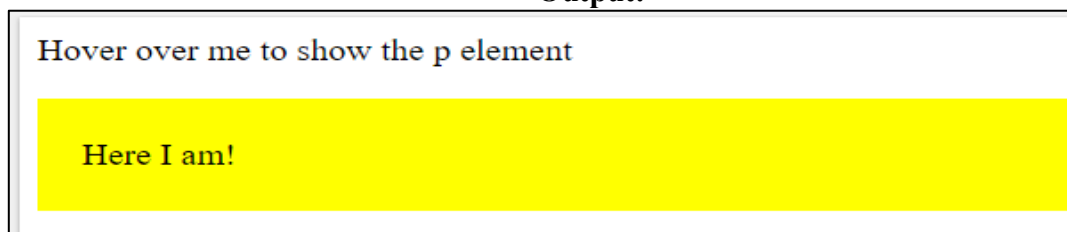
Output:

Figure 71: Output:

Rules for Applying CSS

If there are two or more rules that apply to the same element, it is important to understand which will take precedence.

Last Rule:

If the two selectors are identical, the latter of the two will take precedence. Here you can see the second `i` selector takes precedence over the first.

Example:

```
<html>
  <head>
    <title>CSS</title>
    <style>
      h1{
        color:green;
      }
      h1{
        color:blue;
      }
    </style>
  </head>
  <body>
    <h1>Web Application Development</h1>
  </body>
</html>
```

Figure 72: Example:

Output:

Figure 73: Output

Specificity:

If one selector is more specific than the others, the more specific rule will take precedence over more general ones.

Example:

```
<html>
  <head>
    <title>CSS</title>
    <style>
      *{
        font-size:20px;
      }
      h1{
        font-size:30px;
      }
      h1.heading1{
        font-size:40px;
      }
      p{
        color:green;
      }
      p#firstpara{
        color:red;
      }
    </style>
  </head>
  <body>
    <h1>Web Application Development</h1>
    <h1 class="heading1">Web Application Development</h1>
    <p>Hello World</p>
    <p id="firstpara">First Para</p>
  </body>
</html>
```

Figure 74: Example:

Output:

Figure 75: Output:

Important:

You can add !important after any property value to indicate that it should be considered more important than other rules that apply to the same element.

```
<html>
  <head>
    <title>CSS</title>
    <style>
      *{
        color:red;
      }
      h1.heading1{
        color:yellow;
      }
      h1{
        color:green !important;
      }
    </style>
  </head>
  <body>
    <h1>Web Application Development</h1>
    <h1 class="heading1">Web Application Development</h1>
    <p>Hello World</p>
  </body>
</html>
```

Figure 76: Important:

**CSS Examples****Borders:**

The CSS **border** properties allow you to specify the style, width, and color of an element's border. The **border-style** property specifies what kind of border to display.

```

<html>
<head>
<style>
p.one {
  border-style: solid;
  border-width: 5px;
}

p.two {
  border-style: dotted;
  border-width: thick;
}
</style>
</head>
<body>
  <p class="one"> border with solid lines</p>
  <p class="two"> border with dotted lines</p>
</body>
</html>

```

Figure 77: Borders:

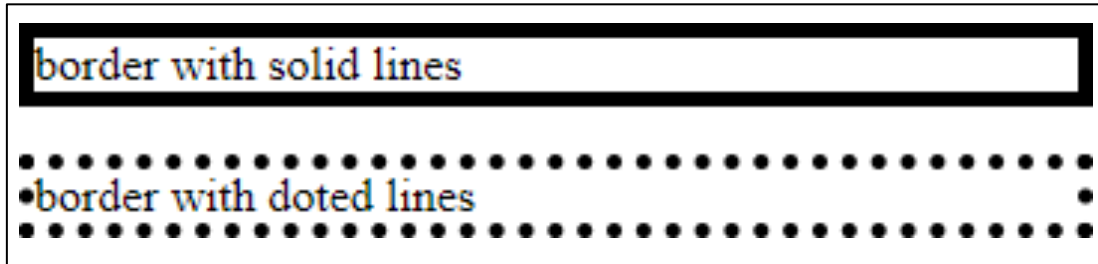


Figure 78: Borders:

List:

HTML lists allow web developers to group a set of related items in lists. List are of two types:

Unordered List

An unordered list starts with the tag. Each list item starts with the tag.

Ordered List

An ordered list starts with the tag. Each list item starts with the tag.

```
<html>
  <head>
    <title>CSS</title>
    <style>
      ol{
        list-style-type: lower-roman;
      }
    </style>
  </head>
  <body>
    <h1>The Complete Poem</h1>
    <h2>Emily Dickinson</h2>
    <ol>
      <li>Life</li>
      <li>Nature</li>
      <li>Love</li>
      <li>Time and Eternity</li>
      <li>The Single Hound</li>
    </ol>
  </body>
</html>
```

Figure 79: Ordered List

Output:

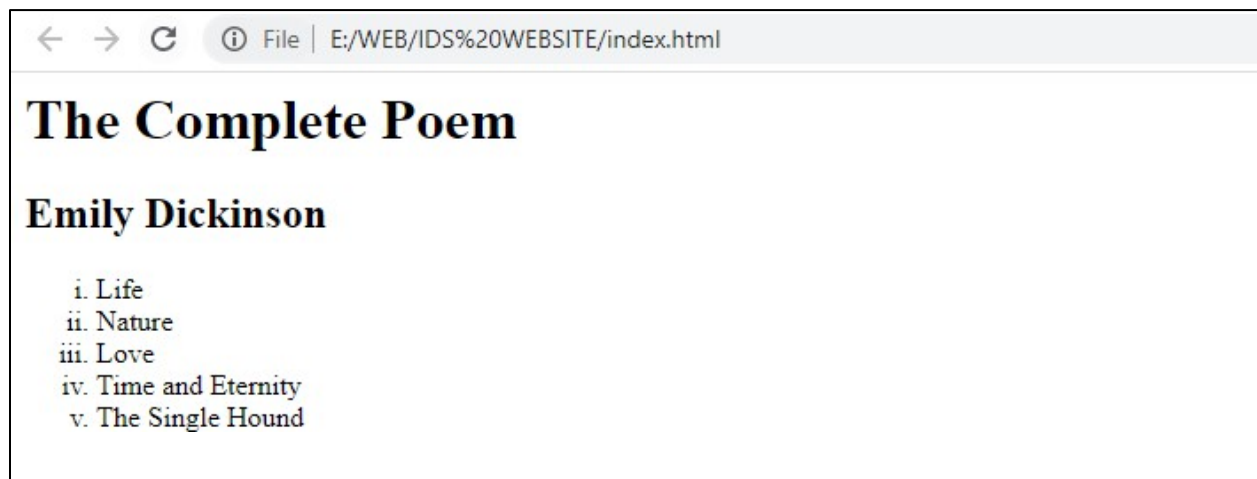


Figure 80: Output:

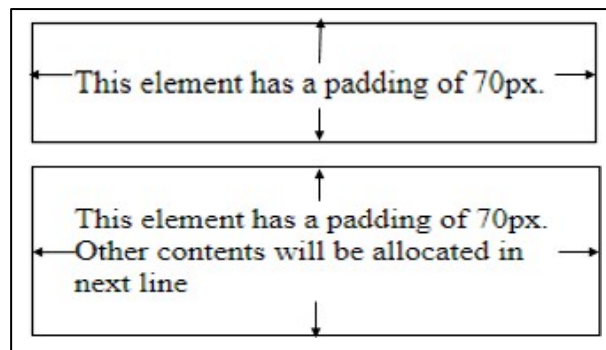
Padding:

Figure 81: Padding:

Output:

```

<html>
<head>
<style>
div {
  padding: 20px;
  border: 1px solid;
}
</style>
</head>
<body>

<div>This element has a padding of 70px. Other contents will
be allocated in next line</div>

</body>
</html>

```

This element has a padding of 70px.
Other contents will be allocated in
next line

Layout Elements:

- <header>	Defines a header for a document or a section
- <nav>	Defines a container for navigation links
- <section>	Defines a section in a document
- <article>	Defines an independent self-contained article
- <aside>	Defines content aside from the content (like a sidebar)
- <footer>	Defines a footer for a document or a section
- <details>	Defines additional details
- <summary>	Defines a heading for the <details> element

Figure 82: Layout Elements

For Example:

```

</head>
<body>
  <header>
    <h2>Header of the page</h2>
  </header>
  <nav>
    <ul>
      <li><a href="#">Link 01</a>
      <li><a href="#">Link 02</a>
      <li><a href="#">Link 03</a>
    </ul>
  </nav>
  <article>
    <h1>page contents</h1>
  </article>
  <footer>
    <h1> Page Footer</h1>
  </footer>
</body>
</html>

```

```

<html>
<head>
<style>
  *body {
    font-family:Arial;
  }
  header {
    background-color:green;
    padding:30px;
    text-align:center;
    font-size:35px;
    color:white;
  }
  nav{
    float:left;
    width:24%;
    height:300px;
    background:brown;
    padding:20px;
  }
  article{
    float:left;
    padding:20px;
    width:70%;
    height:300px;
    background-color:black;
    height:300px;
    color:white;
  }
  footer{
    background-color:blue;
    padding:10px;
    text-align:center;
    color:white;
  }
</style>

```

Figure 83

Output:

Figure 84: Output:

Homework before lab

This homework will help students to study the concepts of topic before start of lab.

Problem Solution Modeling

After studying the introduction and concept map sections you should be ready to provide the solution of following problems.

Practices from home

Solve the examples from following link:

https://www.w3schools.com/css/css_examples.asp

Procedure & Tools

This section provides information about tools and programming procedures used for the lab.

Tools

Students must ensure that Visual studio 2017/2019 with web utility is installed on their computer.

Editing, Compiling and Running programs in Visual Studio

You are required to use Microsoft Visual Studio environment for web application development. It is expected that you are already familiar with this environment in your courses of Computer Programming and Object-Oriented Programming. Beside that you are also required to be proficient in performing line by line debugging and break point based debugging to remove the errors from your code.

1.1. Walk through Task 1 [Expected time = 20 mins] Following program represents the concept related to HTML & CSS; you are required to type this program in editor of Visual Studio 2017, compile and run to see the output of this program.

```

<!DOCTYPE html>
<html>
<head>
  <title>CSS</title>
  <link rel="stylesheet" type="text/css" href="mystyle.css" />
</head>
<body>
  <h1>First Edition Auctions</h1>
  <table>
    <tr>
      <th>Author</th>
      <th>Title</th>
      <th class="money">Reserve Price</th>
      <th class="money">Current Bid</th>
    </tr>
    <tr>
      <td>E.E. Cummings</td>
      <td>Tulips & Chimneys</td>
      <td class="money">$2,000.00</td>
      <td class="money">$2,642.50</td>
    </tr>
    <tr class="even">
      <td>Charles d'Orleans</td>
      <td>Poemes</td>
      <td class="money"></td>
      <td class="money">$5,866.00</td>
    </tr>
    <tr>
      <td>T.S. Eliot</td>
      <td>Poems 1909 - 1925</td>
      <td class="money">$1,250.00</td>
      <td class="money">$8,499.35</td>
    </tr>
  </table>
</body>
</html>

```

```

body {
  font-family: Arial, Verdana, sans-serif;
  color: #111111;
}

table {
  width: 600px;
}

th, td {
  padding: 7px 10px 10px 10px;
}

th {
  text-transform: uppercase;
  letter-spacing: 0.2em;
  font-size: 90%;
  border-bottom: 2px solid #111111;
  border-top: 1px solid #999;
  text-align: left;
}

tr.even {
  background-color: #efefef;
}

tr:hover {
  background-color: green;
}

.money {
  text-align: right;
}

```

Figure 85: Editing, Compiling and Running programs in Visual Studio

Output:

AUTHOR	TITLE	RESERVE PRICE	CURRENT BID
E.E. Cummings	Tulips & Chimneys	\$2,000.00	\$2,642.50
Charles d'Orleans	Poemes		\$5,866.00
T.S. Eliot	Poems 1909 - 1925	\$1,250.00	\$8,499.35

Walk through Task 2

[20 mins]

```

* {
  font-size: 50px;
  font-family: Arial;
}

h1, h3 {
  color: green;
  font-style: normal;
}

p.note {
  font-size: 40px;
  font-style: normal;
  border-style: dotted;
}

li > h2 {
  font-family: 'Courier New';
  font-size: 40px;
  color: blue;
  font-style: italic;
}

div h3 {
  font-family: Calibri;
  color: red;
  font-style: italic;
}

div + h1 {
  border-style: dotted;
  color: navy;
}

```

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Selector Example 1</title>
  <link rel="stylesheet"
        type="text/css"
        href="StyleSheet.css" />
</head>
<body>
  <div>My style</div>
  <br />
  <div>
    <h3> My Style</h3>
    <h2>My Style</h2>
  </div>
  <h3> My Style</h3>
  <div>
    <li>
      My style
      <h2>My Style</h2>
    </li>
    <h1> My Style</h1>
  </div>
  <h1>My Style</h1>
  <p class="note">My Style</p>
</body>
</html>

```

Figure 86: Walk through Task 2 [20 mins]

Output:

My style

My Style

My Style

My Style

- My style*

My Style

My Style

My Style

My Style

My Style

Walk through Task 3[20 mins]

```

<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    .wrapper { overflow: auto; }
    #main { margin-left: 4px; }
    #leftsidebar { float: none; width: auto; }
    #menulist { margin: 0; padding: 0; }
    .menuitem {
      background: #CDF0F6;
      border: 1px solid #d4d4d4;
      border-radius: 4px;
      list-style-type: none;
      margin: 4px;
      padding: 2px;
    }
    @media screen and (min-width: 480px) {
      #leftsidebar {
        width: 200px;
        float: left;
      }
      #main {
        margin-left: 216px;
      }
    }
  </style>
</head>

```

```

<body>
  <div class="wrapper">
    <div id="leftsidebar">
      <ul id="menulist">
        <li class="menuitem">Menu-item 1</li>
        <li class="menuitem">Menu-item 2</li>
        <li class="menuitem">Menu-item 3</li>
        <li class="menuitem">Menu-item 4</li>
        <li class="menuitem">Menu-item 5</li>
      </ul>
    </div>
    <div id="main">
      <h1>Resize the browser window to see the effect!</h1>
      <p>This example shows a menu that will float to the left of the page if the viewport is 480 pixels wide or wider.</p>
    </div>
  </div>
</body>
</html>

```

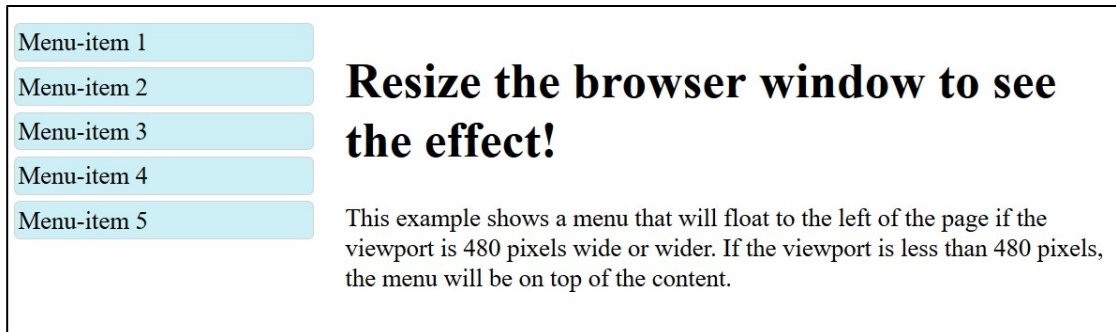


Figure 89: Walk through Task 3 [20 mins]

Practice Tasks

Design solutions of problems discussed in each task and place solution code in a folder specified by your lab instructor.

Practice Task 01 [Expected Time 60mins]

Implement the following task as shown in the picture.

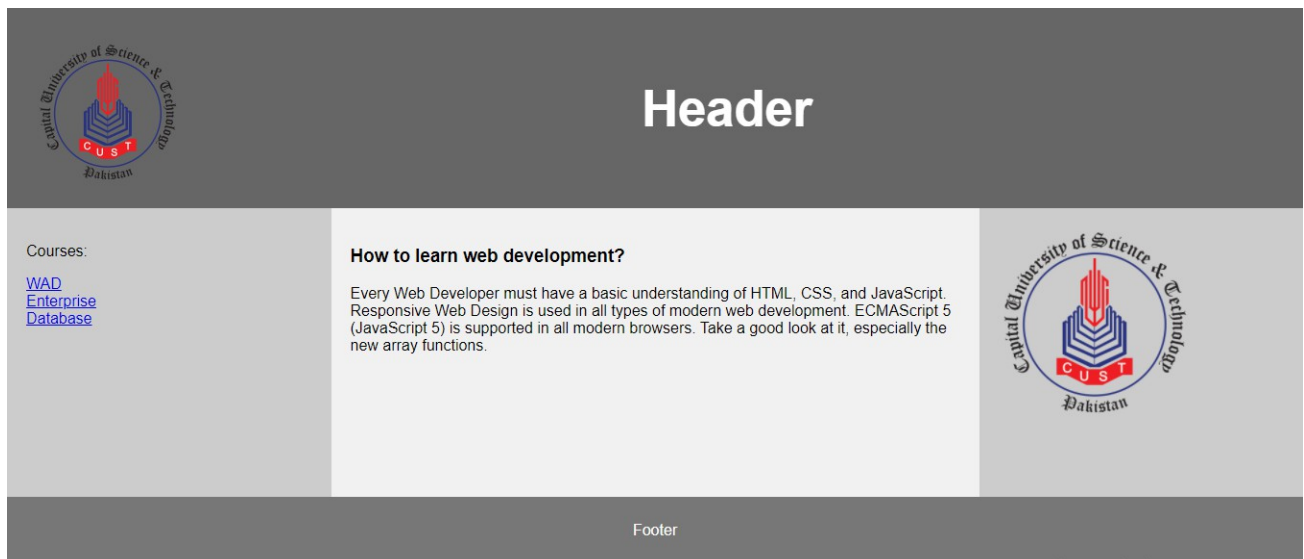


Figure 90: Practice Task 01 [Expected Time 60mins]

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The CSS implementation must produce the required webpage appearance and styling according to the specified requirements.	30%	The webpage will be opened in a browser and its visual output will be checked against the task requirements.
Correct Use of CSS Selectors	Students must correctly use the required CSS selectors, including element, class, ID, grouping, and other appropriate selectors.	25%	Review of the CSS code and browser output to verify that the appropriate selectors have been applied correctly.
Correct Use of CSS Styling Methods	Students must correctly apply CSS properties and values and demonstrate the required styling methods such as inline, internal, or external CSS where applicable.	20%	Code review and browser testing of the applied styles and resulting output.
Code Quality	CSS code should be clean, consistently formatted, properly organized, logically structured, and free of unnecessary rules.	15%	Review of CSS formatting, indentation, selectors, properties, and overall code organization.
Presentation	The final webpage should clearly demonstrate the applied CSS styling and be properly organized according to the given requirements.	10%	Review of the final webpage appearance and demonstration of the implemented CSS features.

Lab 04: Bootstrap CSS

Introduction

Bootstrap is a free and open-source front-end Web framework. Bootstrap was also programmed to support both HTML5 and CSS3. Bootstrap is a free collection of tools for creating a websites and web applications. It contains HTML and CSS-based design templates for typography, forms, buttons, navigation and other interface components, as well as optional JavaScript extensions.

CSS Behind Bootstrap Classes

Bootstrap has made it easier for programmers who sometimes need to create user interfaces, but are not good at it. If you use Bootstrap this way, all you need to know is HTML and you can avoid CSS. Bootstrap has done the CSS work, you just need to use the right class. e.g.

```
<button type="button" class="btn">Basic</button>
```

Bootstrap has already done the CSS work, if we explore the Bootstrap.css file, we will find the normal CSS that we have already learnt.

```
.btn {
    display: inline-block;
    padding: 6px 12px;
    margin-bottom: 0;
    font-size: 14px;
    font-weight: normal;
}
```

Relevant Lecture Readings

- a) Revise Lecture No. 9 and 10 available at \\fs\lectures\$ in instructor's folder.
- b) From web: <https://getbootstrap.com>

Important Selectors in CSS

Bootstrap contains wide range of selectors. Some very common selectors in bootstrap are as following

- Bootstrap Grid System
- Bootstrap typography
- Bootstrap Tables
- Bootstrap buttons
- Bootstrap Pagination
- Bootstrap Navbar

Quick Example

In order to use bootstrap CSS, a developer requires following libraries. You can download them from any bootstrap facilitating website. For instance, some links to libraries are as following:

- <https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css>
- <https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js>

Copy all three files into your local disk named: bootstrap.min.css, jquery.min.js and bootstrap.min.js

Creating a table using bootstrap

Following example illustrates the use of table selector (table) to create a rich HTML table.

```
<html>
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <link rel="stylesheet" href="bootstrap.min.css">
  <script src="jquery.min.js"></script>
  <script src="bootstrap.min.js"></script>
</head>
<body>

  <div class="container">
    <table class="table">
      <thead>
        <tr>
          <th>Firstname</th>
          <th>Lastname</th>
          <th>Email</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>asad</td>
          <td>ameen</td>
          <td>asad@yahoo.com</td>
        </tr>
        <tr>
          <td>numan</td>
          <td>ahmed</td>
          <td>numan@yahoo.com</td>
        </tr>
        <tr>
          <td>Muhammad</td>
          <td>zahed</td>
          <td>mzahed@yahoo.com</td>
        </tr>
      </tbody>
    </table>
  </div>
</body>
</html>
```

Figure 91: Creating a table using bootstrap

Once opened in web browser, the output looks like following

Firstname	Lastname	Email
asad	ameen	asad@yahoo.com
numan	ahmed	numan@yahoo.com
Muhammad	zahed	mzahed@yahoo.com

Figure 92: Creating a table using bootstrap

Creating Alert Box Using Bootstrap CSS

Bootstrap CSS helps to create different type of alert boxes. Common alert boxes are

- Showing success notification
- Showing warning
- Showing some dangerous of potentially negative action

A code excerpt to demonstrate the alert boxes is shown in following figure.

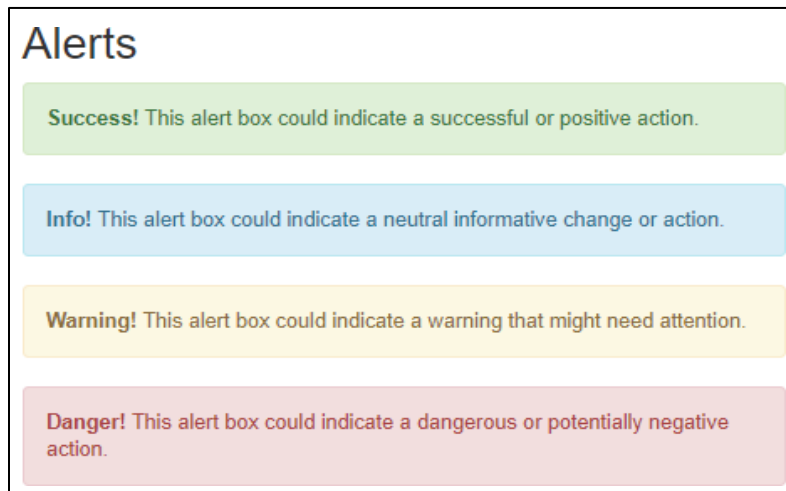
```
<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <link rel="stylesheet" href="bootstrap.min.css">
  <script src="jquery.min.js"></script>
  <script src="bootstrap.min.js"></script>
</head>
<body>

<div class="container">
  <h2>Alerts</h2>
  <div class="alert alert-success">
    <strong>Success!</strong> This alert box could indicate a successful or positive action.
  </div>
  <div class="alert alert-info">
    <strong>Info!</strong> This alert box could indicate a neutral informative change or action.
  </div>
  <div class="alert alert-warning">
    <strong>Warning!</strong> This alert box could indicate a warning that might need attention.
  </div>
  <div class="alert alert-danger">
    <strong>Danger!</strong> This alert box could indicate a dangerous or potentially negative action.
  </div>
</div>

</body>
</html>
```

Figure 93: Creating Alert Box Using Bootstrap CSS

The output of above code is:



Row Selector

One of the important selectors in bootstrap is row selector. A row selector creates a row in web page. It is important to note that *contents inside row allocates themselves in horizontal order*. It is also important to note that *if two rows come in a sequence, the rows allocate themselves in vertical order*.

In following example, two rows are created. It can be observed that the rows allocate themselves in vertical order.

Once the browser is opened, the output appears as shown in Figure 134.



Column Selector

Another important selector in bootstrap is column selector. There are many different selectors that represents column. Following tables describes those selectors

Table 1 Column selector

XL column for desktop	Large column for laptop	Small columns for tablets	Extra small column for mobiles
col-xl	col-lg	col-sm	col-xs

A column has maximum width 12 (col-xl-12, col-lg-12, col-sm-12, col-xs-12) and minimum width 1. It means, if you create a column “**col-xl-12**” it will occupy entire width of the screen. If you create a column “**col-xl-1**” then, it is possible to accommodate **12** columns in a single row. Following code illustrates the use of column selector.

```
<body>
<div class="container">
  <div class="row">
    <div class="col-sm-1" style="background-color:Green;">col-sm-1</div>
    <div class="col-sm-2" style="background-color:red;">col-sm-2</div>
    <div class="col-sm-3" style="background-color:blue;">col-sm-3</div>
    <div class="col-sm-4" style="background-color:grey;">col-sm-4</div>
    <div class="col-sm-5" style="background-color:yellow;">col-sm-5</div>
  </div>
</div>
</body>
```

The code appears in browser as following



Figure 94: Column Selector

The first 4 columns have occupied first row in the browser and hence makes an accumulative width of 10. In this way, the fifth column can not occupy its place in first row and thus, it moves to second row.

If you decrease the size of browser with the help of mouse, the columns will appear as following.

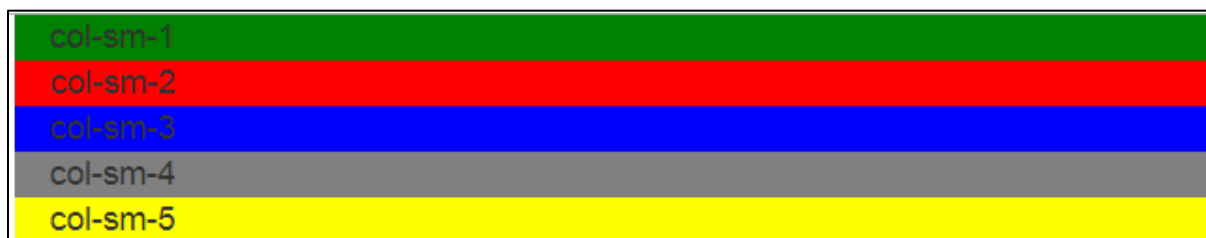


Figure 95: If you decrease the size of browser with the help of mouse, the columns will appear as following.

The **sm** in **col-sm-** means that, the columns will remain in same row until the display size equals tablet. Once the browser size decreased than tablet, each column will automatically capture entire row.

Form Control

Bootstrap CSS can help to generate fascinating forms as shown in following.

```
<head>
  <title>Bootstrap Example</title>
  <link rel="stylesheet" href="bootstrap.min.css">
</head>
<body>
  <div class="container">
    <h2>basic form</h2>
    <form action="xyz.com">
      <div class="row">
        <label for="email">Email:</label>
        <input type="email" class="form-control" id="email" placeholder="Enter email" name="email">
      </div>
      <div class="row">
        <label for="pwd">Password:</label>
        <input type="password" class="form-control" id="pwd" placeholder="Enter password" name="pwd">
      </div>
      <div class="row">
        <label><input type="checkbox" name="remember"> Remember me</label>
      </div>
      <button type="submit" class="btn btn-default">Submit</button>
    </form>
  </div>
</body>
```

Figure 96: Form Control

Output corresponds to above code is

Figure 97: Form Control

Activity Time boxing

Table 2 Activity Timeboxing

Task No.	Activity Name	Activity time	Total Time
Task 1	Design Evaluation	15 minutes	15 minutes
Task 2	Editing and compiling	5 minutes	5 minutes

	program in Visual Studio 2008		
Task 3	Walk Through Task	20 minutes	20 minutes
Task 4	Practice tasks	70 minutes for all tasks	70 minutes
Task 5	Evaluation Task	60 mins for all assigned tasks	60 minutes

Objective of the experiment

- To get a basic understanding of Twitter Bootstrap
- To get a basic understanding of creating responsive designs.

Concept Map

This concept map will help students to understand the main concepts of the topic covered in lab.

Creating HTML file in Visual Studio 2017

Open visual studio 2017 and choose File > New > Project

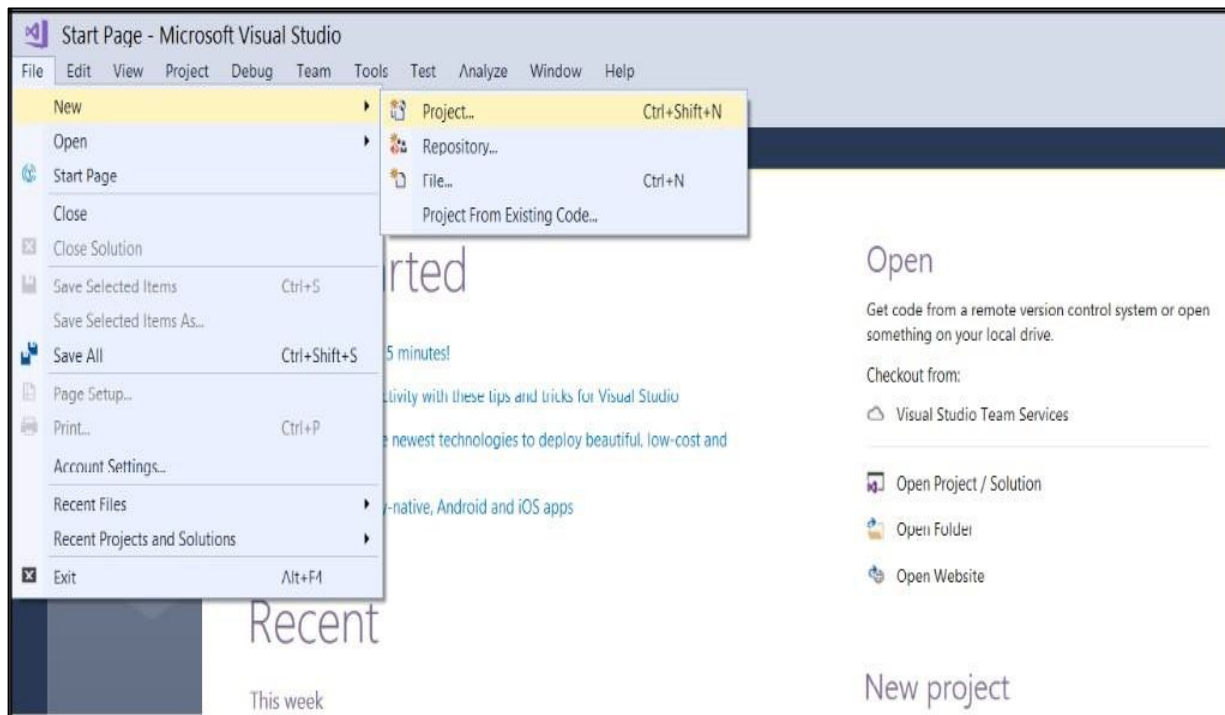


Figure 98: Creating HTML file in Visual Studio 2017

Choose Web from Visual C# and then ASP.Net Web Application (.Net Framework). Assign proper name to your project such as: MyFirstMVCApplication and click OK.

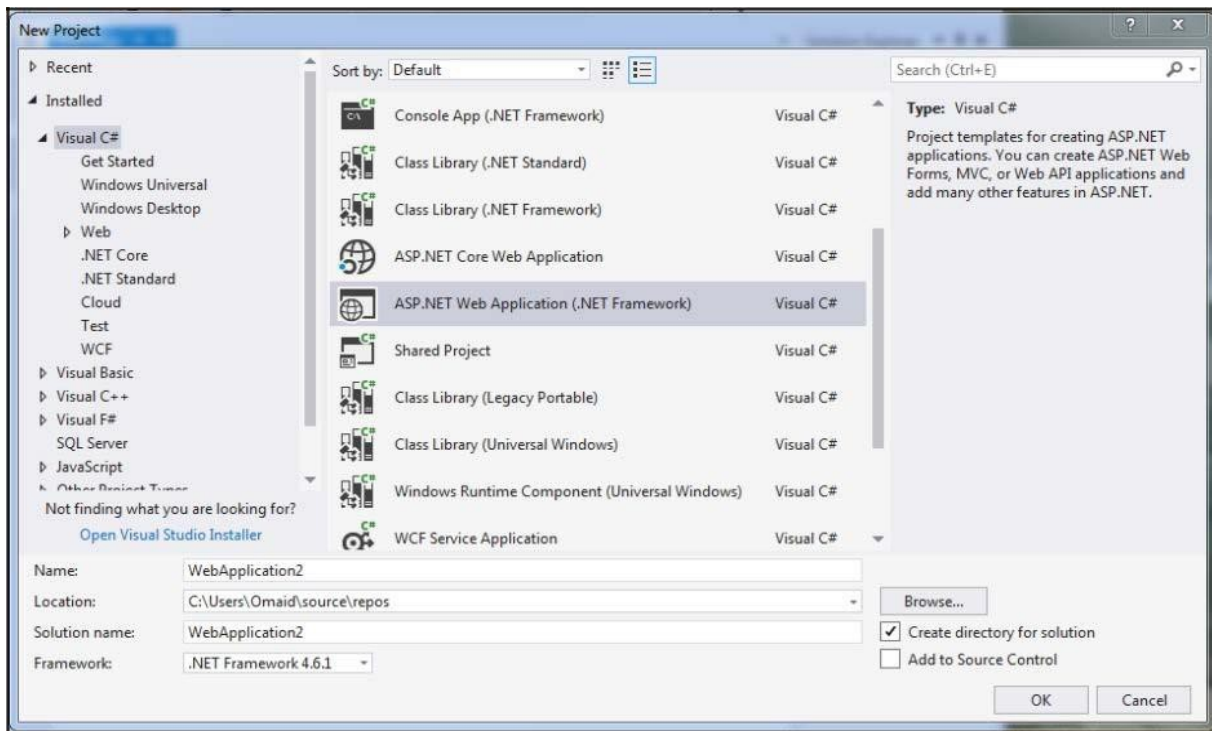


Figure 99: Creating HTML file in Visual Studio 2017

Choose MVC and then check MVC option. Click OK to continue.

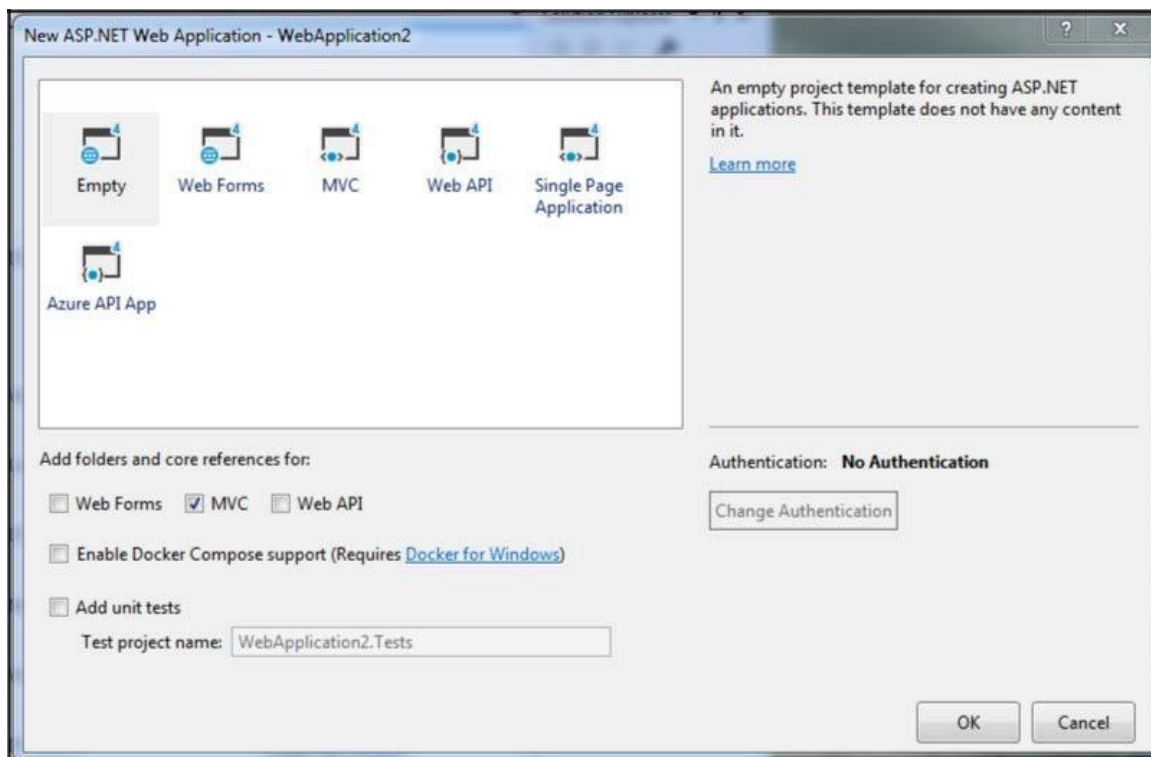


Figure 100: Creating HTML file in Visual Studio 2017

Right Click on the Project Name, Select Add, New Item.

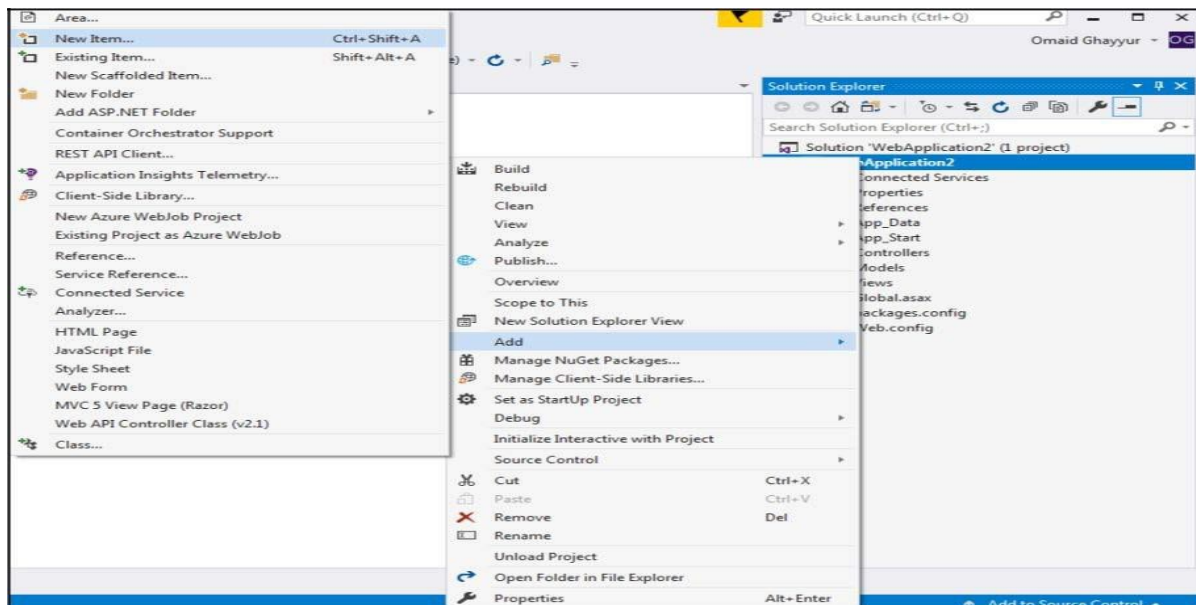


Figure 101: Creating HTML file in Visual Studio 2017

Select the option of Web, Choose HTML, set name for HTML Page and add the page.

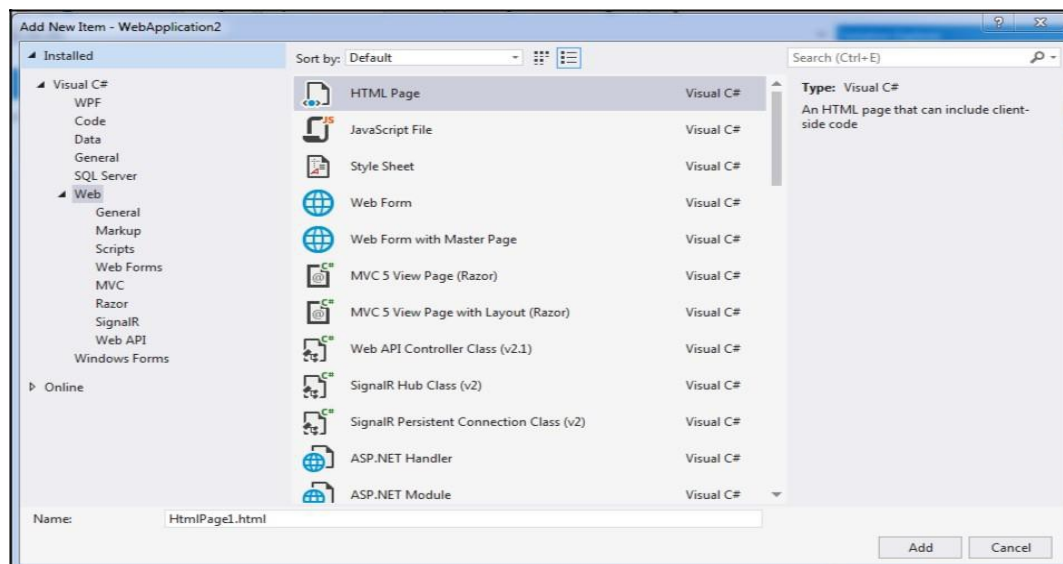
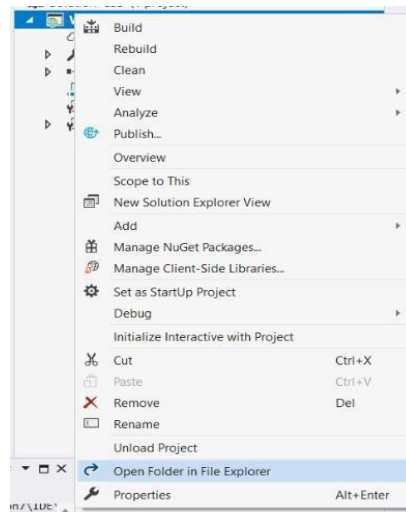


Figure 144 Adding HTML Page

Figure 102: Creating HTML file in Visual Studio 2017

Link Bootstrap with HTML

You can directly download bootstrap file, place it on your project and link the external bootstrap.css file like we did in the previous lab. Copy the bootstrap.css file in the project folder. Right click the project folder and click 'Open Folder in File Explorer'. Paste the bootstrap.css file here.



Refresh the project and you will be able to see bootstrap.css file

Link the external (bootstrap)CSS file with html document using link tag.

```
<link rel="stylesheet" href="bootstrap.css" />
```

Homework before lab

This homework will help students to study the concepts of topic before the start of lab.

Problem solution modeling

After studying the introduction and concept map sections you should be ready to provide the solution of following problems.

Practices form home

Task-1

Implement your task by developing a view as displayed in Figure.

this is header		
left column	central column	right column
footer		

Figure 103: Task-1

Procedure and Tools

This section provides information about tools and programming procedures used for the lab.

Tools

Students must ensure that Visual studio with web utility is installed on their computer.

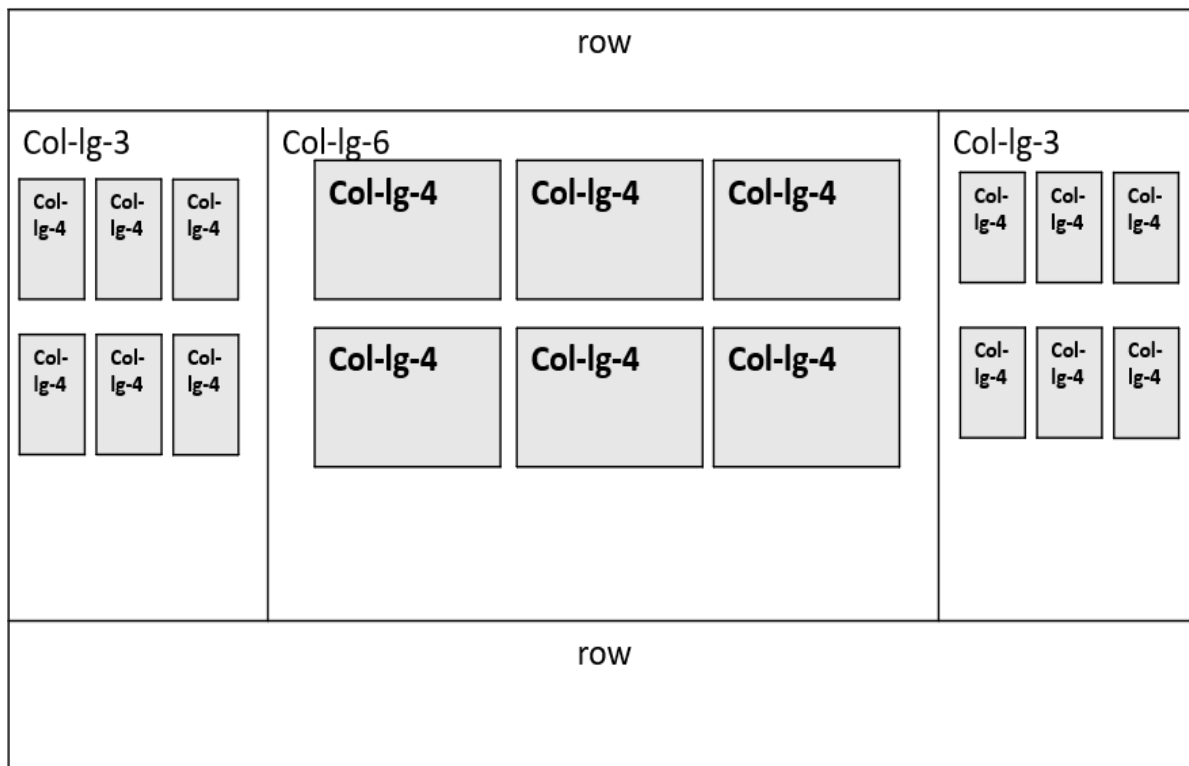
Editing, Compiling and Running programs in Visual Studio

[Expected time = 5mins]

You are required to use Microsoft Visual Studio environment for creation of new HTML file, editing HTML code and open the file in web browser.

Walk through Task 1

Following program represents the concept related to HTML, CSS & Bootstrap; you are required to type this program in editor of Visual Studio, compile and



run to see the output of this program. Let's begin creating a basic site construction. Once you are done with bootstrap.css file, include it in your web page. Our next objective is to develop a layout as shown in following figure.

In order to generate above layout, following code is required.

```

<body>
  <div class="container">
    <header class="row" style="border: 1px solid">
      this is header
    </header>
    <div class="row">
      <div class="col-lg-2" style="border: 1px solid">
        <div class="row"> <!-- this row will align all columns in vertical order-->
          <div class="col-lg-4" style="border: 1px solid">box 1</div>
          <div class="col-lg-4" style="border: 1px solid">box 2</div>
          <div class="col-lg-4" style="border: 1px solid">box 3</div>
          <div class="col-lg-4" style="border: 1px solid">box 4</div>
          <div class="col-lg-4" style="border: 1px solid">box 5</div>
          <div class="col-lg-4" style="border: 1px solid">box 6</div>
        </div>
      </div>
      <div class="col-lg-6" style="border: 1px solid">
        <div class="row"> <!-- this row will align all columns in vertical order-->
          <div class="col-lg-4" style="border: 1px solid">box 1</div>
          <div class="col-lg-4" style="border: 1px solid">box 2</div>
          <div class="col-lg-4" style="border: 1px solid">box 3</div>
          <div class="col-lg-4" style="border: 1px solid">box 4</div>
          <div class="col-lg-4" style="border: 1px solid">box 5</div>
          <div class="col-lg-4" style="border: 1px solid">box 6</div>
        </div>
      </div>
      <div class="col-lg-2" style="border: 1px solid">
        <div class="row"> <!-- this row will align all columns in vertical order-->
          <div class="col-lg-4" style="border: 1px solid">box 1</div>
          <div class="col-lg-4" style="border: 1px solid">box 2</div>
          <div class="col-lg-4" style="border: 1px solid">box 3</div>
          <div class="col-lg-4" style="border: 1px solid">box 4</div>
          <div class="col-lg-4" style="border: 1px solid">box 5</div>
          <div class="col-lg-4" style="border: 1px solid">box 6</div>
        </div>
      </div>
    </div>
    <footer class="row" style="border: 1px solid">
      footer
    </footer>
  </div>
</body>

```

Figure 104: Tools

Practice Tasks

This section will provide information about the practice tasks which are required to be performed in lab session. Design solutions of problems discussed in each task and place solution code in a folder specified by your lab instructor.

Practice Task 01

Implement your task and the view should be displayed like Figure

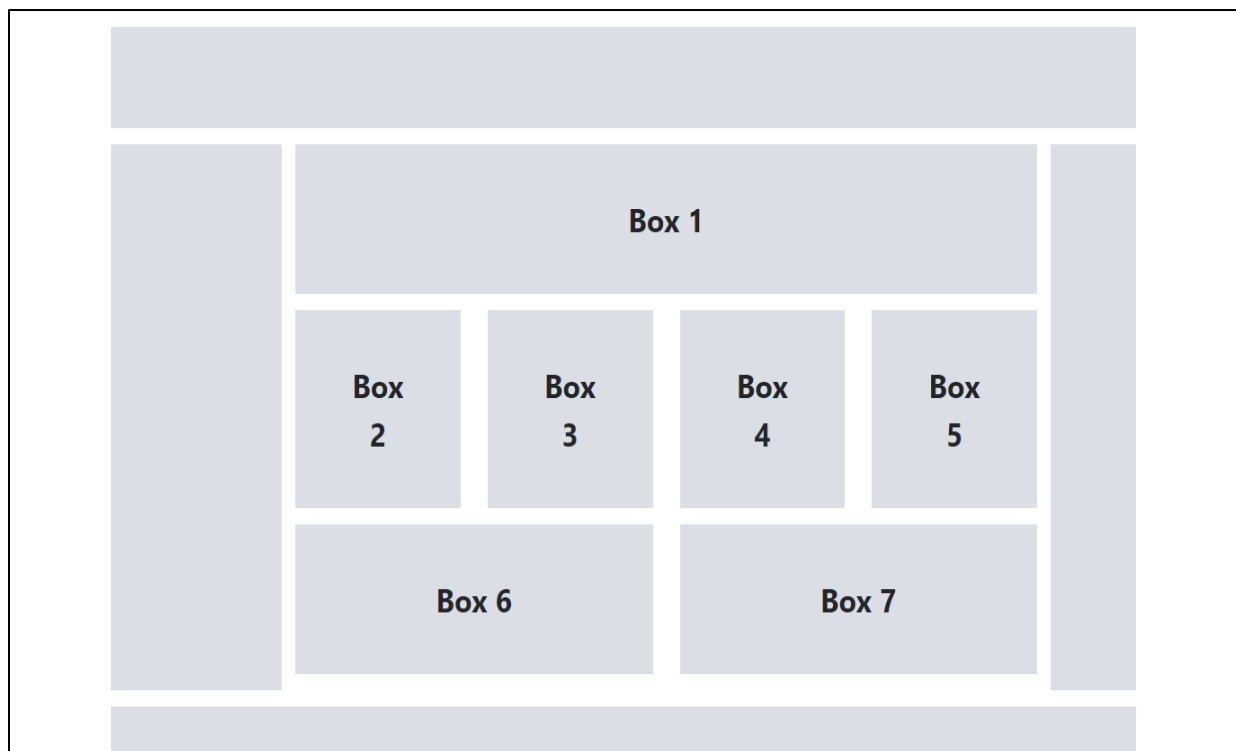


Figure 105: Practice Tasks

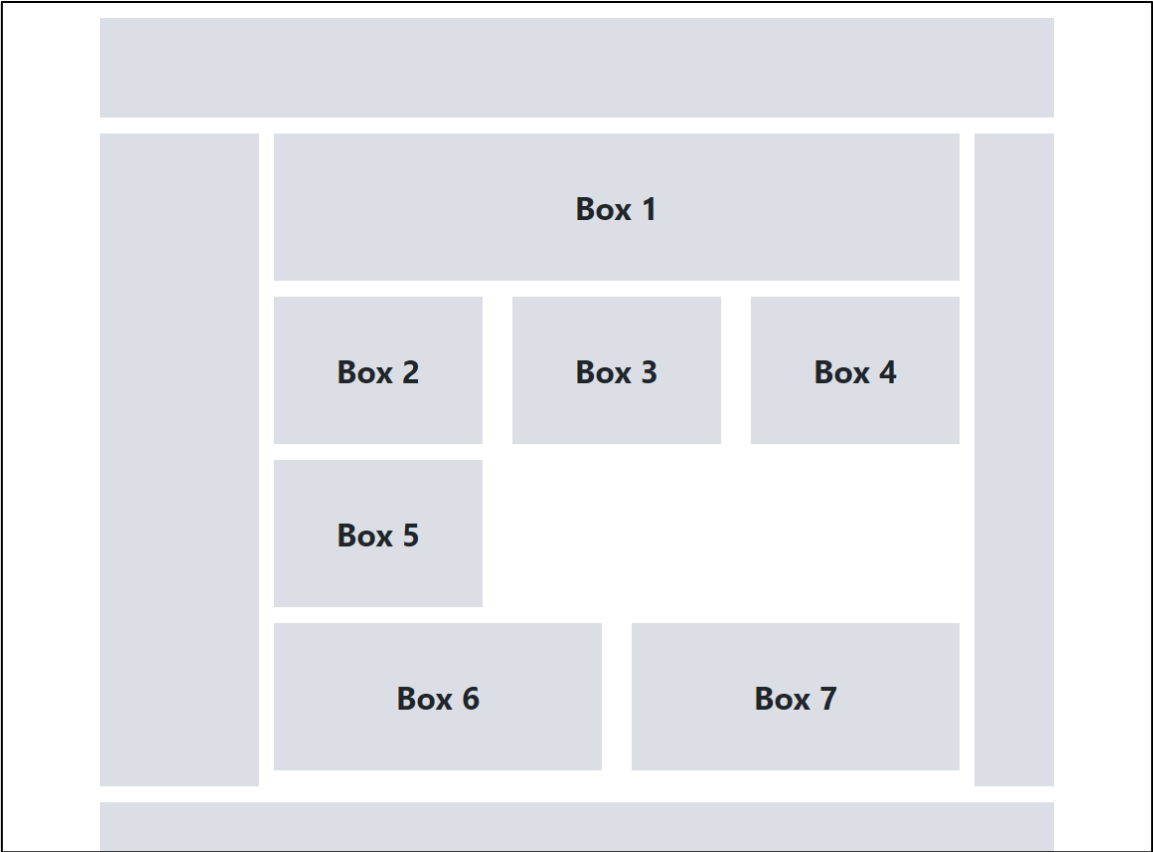


Figure 106: Practice Tasks

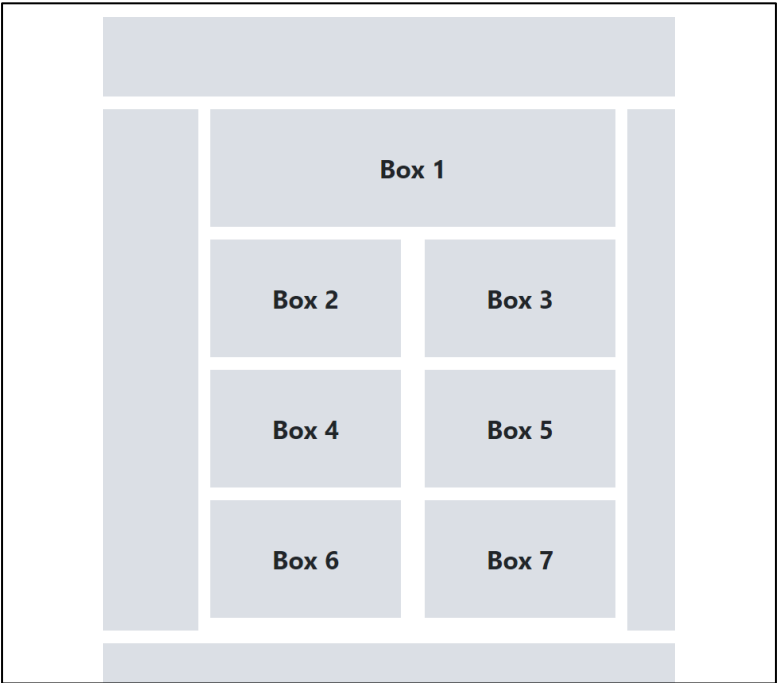


Figure 107: Practice Tasks

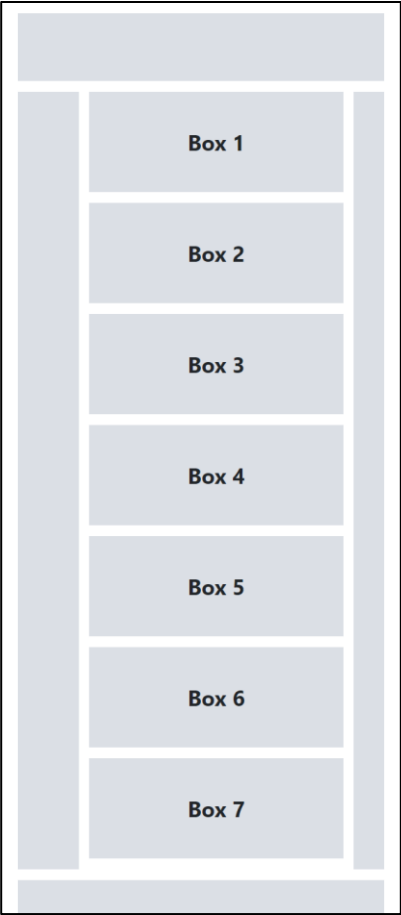


Figure 108: Practice Tasks

Practice Task 02

Implement your task and the view should be displayed like Figure

The screenshot displays a web application titled "Student Management". It features a "Register Form" with the following fields: Name, Education, Address, Gender (a dropdown menu with "Choose..." selected), Age, and a file upload field for "Choose valid CNIC file..." with a "Browse" button. There is also a text area for "About Yourself" and a checkbox for "Agree to terms and conditions". A blue "Register" button is at the bottom of the form. Below the form is a table titled "All Students Data" with the following data:

#	Name	Education	City	Gender	Age
1	Ahmed	BSCS	Rawalpindi	Male	23
1	Ali	BBA	Islamabad	Male	22

Figure 109: Practice Task 02 [Expected Time 30 mins]

Out comes

After completing this lab, students will be able to understand the basics of CSS.

Evaluation

The evaluation criteria for this lab will be based on the completion of the following tasks. Each task is assigned the marks percentage which will be evaluated by the instructor in the lab whether the student has finished the complete/partial task(s).

Table 3 Evaluation of the lab

Sr. No.	Description	Marks
1	Walkthrough Task	20
2	Practice tasks and Testing	50
3	Evaluation Tasks - Quiz (Unseen)	30

Further Readings

Bootstrap material

- <https://startbootstrap.com/snippets/>
- <https://getbootstrap.com>
- <https://bootswatch.com>

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The Bootstrap-based webpage must run without errors and produce the required output, components, and interactions according to the task requirements.	30%	The webpage will be run in a browser and the output will be checked against the specified requirements.
Correct Use of Bootstrap Classes and Components	Students must correctly use Bootstrap classes and components for required elements such as buttons, tables, alerts, forms, navigation, and other interface components.	20%	Review of the HTML source code and browser output to verify correct use of Bootstrap classes and components.
Grid System and Responsive Design	Students must correctly apply the Bootstrap grid system, rows, columns, and responsive design principles to create layouts that adjust to different screen sizes.	25%	The webpage will be tested in the browser at different window sizes to verify the grid structure and responsive behavior.
Code Quality	Bootstrap and HTML code should be clean, properly indented, logically organized, readable, and free of unnecessary code.	15%	Review of the submitted HTML code, Bootstrap classes, formatting, and overall structure.
Presentation	The final webpage should be clearly demonstrated, visually organized, and properly implement the required Bootstrap-based design.	10%	Review and demonstration of the final webpage and submitted project files.

Lab 05: JavaScript Basics

Background

JavaScript is a programming language used to add logic and interactivity to web pages. HTML is responsible for structuring the content of a webpage, while CSS controls its appearance. JavaScript allows a webpage to perform calculations, make decisions, repeat operations, and respond to changing data.

In this lab, students will learn the fundamental programming concepts required for JavaScript development. These concepts include variables, data types, operators, conditional statements, and loops. These concepts form the foundation for more advanced JavaScript programming covered in later labs.

Introduction to JavaScript

JavaScript is commonly executed inside a web browser. A JavaScript program can be written directly inside an HTML document using the `<script>` element or placed in a separate .js file.

For example:

```
<script>
  console.log("Hello, JavaScript!");
</script>
```

The `console.log()` statement displays information in the browser's Developer Tools Console. It is particularly useful for testing JavaScript code and checking the values produced by a program.

A separate JavaScript file is generally preferred because it keeps HTML and JavaScript code organized.

```
<script src="script.js"></script>|
```

The `src` attribute specifies the location of the JavaScript file.

Adding JavaScript to an HTML Page

A JavaScript file can be connected to an HTML document using the `<script>` element.

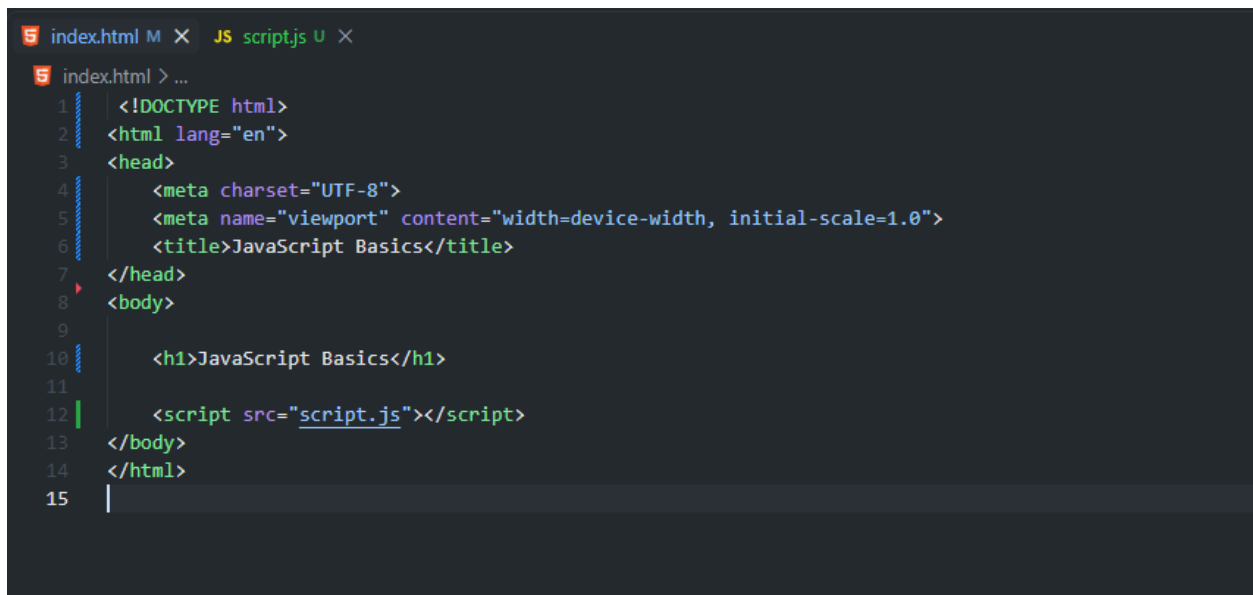
Consider the following project structure:

lab-05/

- └─ index.html
- └─ script.js

The index.html file contains the webpage structure, while script.js contains the JavaScript code.

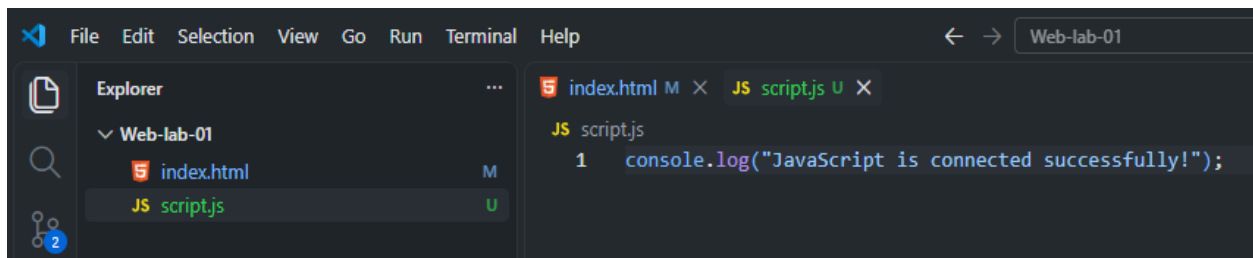
Example:



```
index.html M X JS script.js U X
index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>JavaScript Basics</title>
7 </head>
8 <body>
9
10  <h1>JavaScript Basics</h1>
11
12  <script src="script.js"></script>
13 </body>
14 </html>
15 |
```

Figure 110: Adding JavaScript to an HTML Page

The JavaScript file can then contain:



```
File Edit Selection View Go Run Terminal Help
Web-lab-01
index.html M X JS script.js U X
JS script.js
1 console.log("JavaScript is connected successfully!");
```

Figure 111: Adding JavaScript to an HTML Page

When the page is opened in a browser, the message will appear in the browser Console.

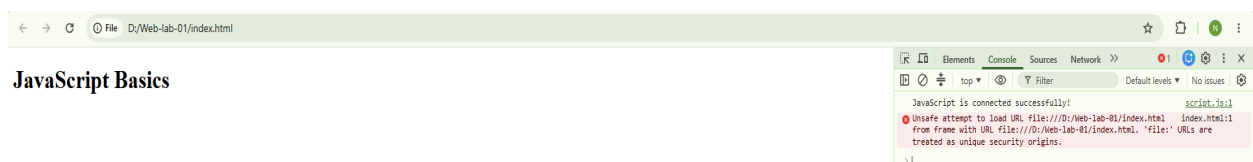


Figure 112: Adding JavaScript to an HTML Page

Running JavaScript Using Browser Developer Tools

Modern browsers provide Developer Tools for testing and debugging webpages. The Console allows developers to execute JavaScript statements and view program output.

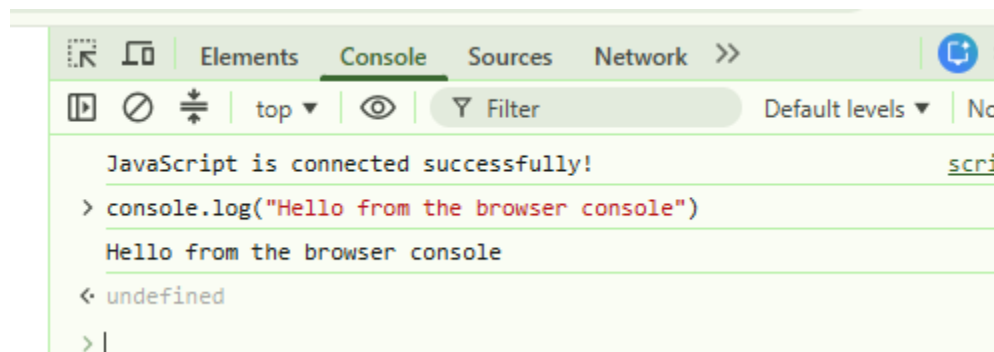
To open Developer Tools:

- Press **F12**, or
- Press **Ctrl + Shift + I** in most browsers.
- Select the **Console** tab.

For example, the following statement can be entered directly into the Console:

```
console.log("Hello from the browser console!");
```

The Console displays the resulting message.



The Console can also be used to test expressions:

```
10 + 20
```

The browser will display:

```
30
```

This makes the Console useful for understanding JavaScript expressions before placing them into a complete program.

Variables in JavaScript

A variable is a named location used to store a value. Variables allow a program to store information and use that information later.

JavaScript provides `let`, `const`, and the older `var` keyword for declaring variables. In modern JavaScript, `let` and `const` are generally preferred.

The `let` Keyword

`let` is used when a variable's value may need to change.

```
JS script.js > ...
1  let age = 20;
2  console.log(age);
3  age = 21;
4  console.log(age);
5  |
```

The first value of `age` is 20. Later, the value is changed to 21.

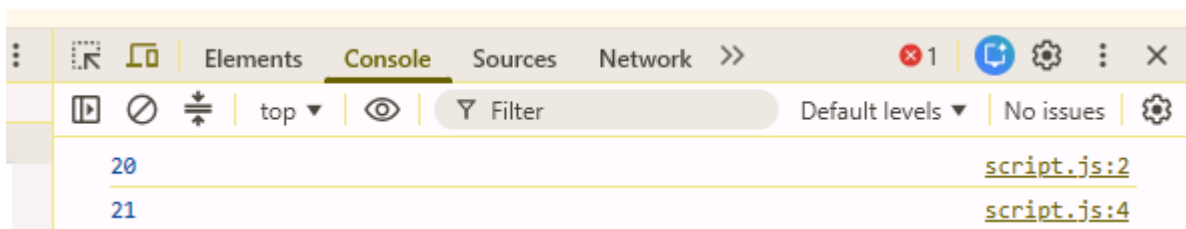


Figure 113: The `let` Keyword

The `const` Keyword

`const` is used when a variable should not be reassigned after it has been initialized.

```
JS script.js > ...
1  const university = "Capital University of Science and Technology";
2
3  console.log(university);
4  university="CUST";
```

Figure 114: The `const` Keyword

The value stored in `university` cannot later be reassigned.

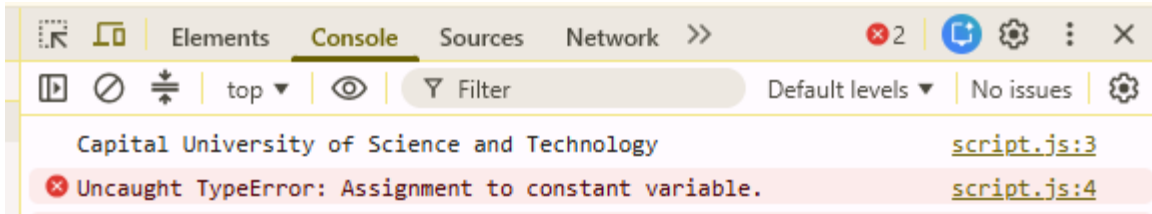


Figure 115: The const Keyword

Declaration and Initialization

A variable can be declared and initialized at the same time:

```
let marks = 85;
```

Here:

- let is the keyword.
- marks is the variable name.
- 85 is the value assigned to the variable.

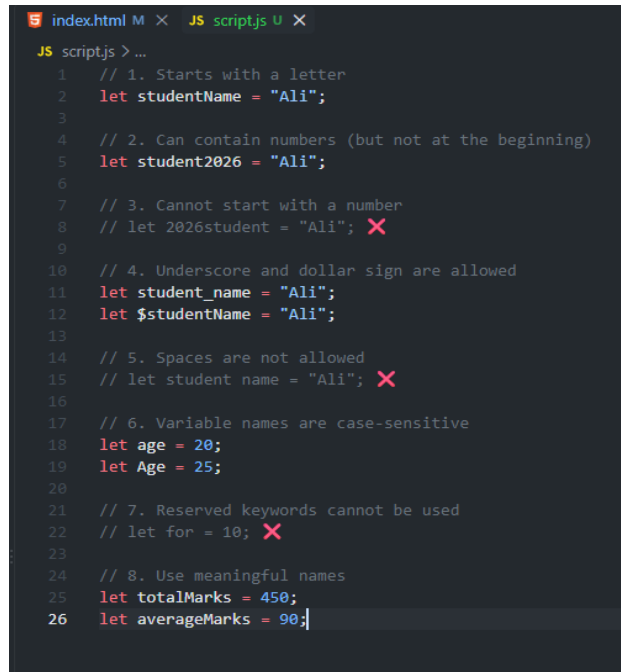
Variables can then be used in expressions:

```
JS script.js > ...
1  let marks = 85;
2  let bonus = 5;
3  let total = marks + bonus;
4  console.log(total);
```

Naming Variables

Rules for Naming Variables

1. A variable name must begin with a letter, underscore (`_`), or dollar sign (`$`).
2. A variable name cannot begin with a number.
3. A variable name can contain letters, numbers, underscores, and dollar signs.
4. Spaces are not allowed in variable names.
5. Variable names are case-sensitive.
6. Reserved JavaScript keywords cannot be used as variable names.
7. Variable names should be meaningful and descriptive.
8. JavaScript commonly uses **camelCase** for variable names.



```

index.html M X JS scriptjs U X
JS scriptjs > ...
1 // 1. Starts with a letter
2 let studentName = "Ali";
3
4 // 2. Can contain numbers (but not at the beginning)
5 let student2026 = "Ali";
6
7 // 3. Cannot start with a number
8 // let 2026student = "Ali"; ❌
9
10 // 4. Underscore and dollar sign are allowed
11 let student_name = "Ali";
12 let $studentName = "Ali";
13
14 // 5. Spaces are not allowed
15 // let student name = "Ali"; ❌
16
17 // 6. Variable names are case-sensitive
18 let age = 20;
19 let Age = 25;
20
21 // 7. Reserved keywords cannot be used
22 // let for = 10; ❌
23
24 // 8. Use meaningful names
25 let totalMarks = 450;
26 let averageMarks = 90;

```

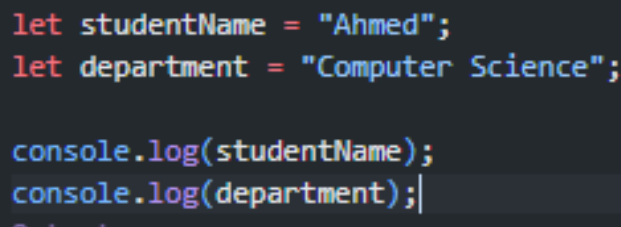
Figure 116: Rules for Naming Variables

Data Types in JavaScript

A data type describes the kind of value stored in a variable. JavaScript supports several built-in data types.

String

A string represents text and is written using quotes.



```

let studentName = "Ahmed";
let department = "Computer Science";

console.log(studentName);
console.log(department);

```

Output:

Ahmed

Computer Science

Number

The number type is used for numerical values, including integers and decimal values.

```
let age = 21;
let percentage = 87.5;
console.log(age);
console.log(percentage);
```

Boolean

A Boolean value can be either true or false.

```
let isPassed = true;
let isPresent = false;

console.log(isPassed);
console.log(isPresent);
```

Boolean values are commonly used when a program needs to make decisions.

Undefined

A variable that has been declared but has not been assigned a value contains undefined.

```
let result;

console.log(result);
```

Output:

Undefined

Null

null represents an intentionally empty or missing value.

```
let selectedCourse = null;
console.log(selectedCourse);
```

Checking Data Types with typeof

JavaScript provides the typeof operator to determine the type of a value.

```
let Name = "Ali";
let age = 20;
let passed = true;

console.log(typeof Name);
console.log(typeof age);
console.log(typeof passed);
```

Output:

string
number
boolean

Operators in JavaScript

Operators are symbols used to perform operations on values and variables.

Arithmetic Operators

Arithmetic operators are used for mathematical calculations.

Operator	Purpose	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
%	Remainder	10 % 3

Example:

```

JS script.js > ...
1  let marks1 = 80;
2  let marks2 = 70;
3
4  let total = marks1 + marks2;
5  let difference = marks1 - marks2;
6  let product = marks1 * marks2;
7  let division = marks1 / marks2;
8  let remainder=marks1%marks2
9  console.log(remainder);
10
11 |

```

The % operator returns the remainder after division:

```
let remainder=marks1%marks2
```

Output:



Figure 117: Arithmetic Operators

The remainder operator is particularly useful for determining whether a number is even or odd.

Assignment Operators

Assignment operators are used to assign or update values.

```

index.html M X JS script.js U X
JS script.js > ...
1  let marks = 50;
2  marks = marks + 10;
3  console.log(marks);
4  |

```

Output:

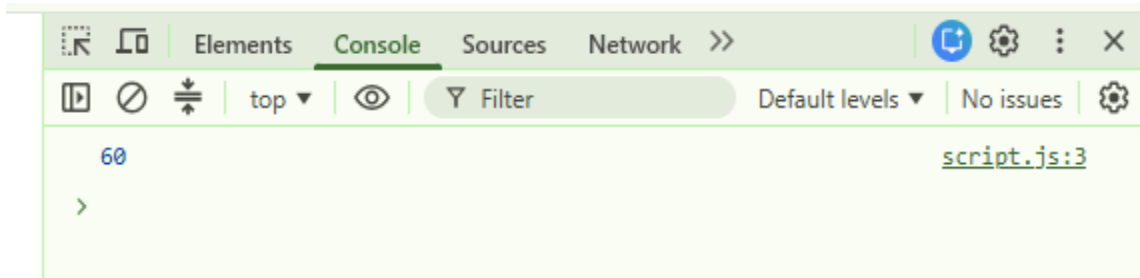
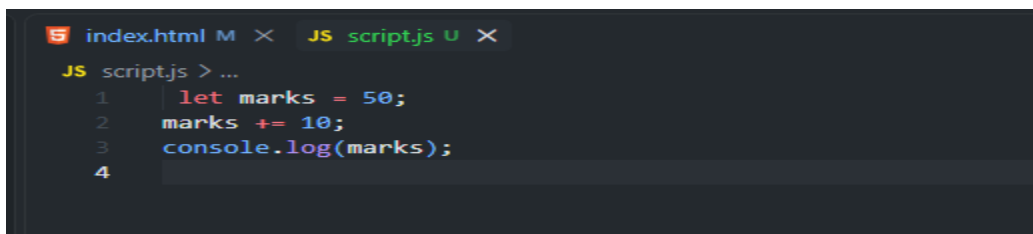


Figure 118: Assignment Operators

➤ JavaScript also provides shorthand assignment operators:



Output:

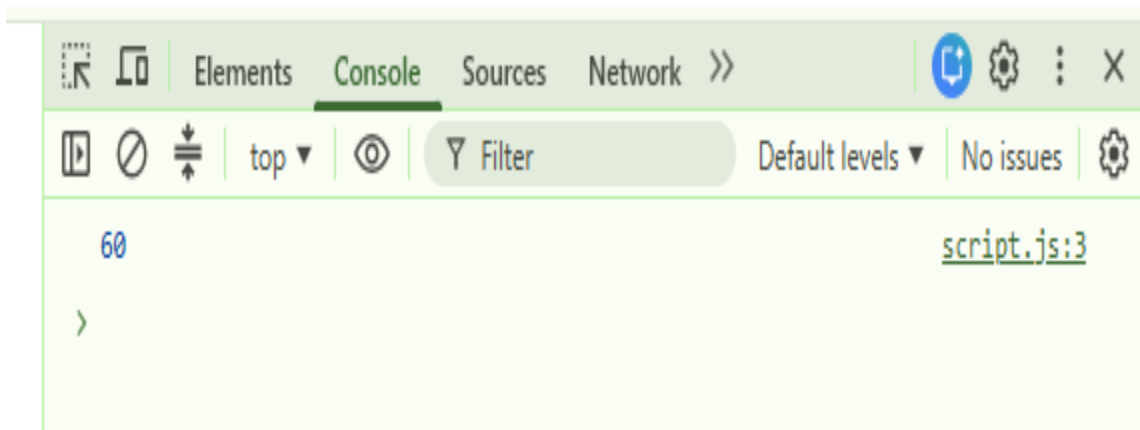


Figure 119: Assignment Operators

The `+=` operator adds the value on the right to the existing value.

Other examples include:

`marks -= 5;`

`marks *= 2;`

`marks /= 2;`

Comparison Operators

Comparison operators compare two values and produce a Boolean result.

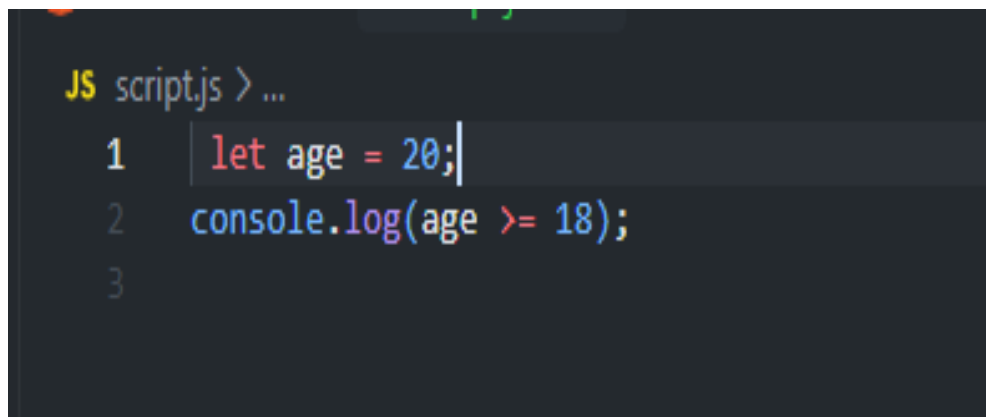
```
let marks = 80;  
console.log(marks > 50);  
console.log(marks < 50);  
console.log(marks === 80);  
console.log(marks !== 70);
```

Common comparison operators include:

Operator Meaning

>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
===	Strictly equal
!==	Strictly not equal

For example:

A screenshot of a code editor with a dark background. The text 'JS script.js > ...' is at the top left. Below it, three lines of code are shown with line numbers 1, 2, and 3 on the left. Line 1: 'let age = 20;'. Line 2: 'console.log(age >= 18);'. Line 3: is empty. The code is color-coded: 'let' is red, 'age' is blue, '=' is white, '20' is white, ';' is white, 'console.log' is blue, 'age' is blue, '>=' is red, '18' is white, and ';' is white.

```
JS script.js > ...  
1  let age = 20;  
2  console.log(age >= 18);  
3
```

Output:

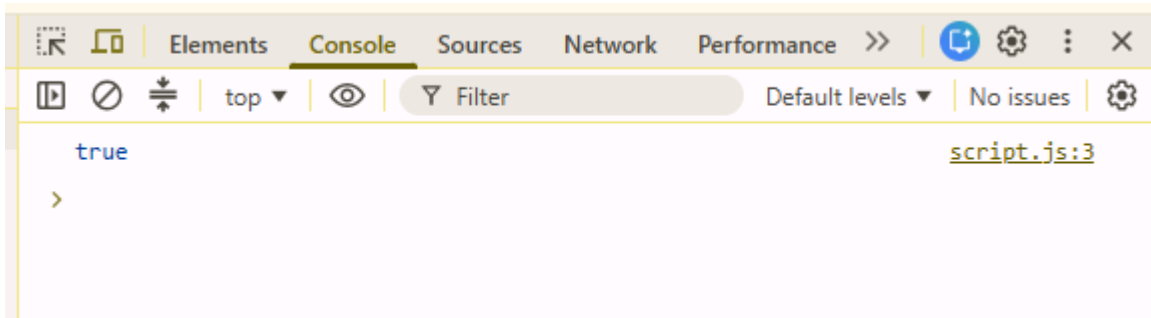


Figure 120: Comparison Operators

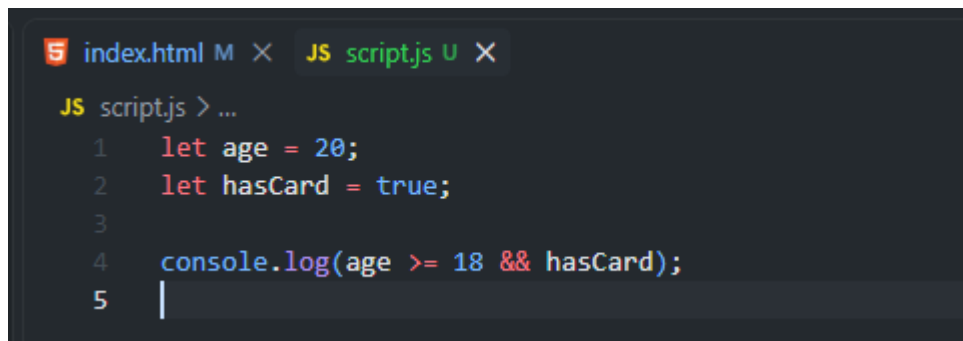
Logical Operators

Logical operators are used to combine conditions.

The three common logical operators are:

- && — AND
- || — OR
- ! — NOT

Example:



Both conditions are true, so the result is:



Figure 121: Logical Operators

Conditional Statements

Conditional statements allow a program to make decisions based on conditions.

The if Statement

The if statement executes a block of code only when its condition is true.

```
JS script.js > ...
1  let marks = 75;
2
3  if (marks >= 50) {
4      console.log("Pass");
5  }
6
```

Since marks is greater than or equal to 50, the program displays:



Figure 122: The if Statement

The if...else Statement

The else block executes when the if condition is false.

```
index.html M X JS script.js U X
JS script.js > ...
1  let marks = 40;
2
3  if (marks >= 50) {
4      console.log("Pass");
5  } else {
6      console.log("Fail");
7  }
8
```

Output:

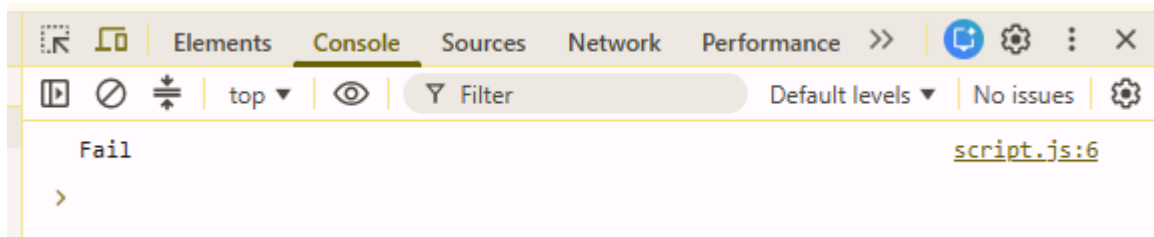


Figure 123: The if...else Statement

The else if Statement

When there are multiple possible conditions, else if can be used.

```
script.js > ...
1  let marks = 82;
2
3  if (marks >= 90) {
4      console.log("Grade A+");
5  } else if (marks >= 80) {
6      console.log("Grade A");
7  } else if (marks >= 70) {
8      console.log("Grade B");
9  } else if (marks >= 60) {
10     console.log("Grade C");
11 } else {
12     console.log("Fail");
13 }
14
```

Since the marks are 82, the output is:



Figure 124: The else if Statement

Loops in JavaScript

A loop repeatedly executes a block of code while a specified condition is satisfied. Loops are useful when the same operation needs to be performed multiple times.

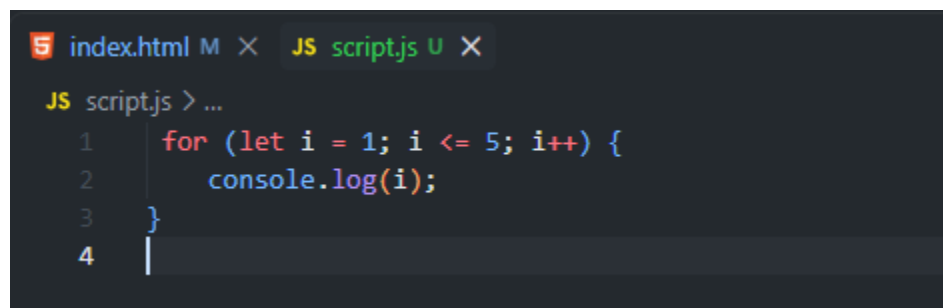
for Loop

A for loop is commonly used when the number of repetitions is known.

The basic structure is:

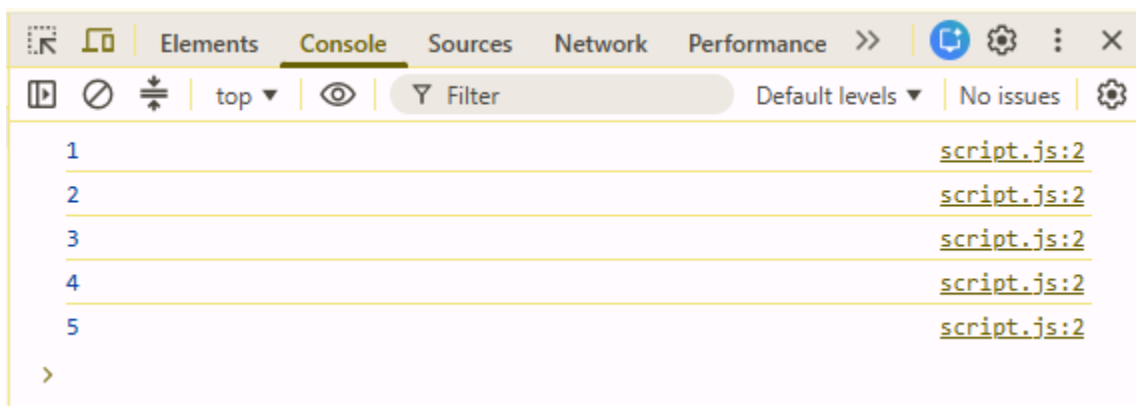
```
for (initialization; condition; update) {  
    // code to repeat  
}
```

Example:



```
index.html M X JS script.js U X  
JS script.js > ...  
1   for (let i = 1; i <= 5; i++) {  
2       console.log(i);  
3   }  
4   |
```

Output:



```
1 script.js:2  
2 script.js:2  
3 script.js:2  
4 script.js:2  
5 script.js:2  
>
```

Figure 125: for Loop

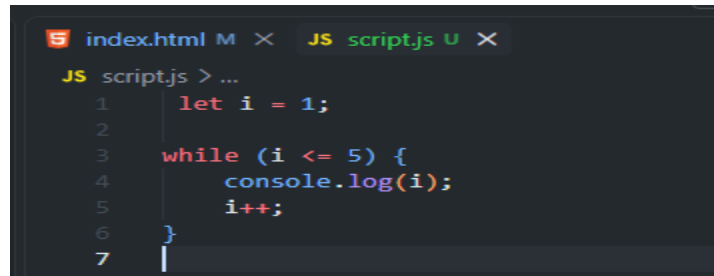
The three parts of the loop are:

1. let i = 1 — initializes the counter.

2. `i <= 5` — determines whether the loop should continue.
3. `i++` — increases the counter after each iteration.

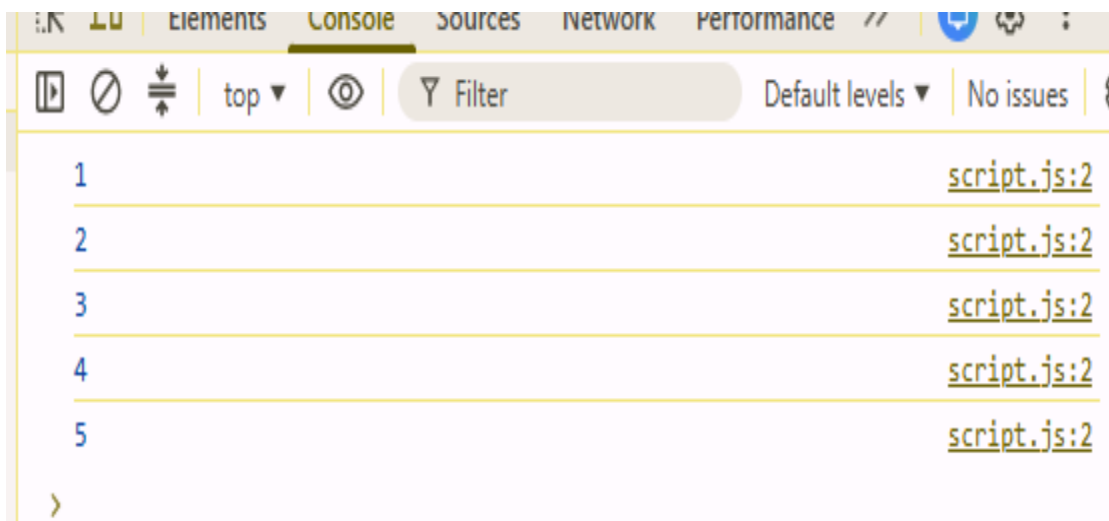
while Loop

A while loop repeats a block of code as long as its condition remains true.



```
index.html M x JS script.js U x
JS script.js > ...
1   let i = 1;
2
3   while (i <= 5) {
4       console.log(i);
5       i++;
6   }
7   |
```

The output is:



The screenshot shows the browser's developer console with the 'Console' tab selected. It displays five log entries, each with the value '1', '2', '3', '4', and '5' on separate lines. Each log entry is linked to 'script.js:2'.

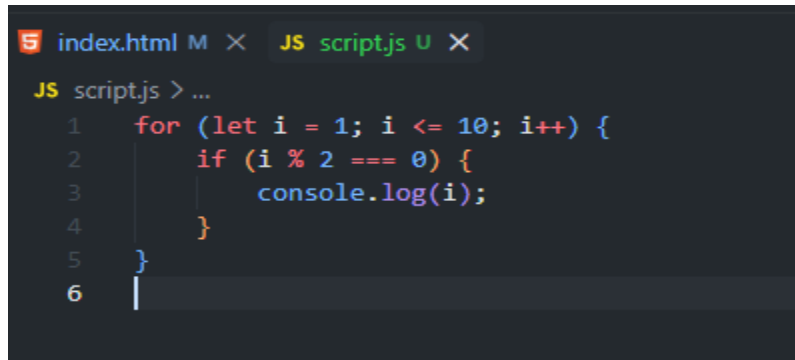
Figure 126: while Loop

The counter must be updated inside the loop. Otherwise, the condition may remain true indefinitely.

Combining Loops with Conditions

Loops and conditional statements can be used together.

For example, the following program displays even numbers from 1 to 10:



```
index.html M X JS script.js U X
JS script.js > ...
1 for (let i = 1; i <= 10; i++) {
2   if (i % 2 === 0) {
3     console.log(i);
4   }
5 }
6 |
```

Output:

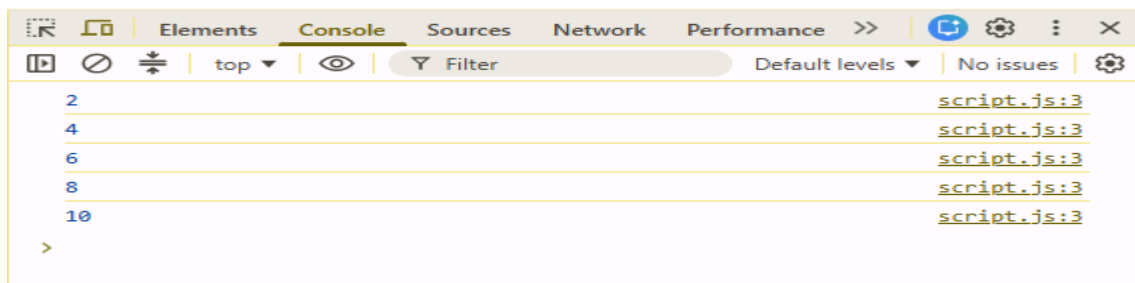


Figure 127: Combining Loops with Conditions

The % operator determines whether the number has a remainder of zero when divided by 2.

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Review JavaScript variables and write one example using let and one example using const.

Home Task 02

Review the JavaScript data types String, Number, Boolean, Undefined, and Null. Write one example value for each.

Home Task 03

Review the syntax of if, if...else, else if, for, and while statements.

Lab Objectives

The students will be able to:

- Explain the role of JavaScript in web development.
- Connect an external JavaScript file with an HTML document.
- Use the browser Developer Tools Console to execute and test JavaScript.
- Declare and modify variables using let and const.
- Identify and use common JavaScript data types.
- Use the typeof operator to inspect data types.
- Apply arithmetic, assignment, comparison, and logical operators.
- Use if, if...else, and else if conditional statements.
- Use for and while loops to repeat operations.
- Combine variables, operators, conditions, and loops to solve simple programming problems.

Walkthrough Task

Task: Student Result Calculator

Build a simple JavaScript program that stores student information, calculates the student's total and average marks, determines whether the student has passed, assigns a grade, and displays a sequence of numbers using a loop.

The program should be executed through the browser Console.

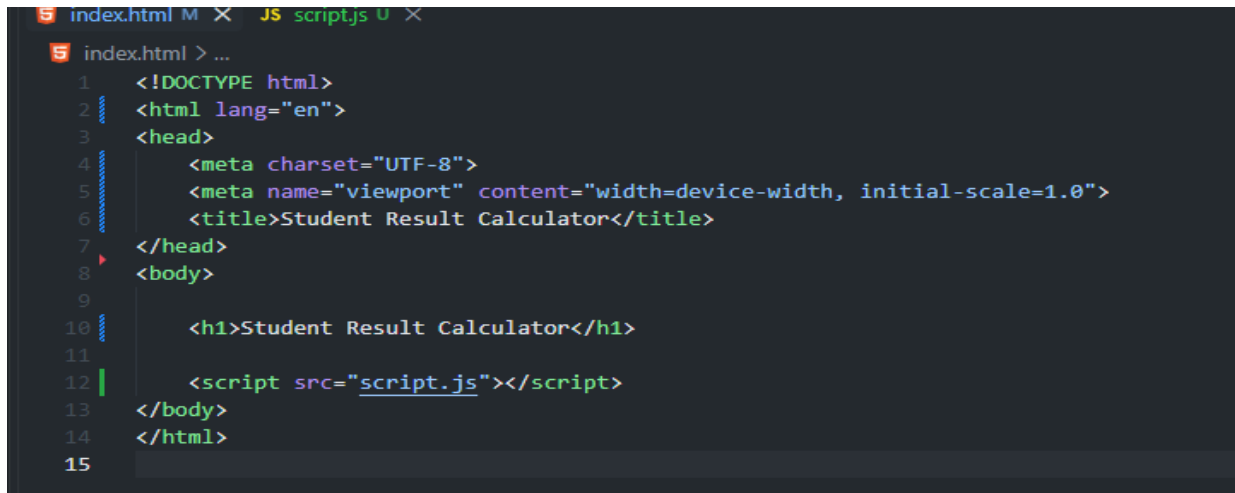
Project Structure

Create a folder named lab-05 and add the following files:

```
lab-05/  
├── index.html  
└── script.js
```

Step 1: Create the HTML File

Create index.html and add the following code:

A screenshot of a code editor with two tabs: 'index.html' and 'JS script.js'. The 'index.html' tab is active, showing the following HTML code:

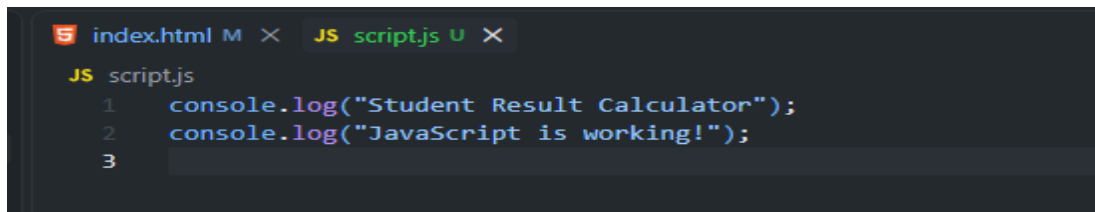
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Student Result Calculator</title>
7 </head>
8 <body>
9
10  <h1>Student Result Calculator</h1>
11
12  <script src="script.js"></script>
13 </body>
14 </html>
15
```

Figure 128: Step 1: Create the HTML File

Save the file and open it in a browser.

Step 2: Connect and Test JavaScript

Open script.js and write:

A screenshot of a code editor with two tabs: 'index.html' and 'JS script.js'. The 'JS script.js' tab is active, showing the following JavaScript code:

```
1 console.log("Student Result Calculator");
2 console.log("JavaScript is working!");
3
```

Figure 129: Step 2: Connect and Test JavaScript

Open the browser Developer Tools and select the **Console** tab.

The following output should appear:

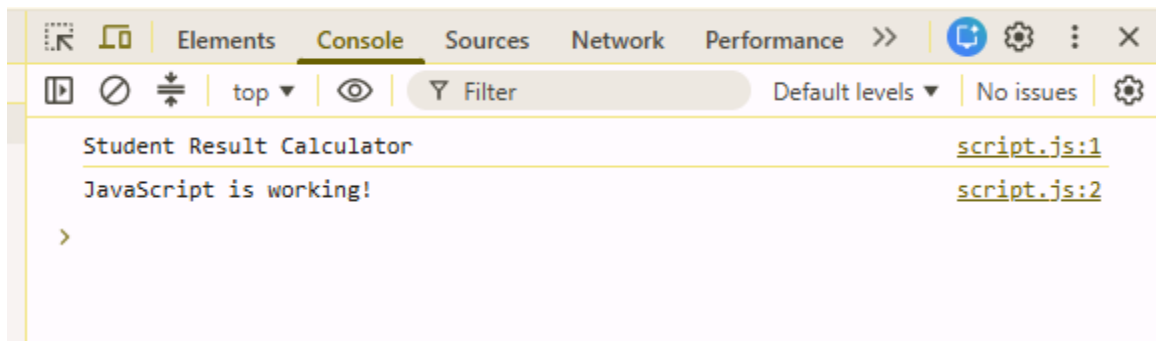


Figure 130: Step 2: Connect and Test JavaScript

Step 3: Declare Student Variables

Add student information to script.js:

```
JS script.js > ...
1  | const studentName = "Ali";
2  | const registrationNumber = "BCS221001";
3  |
4  | let subject1 = 85;
5  | let subject2 = 78;
6  | let subject3 = 92;
7  |
8  | console.log(studentName);
9  | console.log(registrationNumber);
10 | console.log(subject1);
11 | console.log(subject2);
12 | console.log(subject3);
13 |
```

Here, `const` is used for information that will not be reassigned, while `let` is used for marks that could be changed

Step 4: Calculate Total and Average

Use arithmetic operators to calculate the total and average:

```
JS script.js > ...
1  | const totalMarks = subject1 + subject2 + subject3;
2  | const averageMarks = totalMarks / 3;
3  |
4  | console.log("Total Marks:", totalMarks);
5  | console.log("Average Marks:", averageMarks);
6  |
```

Figure 131: Step 4: Calculate Total and Average

- Run it and Check the output on the console.

Step 5: Check the Student's Result

Use an if...else statement to determine whether the student has passed.

```
JS script.js
1  if (averageMarks >= 50) {
2      console.log("Result: Pass");
3  } else {
4      console.log("Result: Fail");
5  }
6
```

- Run it and Check the output on the console.

Step 6: Assign a Grade

Extend the program using else if statements.

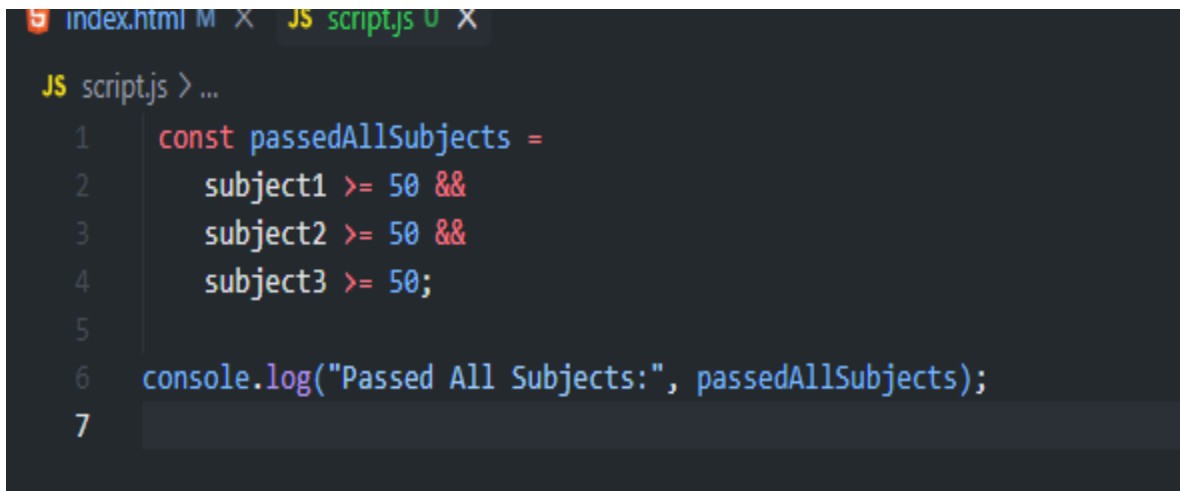
```
index.html M X JS script.js U X
JS script.js
1  if (averageMarks >= 90) {
2      console.log("Grade: A+");
3  } else if (averageMarks >= 80) {
4      console.log("Grade: A");
5  } else if (averageMarks >= 70) {
6      console.log("Grade: B");
7  } else if (averageMarks >= 60) {
8      console.log("Grade: C");
9  } else if (averageMarks >= 50) {
10     console.log("Grade: D");
11 } else {
12     console.log("Grade: F");
13 }
14
```

Figure 132: Step 6: Assign a Grade

- Run it and Check the output on the console.

Step 7: Use Comparison and Logical Operators

The program can also check whether all subjects have marks of at least 50.

A screenshot of a code editor with two tabs: 'index.html M' and 'JS script.js U'. The 'JS script.js' tab is active, showing the following code:

```
JS script.js > ...
1  const passedAllSubjects =
2      subject1 >= 50 &&
3      subject2 >= 50 &&
4      subject3 >= 50;
5
6  console.log("Passed All Subjects:", passedAllSubjects);
7
```

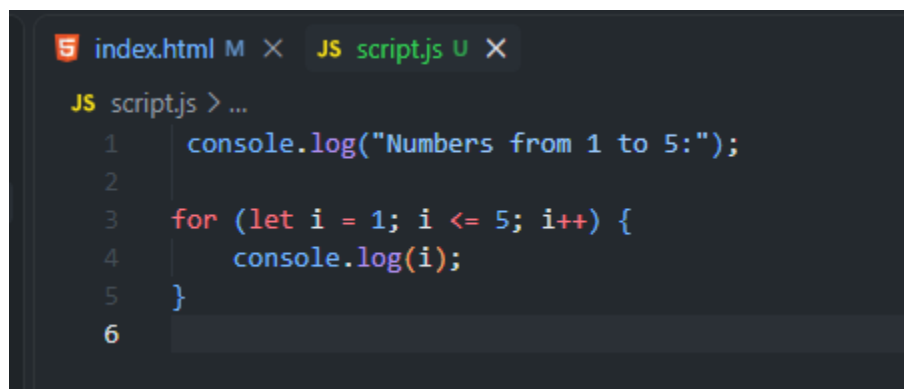
Figure 133: Step 7: Use Comparison and Logical Operators

➤ Run it and Check the output on the console.

The && operator requires all three conditions to be true.

Step 8: Use a for Loop

Use a for loop to display numbers from 1 to 5:

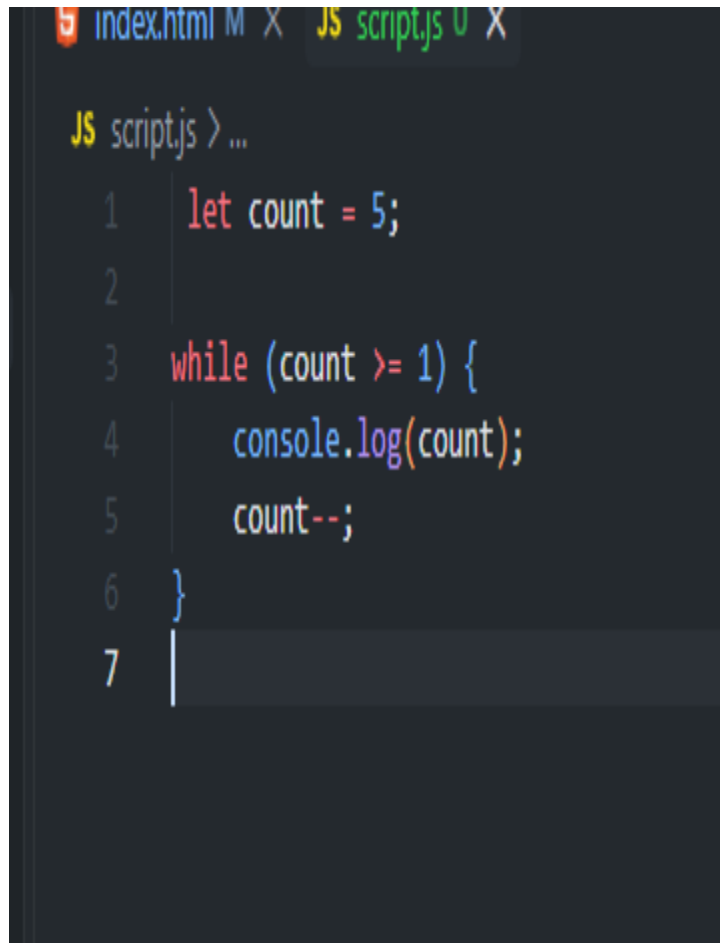
A screenshot of a code editor with two tabs: 'index.html M' and 'JS script.js U'. The 'JS script.js' tab is active, showing the following code:

```
JS script.js > ...
1  console.log("Numbers from 1 to 5:");
2
3  for (let i = 1; i <= 5; i++) {
4      console.log(i);
5  }
6
```

➤ Run it and Check the output on the console.

Step 9: Use a while Loop

Now use a while loop to display a countdown:

A screenshot of a code editor with a dark background. At the top, there are two tabs: 'index.html' and 'script.js'. The 'script.js' tab is active. Below the tabs, the text 'JS script.js > ...' is visible. The code is as follows:

```
1 let count = 5;
2
3 while (count >= 1) {
4   console.log(count);
5   count--;
6 }
7
```

➤ Run it and Check the output on the console.

Step 10: Complete the Program

After completing the previous steps, the script.js file should contain:

```

JS script.js > ...
1  |  const studentName = "Ali";
2  |  const registrationNumber = "BCS221001";
3  |
4  |  let subject1 = 85;
5  |  let subject2 = 78;
6  |  let subject3 = 92;
7  |
8  |  console.log("Student:", studentName);
9  |  console.log("Registration Number:", registrationNumber);
10 |
11 |  const totalMarks = subject1 + subject2 + subject3;
12 |  const averageMarks = totalMarks / 3;
13 |
14 |  console.log("Total Marks:", totalMarks);
15 |  console.log("Average Marks:", averageMarks);
16 |
17 |  if (averageMarks >= 90) {
18 |      console.log("Grade: A+");
19 |  } else if (averageMarks >= 80) {
20 |      console.log("Grade: A");
21 |  } else if (averageMarks >= 70) {
22 |      console.log("Grade: B");
23 |  } else if (averageMarks >= 60) {
24 |      console.log("Grade: C");
25 |  } else if (averageMarks >= 50) {
26 |      console.log("Grade: D");
27 |  } else {
28 |      console.log("Grade: F");
29 |  }
30 |
31 |  const passedAllSubjects =
32 |      subject1 >= 50 &&
33 |      subject2 >= 50 &&
34 |      subject3 >= 50;
35 |
36 |  console.log("Passed All Subjects:", passedAllSubjects);
37 |
38 |  console.log("Numbers from 1 to 5:");
39 |
40 |  for (let i = 1; i <= 5; i++) {
41 |      console.log(i);
42 |  }
43 |

```

Figure 134: Step 10: Complete the Program

The Console should display the student's information, total marks, average, grade, pass status, and numbers generated by the loop.

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in the folder specified by the lab instructor.

Practice Task 1

Modify the Student Result Calculator so that it uses **five subjects** instead of three and calculates the total and average marks.

Practice Task 2

Create a JavaScript program that stores a student's marks and displays:

- "Excellent" if the marks are 90 or above.
- "Good" if the marks are between 70 and 89.
- "Needs Improvement" if the marks are between 50 and 69.
- "Fail" if the marks are below 50.

Practice Task 3

Write a JavaScript program that uses a for loop to display the multiplication table of 5 from 5×1 to 5×10 .

The expected output should follow the format:

$5 \times 1 = 5$

$5 \times 2 = 10$

...

$5 \times 10 = 50$

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	JavaScript programs must run without errors and produce the expected output.	30%	Programs will be executed and their output will be checked.
Variables and Data Types	Variables must be declared appropriately using let and const, and correct data types must be used.	20%	Code review of variable declarations and data types.
Operators and Conditional Logic	Arithmetic, comparison, logical operators, and conditional statements must be applied correctly.	25%	Code review and testing with different values.
Loops	for and while loops must be implemented correctly according to the task requirements.	15%	Execution and review of loop output.
Presentation and Code Quality	Code should be properly formatted, clearly organized, and use meaningful variable names.	10%	Review of code formatting and submitted screenshots.
Total	-	100%	-

Lab 06: JavaScript Functions & Arrays

Background

JavaScript functions and arrays are fundamental building blocks for developing interactive web applications. Functions allow developers to organize reusable pieces of code, while arrays provide a convenient way to store and process collections of data.

In this lab, students will work with regular functions, parameters, return values, arrow functions, arrays, objects, and commonly used array methods. These concepts will be combined to process practical data such as students, products, and employees.

JavaScript Functions

A function is a reusable block of code designed to perform a particular task. A function can accept values through parameters and can return a result.

Defining a Function

A JavaScript function can be defined using the function keyword:

```
function greetUser() {  
    console.log("Welcome to Web Development Lab");  
}
```

The function can be called using its name:

greetUser();

Function Parameters

Parameters allow information to be passed into a function:

```
function greetUser(name) {  
    console.log("Welcome " + name);  
}  
  
greetUser("Afnan");
```

Returning a Value

A function can return a result using the return statement:

```
function addNumbers(a, b) {  
    return a + b;  
}  
let result = addNumbers(10, 20);  
console.log(result);
```

The returned value can be stored in a variable or used in another expression.

Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

A normal function can be written as:

```
function square(number) {  
    return number * number;  
}
```

The same function can be written as an arrow function:

```
const square = (number) => {  
    return number * number;  
};
```

For a single expression, the function can be shortened further:

```
const square = number => number * number;
```

Arrow functions are commonly used with array methods such as **map()**, **filter()**, and **forEach()**.

Arrays

An array is used to store multiple values in a single variable.

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];
```

Array elements are accessed using their index:

```
console.log(fruits[0]);  
console.log(fruits[2]);
```

The first element has index 0.

The length property returns the number of elements:

```
console.log(fruits.length);
```

New elements can be added using **push()**:

```
fruits.push("Grapes");|
```

An element can be removed from the end using **pop()**:

```
fruits.pop();
```

JavaScript Objects

Objects store related information using properties and values.

```
let student = {  
  name: "Ali",  
  age: 21,  
  marks: 85  
};
```

Object properties can be accessed using dot notation:

```
console.log(student.name);  
console.log(student.marks);
```

An array can also contain multiple objects:

```
let students = [  
  { name: "Ali", marks: 85 },  
  { name: "Sara", marks: 92 },  
  { name: "Ahmed", marks: 68 }  
];
```

This structure is useful for representing real-world collections of data.

Array Methods

JavaScript provides several methods for processing array data.

`forEach()`

`forEach()` executes a function for every element in an array.

```
students.forEach(student => {  
  console.log(student.name);  
});
```

`map()`

`map()` creates a new array by applying a function to every element.

```
let names = students.map(student => student.name);
```

`filter()`

`filter()` creates a new array containing only elements that satisfy a condition.

```
let passedStudents = students.filter(student => student.marks >= 50);
```

`find()`

`find()` returns the first element that satisfies a condition.

```
let student = students.find(student => student.name === "Sara");
```

reduce()

reduce() can be used to calculate a single result from an array.

```
let totalMarks = students.reduce((total, student) => {  
  return total + student.marks;  
}, 0);
```

sort()

sort() can be used to arrange array elements.

For numerical values:

```
let marks = [65, 90, 45, 78];  
marks.sort((a, b) => a - b);
```

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Review the syntax of JavaScript functions and write two examples of functions that accept parameters and return values.

Home Task 02

Create an array containing five products and practice using `push()`, `pop()`, and the `length` property.

Home Task 03

Review the difference between `forEach()`, `map()`, and `filter()`, and write one example of each.

Lab Objectives

The students will be able to:

- Define and call JavaScript functions.
- Use parameters and return values in functions.
- Create and use arrow functions.

- Create and manipulate JavaScript arrays.
 - Create objects and arrays containing objects.
 - Use `forEach()`, `map()`, `filter()`, `find()`, `reduce()`, and `sort()`.
 - Combine functions and array methods to process structured data.
 - Apply JavaScript functions and arrays to practical web-development problems.
-

Walkthrough Task

Task 1: Functions and Arrow Functions

Objective

To create reusable JavaScript functions using parameters, return values, and arrow-function syntax.

Description

Create a JavaScript program that calculates the total price of an item based on its quantity and unit price. Implement the calculation using a normal function and then create an arrow function that performs the same operation.

Steps

Step 1: Create an HTML page and include a JavaScript file.

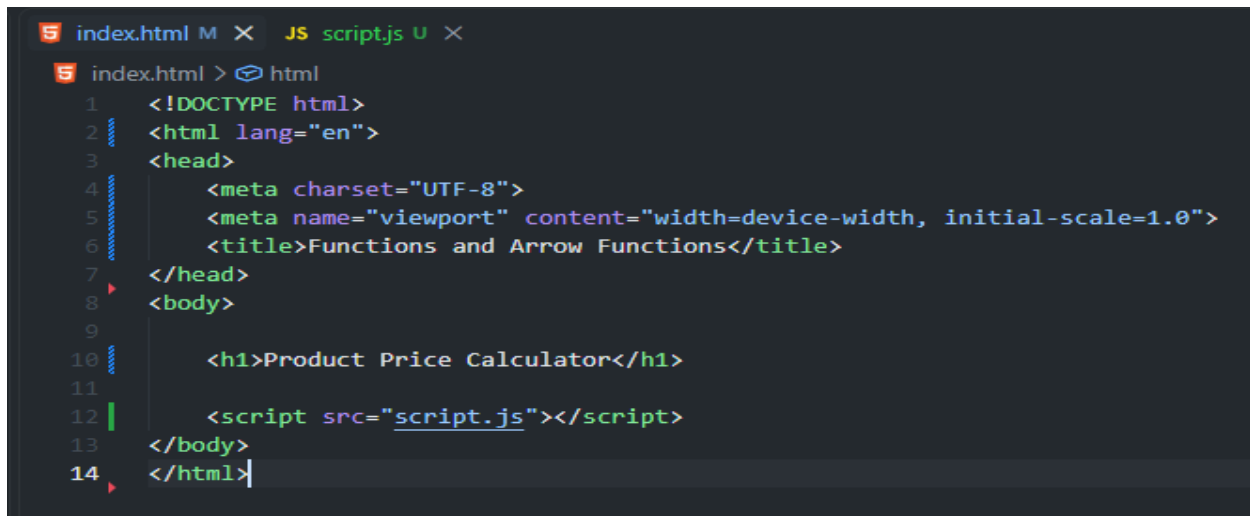
Step 2: Create a function named `calculateTotal` that accepts price and quantity as parameters.

Step 3: Return the total price from the function.

Step 4: Call the function with suitable values and display the result in the browser console.

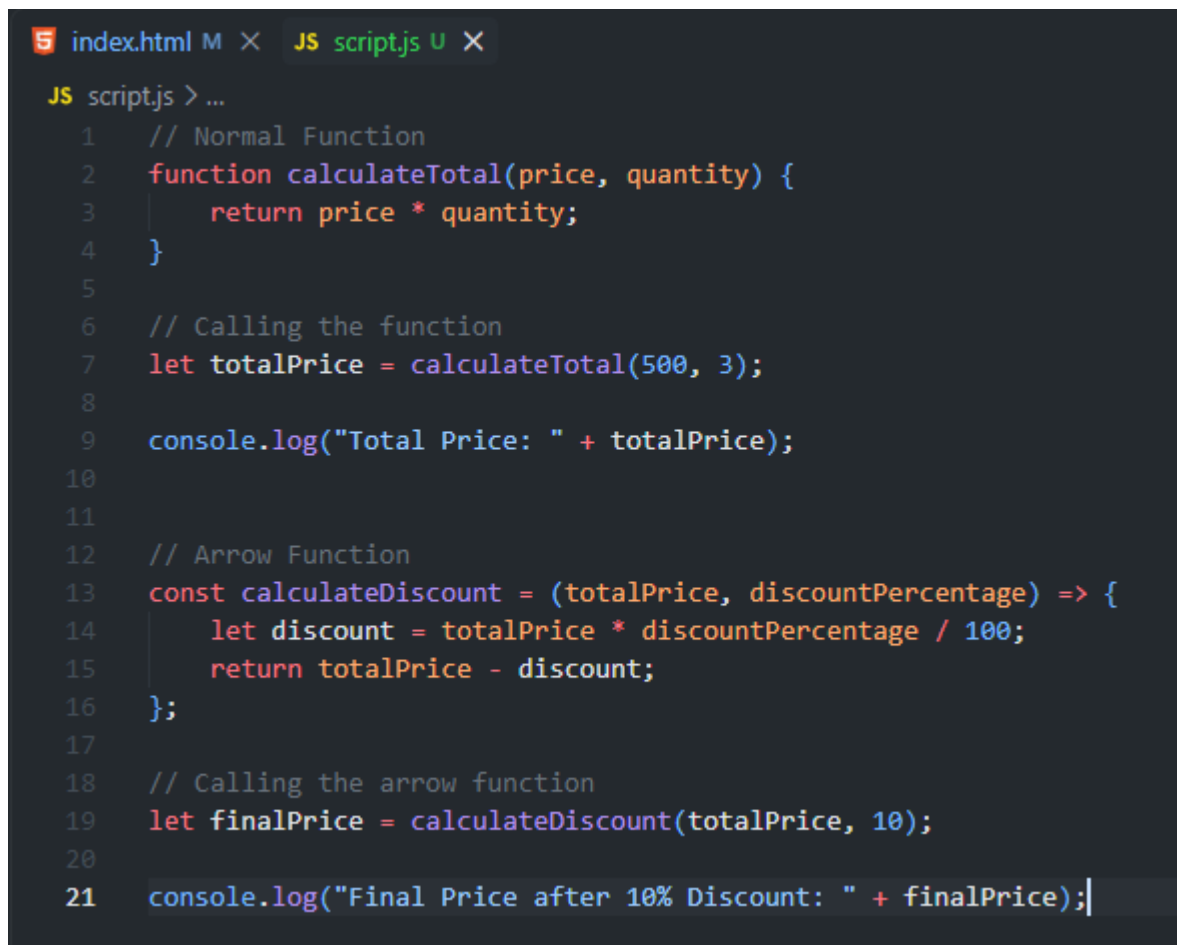
Step 5: Create an arrow function named `calculateDiscount` that accepts the total price and discount percentage.

Step 6: Calculate the discounted price and display the result.

A screenshot of a code editor with two tabs: 'index.html' and 'JS script.js'. The 'index.html' tab is active, showing the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Functions and Arrow Functions</title>
7 </head>
8 <body>
9
10  <h1>Product Price Calculator</h1>
11
12  <script src="script.js"></script>
13 </body>
14 </html>
```

Figure 135: Steps

A screenshot of a code editor with two tabs: 'index.html' and 'JS script.js'. The 'JS script.js' tab is active, showing the following JavaScript code:

```
JS script.js > ...
1 // Normal Function
2 function calculateTotal(price, quantity) {
3   return price * quantity;
4 }
5
6 // Calling the function
7 let totalPrice = calculateTotal(500, 3);
8
9 console.log("Total Price: " + totalPrice);
10
11
12 // Arrow Function
13 const calculateDiscount = (totalPrice, discountPercentage) => {
14   let discount = totalPrice * discountPercentage / 100;
15   return totalPrice - discount;
16 };
17
18 // Calling the arrow function
19 let finalPrice = calculateDiscount(totalPrice, 10);
20
21 console.log("Final Price after 10% Discount: " + finalPrice);
```

Figure 136: Steps

Output:

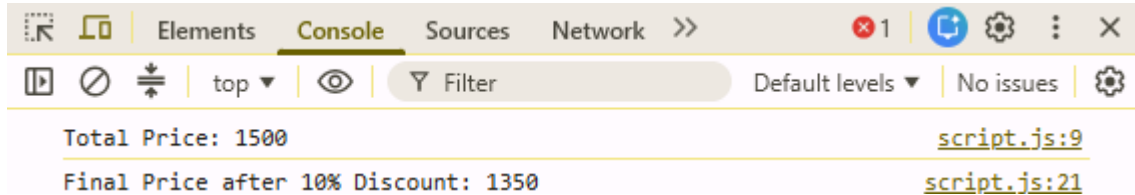


Figure 137: Steps

Task 2: Arrays and Array Methods

Objective

To create arrays and process their elements using common JavaScript array methods.

Description

Create an array containing the marks of several students. Use array methods to display the marks, create a new array, find students who passed, and calculate the total marks.

Steps

Step 1: Create an array containing at least six student marks.

Step 2: Use `forEach()` to display each mark in the console.

Step 3: Use `map()` to create a new array containing the marks increased by 5.

Step 4: Use `filter()` to create an array containing only marks greater than or equal to 50.

Step 5: Use `find()` to find the first mark greater than or equal to 80.

Step 6: Use `reduce()` to calculate the total of all marks.

Step 7: Display all results in the console.

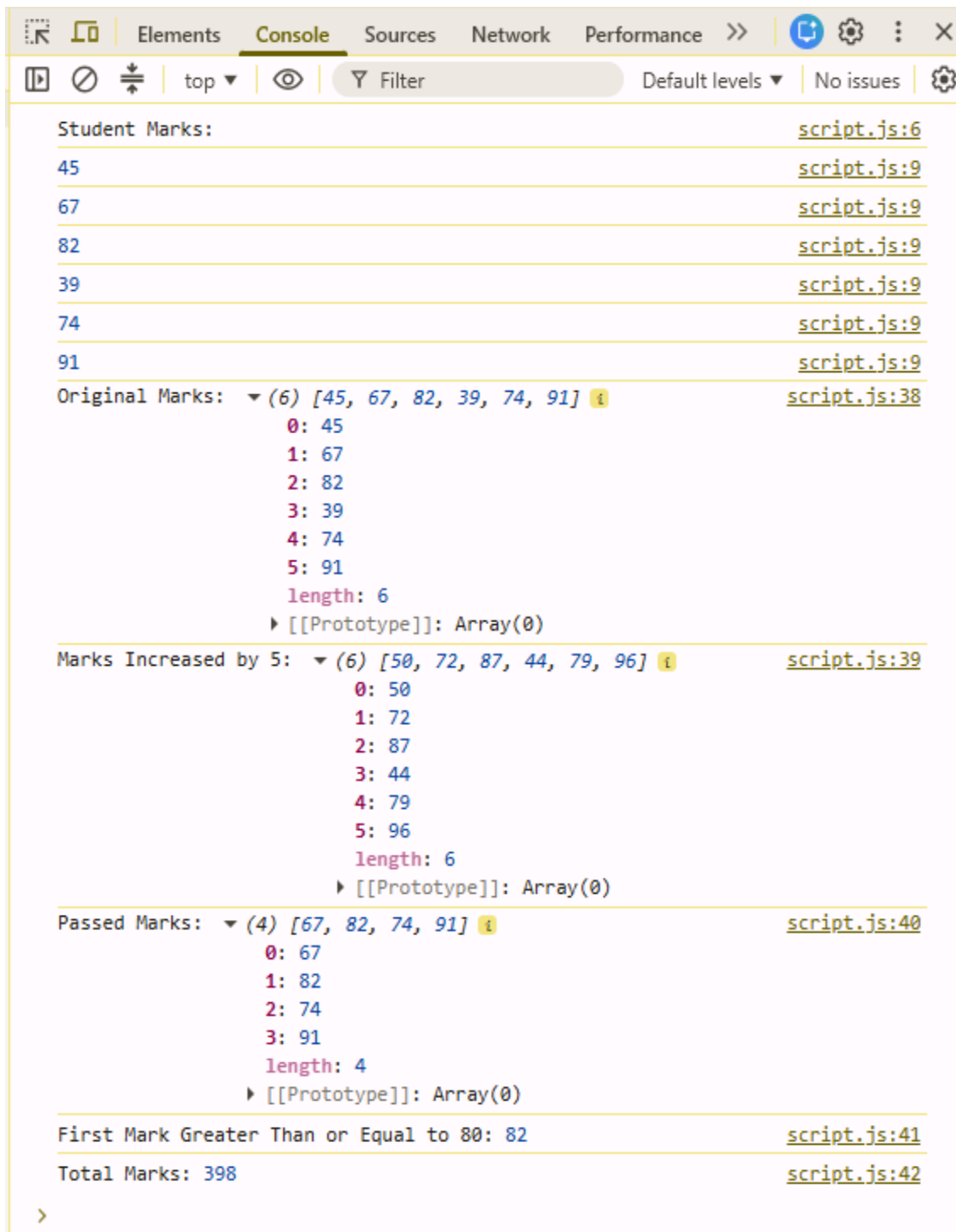
index.html M X JS script.js U X

JS script.js > ...

```
1 // Step 1: Create an array containing student marks
2 let marks = [45, 67, 82, 39, 74, 91];
3
4
5 // Step 2: Use forEach() to display each mark
6 console.log("Student Marks:");
7
8 marks.forEach((mark) => {
9     console.log(mark);
10 });
11
12
13 // Step 3: Use map() to increase each mark by 5
14 let increasedMarks = marks.map((mark) => {
15     return mark + 5;
16 });
17
18
19 // Step 4: Use filter() to find marks greater than or equal to 50
20 let passedMarks = marks.filter((mark) => {
21     return mark >= 50;
22 });
23
24
25 // Step 5: Use find() to find the first mark greater than or equal to 80
26 let firstHighMark = marks.find((mark) => {
27     return mark >= 80;
28 });
29
30
31 // Step 6: Use reduce() to calculate the total marks
32 let totalMarks = marks.reduce((total, mark) => {
33     return total + mark;
34 }, 0);
35
36
37 // Step 7: Display all results
38 console.log("Original Marks:", marks);
39 console.log("Marks Increased by 5:", increasedMarks);
40 console.log("Passed Marks:", passedMarks);
41 console.log("First Mark Greater Than or Equal to 80:", firstHighMark);
42 console.log("Total Marks:", totalMarks);
```

Figure 138: Steps

Output:



```
Student Marks: script.js:6
45 script.js:9
67 script.js:9
82 script.js:9
39 script.js:9
74 script.js:9
91 script.js:9
Original Marks: (6) [45, 67, 82, 39, 74, 91] script.js:38
  0: 45
  1: 67
  2: 82
  3: 39
  4: 74
  5: 91
  length: 6
  [[Prototype]]: Array(0)
Marks Increased by 5: (6) [50, 72, 87, 44, 79, 96] script.js:39
  0: 50
  1: 72
  2: 87
  3: 44
  4: 79
  5: 96
  length: 6
  [[Prototype]]: Array(0)
Passed Marks: (4) [67, 82, 74, 91] script.js:40
  0: 67
  1: 82
  2: 74
  3: 91
  length: 4
  [[Prototype]]: Array(0)
First Mark Greater Than or Equal to 80: 82 script.js:41
Total Marks: 398 script.js:42
>
```

Figure 139: Steps

Task 03: Student Result Processor

Build a JavaScript program that processes the results of a group of students.

The program should contain an array of student objects, where each student has a name, age, and marks. Create functions to calculate the average marks and determine whether a student has passed. Use array methods to display names, filter passing students, and calculate the average marks.

Step 1: Create the Student Array

Create an array containing at least five student objects.

Example structure:

```
const students = [  
  { name: "Ali", age: 20, marks: 75 },  
  { name: "Sara", age: 21, marks: 88 },  
  { name: "Ahmed", age: 19, marks: 45 }  
];
```

Step 2: Create a Passing Function

Create a function named `isPassed()` that accepts marks as a parameter and returns true if the marks are 50 or above.

```
function isPassed(marks) {  
  return marks >= 50;  
}
```

Step 3: Create an Arrow Function

Create an arrow function named `calculateAverage` that calculates the average marks of the students.

Step 4: Display Student Names

Use `forEach()` to display each student's name in the console.

Step 5: Find Passing Students

Use `filter()` together with the `isPassed()` function to create a new array containing only students who passed.

Step 6: Create a Names Array

Use `map()` to create a new array containing only the names of the students.

Step 7: Calculate Total Marks

Use `reduce()` to calculate the total marks obtained by all students.

Step 8: Sort the Students

Use `sort()` to arrange the students according to their marks from highest to lowest.

Step 9: Display the Results

Display the following information in the browser console:

- All student names
- Passing students
- Student names array
- Total marks
- Average marks
- Students sorted by marks

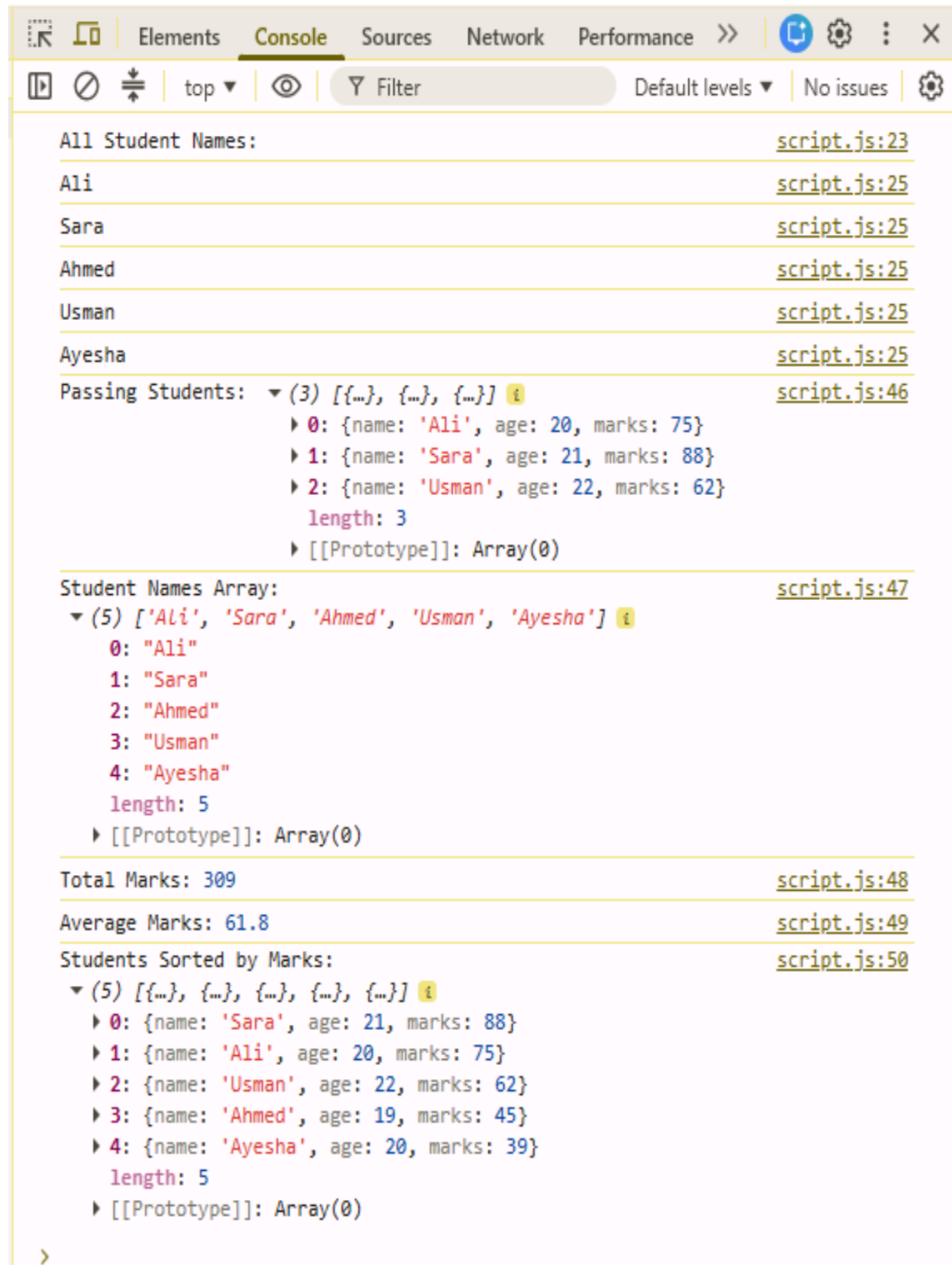
```

index.html M X JS script.js U X
JS script.js > ...
1 // script.js
2 // Step 1: Create the Student Array
3 const students = [
4     { name: "Ali", age: 20, marks: 75 },
5     { name: "Sara", age: 21, marks: 88 },
6     { name: "Ahmed", age: 19, marks: 45 },
7     { name: "Usman", age: 22, marks: 62 },
8     { name: "Ayesha", age: 20, marks: 39 }
9 ];
10 // Step 2: Create a Passing Function
11 function isPassed(marks) {
12     return marks >= 50;
13 }
14 // Step 3: Create an Arrow Function
15 const calculateAverage = (students) => {
16     let total = students.reduce((sum, student) => {
17         return sum + student.marks;
18     }, 0);
19     return total / students.length;
20 };
21 // Step 4: Display Student Names
22 console.log("All Student Names:");
23 students.forEach((student) => {
24     console.log(student.name);
25 });
26 // Step 5: Find Passing Students
27 const passingStudents = students.filter((student) => {
28     return isPassed(student.marks);
29 });
30 // Step 6: Create a Names Array
31 const studentNames = students.map((student) => {
32     return student.name;
33 });
34 // Step 7: Calculate Total Marks
35 const totalMarks = students.reduce((total, student) => {
36     return total + student.marks;
37 }, 0);
38 // Step 8: Sort the Students
39 const sortedStudents = [...students].sort((a, b) => {
40     return b.marks - a.marks;
41 });
42 // Calculate Average Marks
43 const averageMarks = calculateAverage(students);
44 // Step 9: Display the Results
45 console.log("Passing Students:", passingStudents);
46 console.log("Student Names Array:", studentNames);
47 console.log("Total Marks:", totalMarks);
48 console.log("Average Marks:", averageMarks);
49 console.log("Students Sorted by Marks:", sortedStudents);
50

```

Figure 140: Step 9: Display the Results

Output:



```
All Student Names: script.js:23
Ali script.js:25
Sara script.js:25
Ahmed script.js:25
Usman script.js:25
Ayesha script.js:25
Passing Students: (3) [{...}, {...}, {...}] i script.js:46
  0: {name: 'Ali', age: 20, marks: 75}
  1: {name: 'Sara', age: 21, marks: 88}
  2: {name: 'Usman', age: 22, marks: 62}
  length: 3
  [[Prototype]]: Array(0)
Student Names Array: script.js:47
  (5) ['Ali', 'Sara', 'Ahmed', 'Usman', 'Ayesha'] i
  0: "Ali"
  1: "Sara"
  2: "Ahmed"
  3: "Usman"
  4: "Ayesha"
  length: 5
  [[Prototype]]: Array(0)
Total Marks: 309 script.js:48
Average Marks: 61.8 script.js:49
Students Sorted by Marks: script.js:50
  (5) [{...}, {...}, {...}, {...}, {...}] i
  0: {name: 'Sara', age: 21, marks: 88}
  1: {name: 'Ali', age: 20, marks: 75}
  2: {name: 'Usman', age: 22, marks: 62}
  3: {name: 'Ahmed', age: 19, marks: 45}
  4: {name: 'Ayesha', age: 20, marks: 39}
  length: 5
  [[Prototype]]: Array(0)
```

Figure 141: Step 9: Display the Results

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in a folder specified by your lab instructor.

Practice Task 1

Create an array of product objects containing name, price, and category.

Use:

- `forEach()` to display product names.
- `filter()` to find products with a price greater than 1000.
- `map()` to create an array containing product names.
- `reduce()` to calculate the total price of all products.

Practice Task 2

Create an array of employee objects containing name, designation, and salary.

Create a function that calculates a 10% salary bonus.

Use `map()` to create a new list containing each employee's name and updated salary after the bonus.

Then use `filter()` to find employees whose updated salary is greater than 50,000.

Practice Task 3

Create an array of student objects containing name, marks, and age.

Create functions to:

- Determine whether a student passed.
- Calculate the average marks.
- Find the student with the highest marks.

Use appropriate array methods to display the required results.

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The JavaScript program must run without errors and produce the required results.	35%	The program will be executed and the output will be checked against the task requirements.
Correct use of functions	Functions and arrow functions must be defined and used appropriately with parameters and return values.	25%	Code review of function definitions and their usage.
Array and object processing	Arrays, objects, and appropriate array methods must be used correctly to process the data.	20%	Testing forEach(), map(), filter(), find(), reduce(), and sort() where required.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	10%	Review of the submitted output and project folder.

Lab 07: DOM & Events

Background

Every HTML page loaded into a browser is turned into a live, in-memory tree of objects called the Document Object Model (DOM). While Lab 06 focused on processing data with functions and arrays, this lab focuses on the page itself — reading and changing what is on screen, and reacting to what the user does.

In this lab, students will work with selecting DOM elements, modifying their content and appearance, creating and removing elements dynamically, and handling browser events, including events that fire on forms and input fields. These concepts are combined to build small, interactive, event-driven web pages.

Selecting DOM Elements

Before anything on a page can be changed with JavaScript, it must first be selected — i.e. you need a JavaScript reference to the HTML element(s) you want to work with.

Using `getElementById()`

`getElementById()` returns a single element that matches the given id:

```
const heading = document.getElementById("pageTitle");
```

Using `querySelector()` and `querySelectorAll()`

`querySelector()` returns the first element matching a CSS selector. `querySelectorAll()` returns every matching element as a `NodeList`:

```
const email = document.querySelector("input[type='email']");  
const allNotes = document.querySelectorAll(".note"); // NodeList of all matches
```

Figure 142: Selecting DOM Elements

Tip: `querySelector()` and `querySelectorAll()` accept any valid CSS selector ("`#id`", "`.class`", "`ul li`" etc.), which makes them the most flexible and the ones used throughout this lab.

Modifying DOM Elements

Changing Text and HTML

Once selected, an element's content can be changed using `textContent` (plain text) or `innerHTML` (HTML markup):

```
heading.innerHTML = "<b>Bold!</b>"; // sets HTML markup – use carefully
heading.textContent = "New Title"; // sets plain text (safer, most common)
```

Figure 143: Modifying DOM Elements

Changing Styles, Attributes and Classes

```
heading.style.color = "red"; // one inline CSS property at a time
heading.setAttribute("title", "Hello"); // set any HTML attribute

heading.classList.add("active"); // add a CSS class
heading.classList.remove("active"); // remove a CSS class
heading.classList.toggle("active"); // add if missing, remove if present
```

Figure 144: Changing Styles, Attributes and Classes

Creating and Removing Elements

JavaScript can also build brand-new elements and insert them into the page, or remove existing ones:

```
const li = document.createElement("li"); // 1. create it
li.textContent = "Buy milk"; // 2. fill it in
document.querySelector("ul").appendChild(li); // 3. insert it into the DOM

li.remove(); // remove an element directly (modern browsers)
parent.removeChild(li); // older / more explicit way
```

Figure 145: Creating and Removing Elements

Handling Events

To react to something the user does, attach a listener function to an element with `addEventListener()`. The listener runs every time that event fires on that element.

```
element.addEventListener("click", function (event) {
  console.log("Clicked:", event.target);
});
```

any

Figure 146: Handling Events

Common Event Types

Event	Fires When...
click	the element is clicked (mouse or tap)
mouseover / mouseout	the pointer enters / leaves the element
keydown / keyup	a key is pressed down / released
load	the page (or an image) has finished loading

The Event Object

Every listener function automatically receives an event object. `event.target` tells you exactly which element triggered the event — useful when the same listener is reused for many elements, and `event.key` tells you which key was pressed during a keyboard event.

Form and Input Events

The Input Event

The input event fires on every keystroke inside an `<input>` or `<textarea>` — perfect for live validation or live previews:

```
nameInput.addEventListener("input", function (event) {  
    cardName.textContent = event.target.value || "Your Name";  
});
```

Figure 147: Form and Input Events

The Submit Event and `preventDefault()`

Clicking a submit button (or pressing Enter inside a form) fires a submit event on the `<form>` — not a click event. By default, submitting a form reloads the page. Calling `event.preventDefault()` stops that so the data can be handled with JavaScript instead:

```
document.querySelector("form").addEventListener("submit", function (event) {  
    event.preventDefault();           // stop the page from reloading  
    console.log("Form submitted!");  
});
```

Figure 148: Form and Input Events

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Review the three main ways to select an element (`getElementById`, `querySelector`, `querySelectorAll`) and write one short example of each.

Home Task 02

Create a web page with one button labelled "Click Me" and one paragraph that says "Hello!". Using JavaScript, change the paragraph's text to "You clicked the button!" whenever the button is clicked.

Home Task 03

Review the difference between the click, input and submit events, and write one short example of each.

Lab Objectives

The students will be able to:

- Select DOM elements using `getElementById`, `querySelector` and `querySelectorAll`.
- Modify element content, inline styles, attributes and CSS classes.
- Create new DOM elements and insert or remove them dynamically.
- Attach event listeners using `addEventListener` and use the event object.
- Handle keyboard and mouse events such as click and keydown.
- Handle form submission and input events, including `preventDefault()`.
- Combine the above to build small, interactive, event-driven web pages.

Walkthrough Task

Task 1: Selecting and Modifying DOM Elements

Objective: To select DOM elements using different selector methods and modify their content, styles and classes.

Description: Create a page with a heading and three short notes. Add a button that, when clicked, selects all the notes at once and highlights them, and updates the heading text to show how many notes were highlighted.

Steps

Step 1: Create an HTML page with a heading, three `<p class="note">` elements, and a button.

Step 2: In script.js, select the heading with `querySelector`, the button with `querySelector`, and all notes at once with `querySelectorAll`.

Step 3: Add a "click" listener on the button.

Step 4: Inside the listener, loop over the `NodeList` of notes with `forEach()` and add a "highlight" class to each one using `classList.add()`.

Step 5: Inside the same listener, update the heading's `textContent` to show the number of notes highlighted (`notes.length`).

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>querySelectorAll Example</title>
  <style>
    .note {
      padding: 10px;
      margin: 5px;
    }
    .highlight {
      background-color: yellow;
      font-weight: bold;
    }
  </style>
</head>
<body>

  <h1>My Notes</h1>

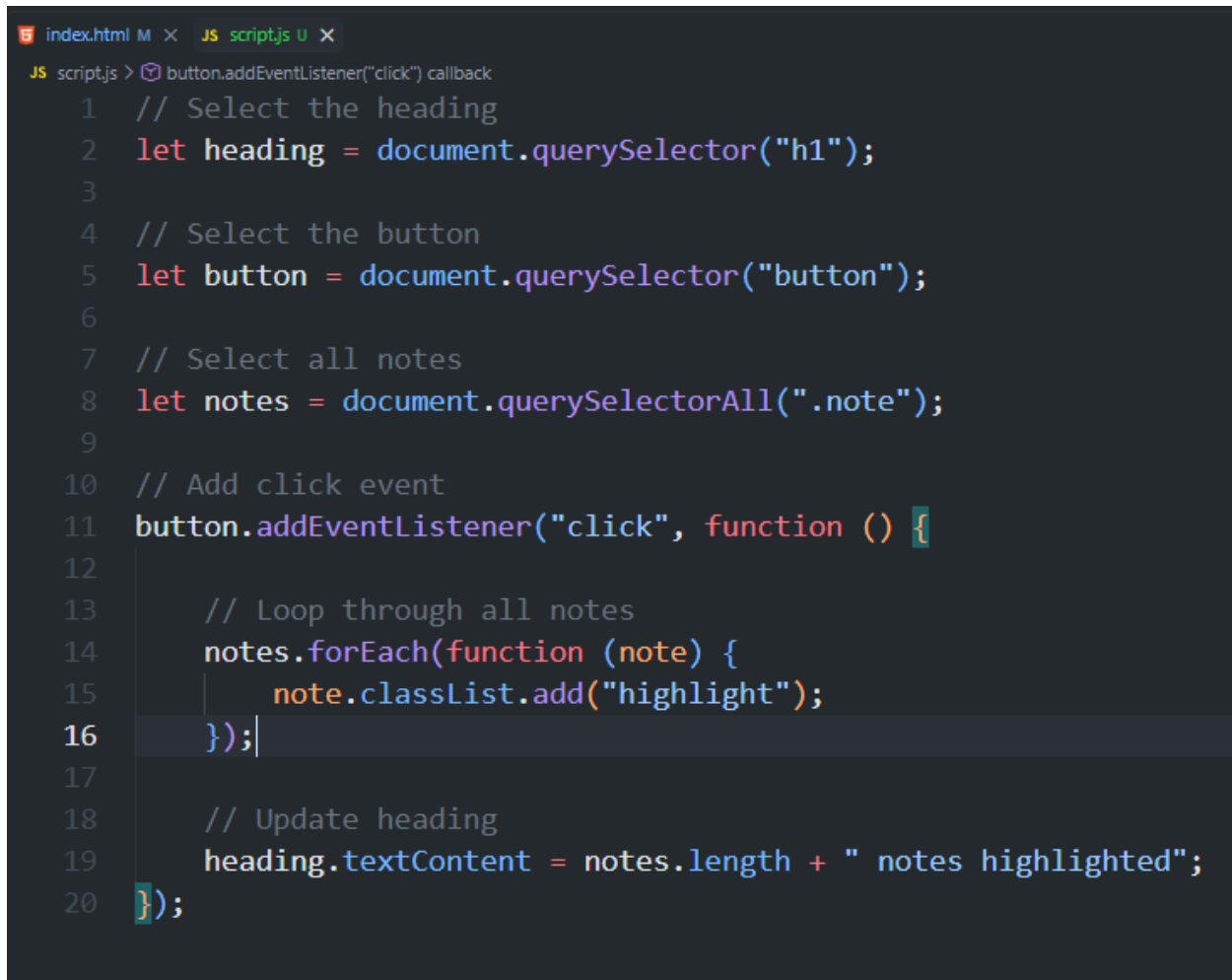
  <p class="note">Note 1: Learn JavaScript</p>
  <p class="note">Note 2: Practice DOM</p>
  <p class="note">Note 3: Learn querySelectorAll</p>

  <button id="highlightBtn">Highlight Notes</button>

  <script src="script.js"></script>
</body>
</html>
```

Figure 149: Index.html:

Script.js:



```
index.html M x JS script.js U x
JS script.js > button.addEventListener("click") callback
1 // Select the heading
2 let heading = document.querySelector("h1");
3
4 // Select the button
5 let button = document.querySelector("button");
6
7 // Select all notes
8 let notes = document.querySelectorAll(".note");
9
10 // Add click event
11 button.addEventListener("click", function () {
12
13     // Loop through all notes
14     notes.forEach(function (note) {
15         note.classList.add("highlight");
16     });
17
18     // Update heading
19     heading.textContent = notes.length + " notes highlighted";
20 });
```

Figure 150: Index.html:

Output:

3 notes highlighted

Note 1: Learn JavaScript

Note 2: Practice DOM

Note 3: Learn querySelectorAll

Highlight Notes

Figure 151: Index.html:

Task 2: Handling Click and Keyboard Events

Objective: To attach event listeners for mouse and keyboard events and use the event object.

Description: Build a coloured box that keeps a click count and changes colour on every click, plus a small indicator that shows the last key pressed anywhere on the page.

Steps

Step 1: Create a box element with a starting click count in it, and a paragraph to show the last key pressed.

Step 2: Select the box and the key-status element.

Step 3: Keep a clicks variable, starting at 0.

Step 4: Add a "click" listener on the box: increment clicks, update the box's textContent, and change box.style.backgroundColor using a small array of colours indexed by the click count.

Step 5: Add a "keydown" listener on document (not just the box) so any key press anywhere on the page is detected, and display event.key in the key-status element.

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Click and Key Events</title>
<style>
  body {
    font-family: Segoe UI, Arial, sans-serif;
    background: #f4f6fb;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding-top: 40px;
    margin: 0;
    gap: 16px;
  }
  .box {
    width: 160px;
    height: 100px;
    background: #4f46e5;
    border-radius: 12px;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #fff;
    font-weight: bold;
    font-size: 18px;
    cursor: pointer;
    box-shadow: 0 6px 16px rgba(0,0,0,.15);
  }
  p {
    color: #374151;
    font-size: 14px;
    margin: 0;
  }
  #keyStatus {
    font-weight: 600;
    color: #4f46e5;
  }
</style>
</head>
<body>

  <div class="box" id="clickBox">Clicks: 0</div>
  <p>Click the box – it will count clicks and change colour.</p>
  <p>Last key pressed: <span id="keyStatus">—</span></p>
```

Figure 152: Index.html:

```

<script>
  // Step 3: colours to cycle through, and element references
  const colors = ["#38bdf8", "#4f46e5", "#f472b6", "#34d399", "#fbbf24"];
  const box = document.querySelector("#clickBox");
  const keyStatus = document.querySelector("#keyStatus");
  let clicks = 0;

  // Step 4: click listener – count clicks, update text, cycle colour
  box.addEventListener("click", function () {
    clicks++;
    box.textContent = "Clicks: " + clicks;
    box.style.background = colors[clicks % colors.length];
  });

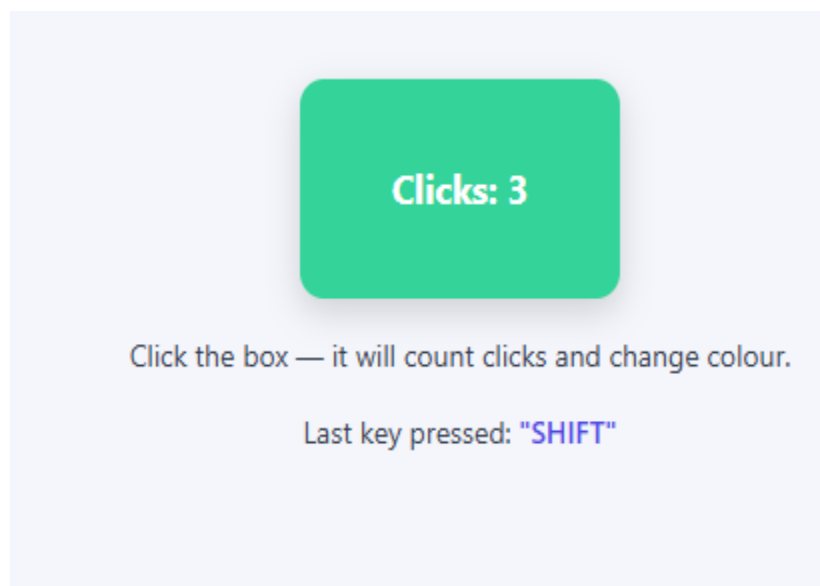
  // Step 5: keydown listener on document – catches key presses anywhere on the page
  document.addEventListener("keydown", function (event) {
    keyStatus.textContent = '' + event.key.toUpperCase() + '';
  });
</script>

</body>
</html>

```

Figure 153: Index.html:

Output: box shows "Clicks: 3" in green after three clicks, and the key-status text shows "W" after that key is pressed.



Task 3: Building an Interactive Profile Card (Form & Input Events)

Objective: To combine selecting, modifying and event handling into one page: update a live preview as the user types, and toggle a class on click.

Description: Build a small profile card editor: two text inputs (name and bio) update a live preview card as the user types, and a button toggles dark mode by adding/removing a class on <body>.

Steps

Step 1: Create two labelled inputs (name, bio), a "Toggle Dark Mode" button, and a card containing a name heading and a bio paragraph.

Step 2: Select both inputs, both card elements, and the button, once, at the top of script.js.

Step 3: Add an "input" listener on each field. Inside it, set the matching card element's textContent to event.target.value (falling back to placeholder text if empty).

Step 4: In your CSS, define the page's colours as CSS variables under :root, then redefine them under a body.dark rule.

Step 5: Add a "click" listener on the button that calls document.body.classList.toggle("dark").

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Profile Card Editor</title>
<style>
/* Step 4: colours as CSS variables, redefined for dark mode */
:root {
  --bg: #f4f6fb;
  --card-bg: #ffffff;
  --text: #1f2937;
  --accent: #4f46e5;
}
body.dark {
  --bg: #111827;
  --card-bg: #1f2937;
  --text: #f9fafb;
  --accent: #818cf8;
}

body {
  font-family: Segoe UI, Arial, sans-serif;
  background: var(--bg);
  color: var(--text);
  margin: 0;
  min-height: 100vh;
  display: flex;
  gap: 40px;
  justify-content: center;
  align-items: flex-start;
  padding: 40px;
  transition: background .3s, color .3s;
}
.panel { width: 280px; }
label { display: block; margin: 12px 0 4px; font-size: 14px; font-weight: 600; }
input {
  width: 100%;
  padding: 8px;
  border-radius: 6px;
  border: 1px solid #cbd5e1;
  box-sizing: border-box;
}
```

Figure 154: Index.html:

```

button {
  margin-top: 18px;
  padding: 10px 16px;
  background: var(--accent);
  color: #fff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 14px;
}

.card {
  width: 260px;
  background: var(--card-bg);
  border-radius: 14px;
  padding: 24px;
  box-shadow: 0 8px 20px rgba(0,0,0,.15);
  text-align: center;
}

.avatar {
  width: 70px;
  height: 70px;
  border-radius: 50%;
  background: var(--accent);
  margin: 0 auto 14px;
}

#cardName { margin: 0 0 6px; font-size: 20px; }
#cardBio { margin: 0; font-size: 14px; opacity: .8; }
</style>
</head>
<body>

  <!-- Step 1: inputs, button, and card -->
  <div class="panel">
    <label for="nameInput">Full Name</label>
    <input id="nameInput" type="text" placeholder="e.g. Ayesha Khan">

    <label for="bioInput">Short Bio</label>
    <input id="bioInput" type="text" placeholder="e.g. Frontend Developer">

    <button id="themeBtn">Toggle Dark Mode</button>
  </div>

  <div class="card">
    <div class="avatar"></div>
    <h3 id="cardName">Your Name</h3>
    <p id="cardBio">Your bio goes here</p>

```

Figure 155: Index.html:

```

<script>
  // Step 2: select every element once, at the top
  const nameInput = document.querySelector("#nameInput");
  const bioInput  = document.querySelector("#bioInput");
  const cardName  = document.querySelector("#cardName");
  const cardBio   = document.querySelector("#cardBio");
  const themeBtn  = document.querySelector("#themeBtn");

  // Step 3: update the card live as the user types
  nameInput.addEventListener("input", function (event) {
    cardName.textContent = event.target.value || "Your Name";
  });

  bioInput.addEventListener("input", function (event) {
    cardBio.textContent = event.target.value || "Your bio goes here";
  });

  // Step 5: toggle dark mode on click
  themeBtn.addEventListener("click", function () {
    document.body.classList.toggle("dark");
  });
</script>

</body>
</html>

```

Figure 156: Index.html:

Output: the card's name and bio update on every keystroke, and clicking "Toggle Dark Mode" switches the whole page to a dark colour scheme.

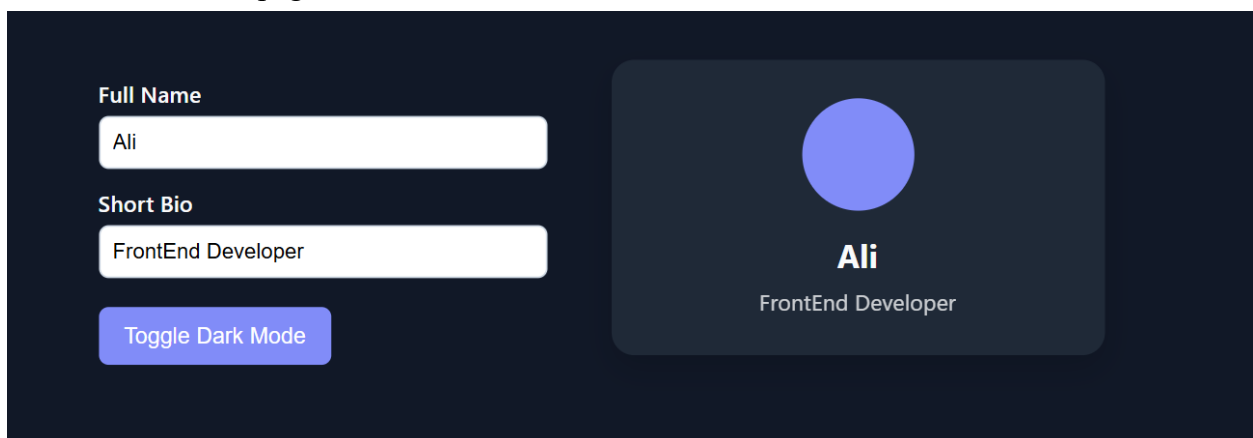


Figure 157: Index.html:

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in a folder specified by your lab instructor.

Practice Task 1: Color Switcher

Build a page with one large box and four small circular swatch buttons, each a different colour. Use:

- `querySelectorAll()` to select all four swatches at once.
- A `forEach()` loop to attach a "click" listener to each swatch.
- `event.target` inside the listener to read the swatch's own colour and apply it to the box's `style.backgroundColor`.

Practice Task 2: Dynamic To-Do List

Build a to-do list: a text input, an "Add" button, and a list. Typing a task and clicking "Add" (or pressing Enter) should add it as a new list item with a small delete "×" button.

Use:

- `document.createElement()` and `appendChild()` to build and insert each new item.
- A "keydown" listener that checks `event.key === "Enter"` so pressing Enter also adds the task.
- A "click" listener on each delete "×" that removes only its own list item (`event.target.parentElement`).

Practice Task 3: Live Registration Form Validation

Build a registration form with Name, Email and Password fields, plus a Submit button. Validate each field live as the user types (green border + hint if valid, red border + hint if not), keep Submit disabled until all fields are valid, and prevent the page from reloading on submit.

Use:

- An "input" listener on each field that toggles "ok"/"bad" classes with `classList.toggle()`.
 - A function that re-checks all three fields and enables/disables the Submit button whenever any field changes.
 - A "submit" listener on the `<form>` that calls `event.preventDefault()` and displays a success message.
-

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The JavaScript program must run without errors and produce the required DOM changes and interactions.	35%	The program will be executed and the output will be checked against the task requirements.
Correct use of selecting & modifying elements	Elements must be selected with the appropriate method and modified correctly (text, style, attributes, classes).	25%	Code review of selector usage and element modification.
Event handling	Event listeners, the event object, and form/input events (including preventDefault where required) must be used correctly.	20%	Testing of click, input, keydown and submit events.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	10%	Review of the submitted output and project folder.

Lab 08: React Basics

Background

React is a JavaScript library for building user interfaces out of small, reusable pieces called components. Where Lab 07 changed the DOM directly with `document.querySelector` and `addEventListener`, React takes a different approach: you describe what the UI should look like for a given set of data, and React handles updating the page for you whenever that data changes.

In this lab, students will set up a React project, write JSX, build functional components, pass data between components using props, and manage a component's own data using the `useState` hook. These concepts are combined to build small, interactive React components.

Setting Up a React Project

React projects are usually created with a build tool rather than by hand. Vite is fast and simple, so it's used throughout this lab:

Open Visual Studio Code and go to Terminal and type:

```
PS D:\> npm create vite@latest
PS D:\> npm create vite@latest
Need to install the following packages:
create-vite@9.2.1
Ok to proceed? (y) y

> npx
> create-vite

◇ Project name:
  my-app
◆ Select a framework:
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◇ Which linter to use?
  ESLint
◇ Install with npm and start now?
  Yes
◇ Scaffolding project in D:\my-app...
◇ Installing dependencies with npm...

added 143 packages, and audited 144 packages in 4m

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
◇ Starting dev server...

> my-app@0.0.0 dev
> vite

VITE v8.3.0 ready in 934 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
```

Figure 158: Setting Up a React Project

This creates a project folder with a few key files:

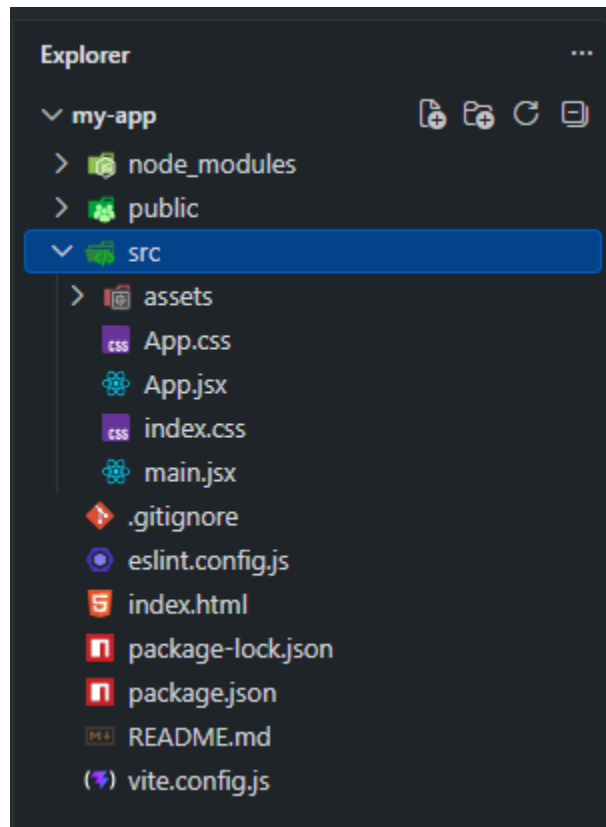


Figure 159: Setting Up a React Project

Tip: `npm run dev` starts a local development server (usually at `http://localhost:5173`) that automatically refreshes the browser every time you save a file.

JSX

JSX is a syntax extension that lets you write HTML-like markup directly inside JavaScript. The browser doesn't understand it directly — the build tool converts it into plain JavaScript behind the scenes.

A few rules that are different from plain HTML:

- A component must return a single parent element (or an empty `<>...</>` fragment) — you cannot return two sibling elements side by side.
- Use `className` instead of `class`, since `class` is a reserved word in JavaScript.
- JavaScript expressions can be embedded directly using curly braces `{ }`:

```
const name = "Ayesha";  
return <h2>Welcome, {name}!</h2>;  
}
```

Components

A component is just a JavaScript function that returns JSX. Component names always start with a capital letter, so React can tell them apart from regular HTML tags.

App.jsx:

```
App.jsx X  
src > App.jsx > default  
1  function App() {  
2    return (  
3      <div>  
4        <h1>Hello, React!</h1>  
5        <p>This is JSX.</p>  
6      </div>  
7    );  
8  }  
9  export default App;
```

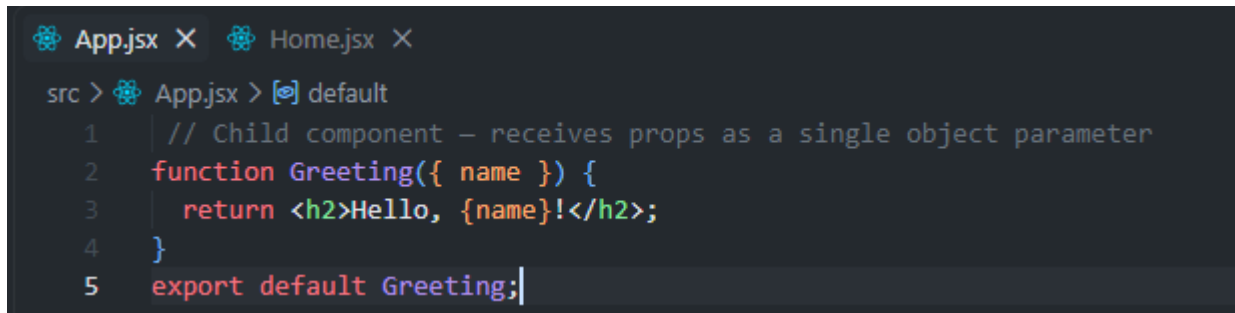
Components can be imported and used inside other components, just like nesting HTML tags:

Home.jsx

```
App.jsx X Home.jsx X  
src > Home.jsx > Home  
1  import App from "./App";  
2  
3  function Home() {  
4    return (  
5      <div>  
6        <App />  
7        <App />  
8      </div>  
9    );  
10 }  
11 export default Home;
```

Props

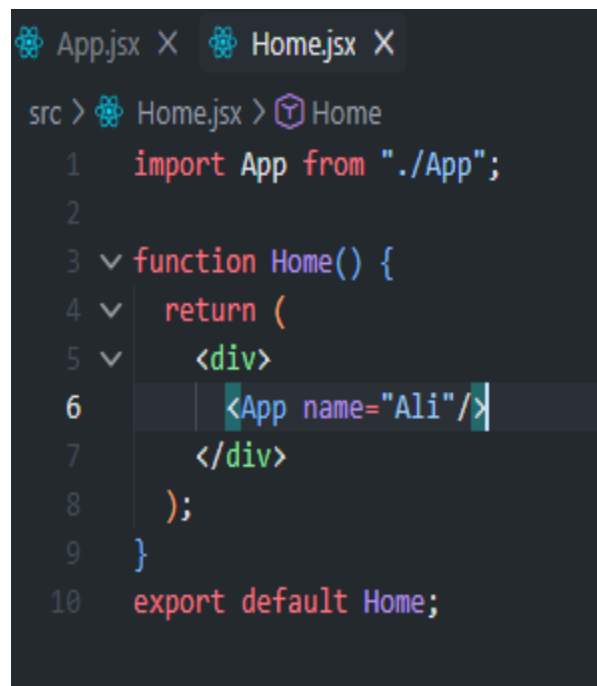
Props ("properties") let a parent component pass data down into a child component. Props are read-only inside the child — the child should never modify them directly.



```
App.jsx X Home.jsx X
src > App.jsx > [🔍] default
1 // Child component – receives props as a single object parameter
2 function Greeting({ name }) {
3   return <h2>Hello, {name}</h2>;
4 }
5 export default Greeting;
```

Figure 160: Props

// Parent component — passes data via JSX attributes



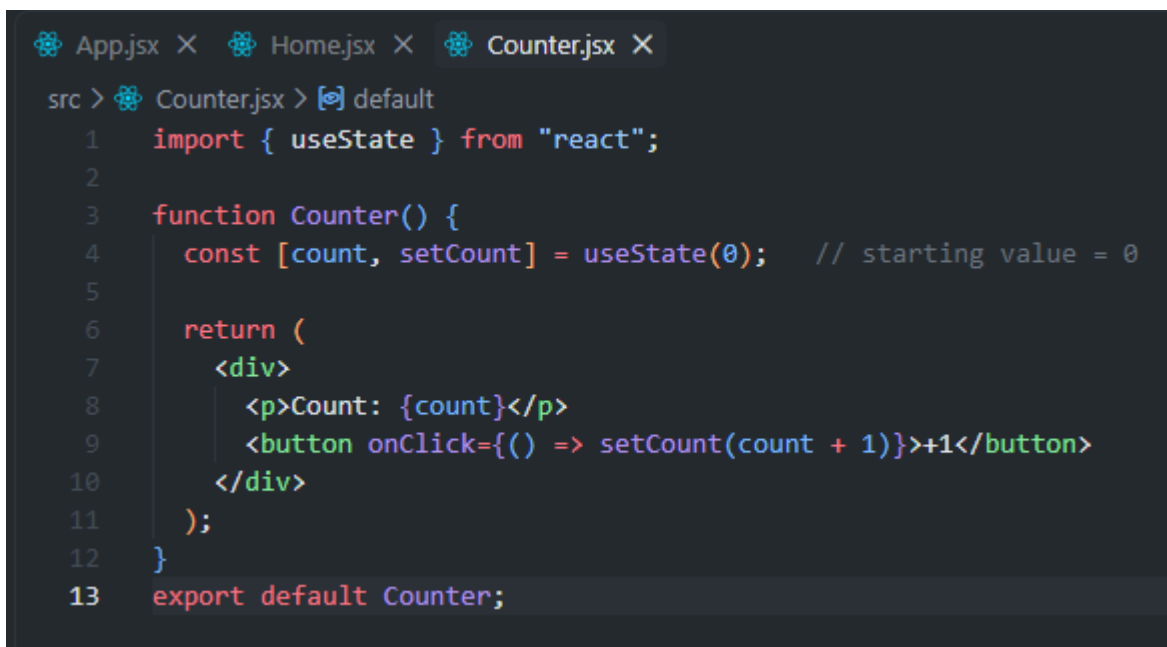
```
App.jsx X Home.jsx X
src > Home.jsx > 🏠 Home
1 import App from './App';
2
3 function Home() {
4   return (
5     <div>
6       <App name="Ali"/>
7     </div>
8   );
9 }
10 export default Home;
```

Tip: { name } in the function parameter is destructuring — it pulls the name value straight out of the props object, instead of writing props.name every time.

State (useState)

Props come from outside a component. State is data a component manages itself, and it can change over time — usually in response to user interaction. The useState hook is how a functional component gets its own state.

Counter.jsx



```
App.jsx X Home.jsx X Counter.jsx X
src > Counter.jsx > default
1  import { useState } from "react";
2
3  function Counter() {
4    const [count, setCount] = useState(0); // starting value = 0
5
6    return (
7      <div>
8        <p>Count: {count}</p>
9        <button onClick={() => setCount(count + 1)}>+1</button>
10     </div>
11   );
12 }
13 export default Counter;
```

Figure 161: State (useState)

- `useState(0)` returns two things: the current value (`count`) and a function to update it (`setCount`). Calling `setCount` with a new value tells React "this data changed" — React then automatically re-runs the component and updates the page to match.

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Install Node.js if not already installed, verify it with `node -v` and `npm -v`, then create a new React project using Vite (`npm create vite@latest`) and confirm it runs with `npm run dev`.

Home Task 02

Review three ways JSX differs from plain HTML (single root element, `className`, curly-brace expressions) and write one short JSX snippet demonstrating each.

Home Task 03

In your own words, write one sentence explaining the difference between props and state.

Lab Objectives

The students will be able to:

- Set up a new React project using Vite.
- Read and write JSX syntax correctly.
- Create functional React components.

- Pass data between components using props.
- Manage local component state using the useState hook.
- Combine props and state to build small, interactive React components.

Walkthrough Task

Task 1: Project Setup and Your First Component

Objective: To set up a React project and create a simple functional component using JSX.

Description: Create a new React project, then replace the default App content with a custom Welcome component that displays a heading and a short paragraph.

Steps

Step 1: Create a new project with `npm create vite@latest my-lab08 -- --template react`, then run `cd my-lab08`, `npm install`, and `npm run dev`.

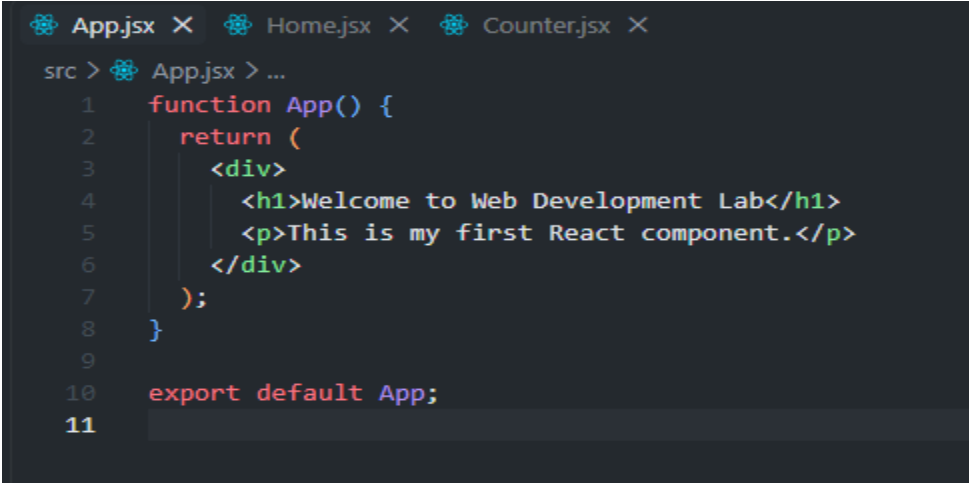
Step 2: Open `src/App.jsx` and delete the default starter content.

Step 3: Write a functional component named `Welcome` that returns a `<h1>` heading and a `<p>` paragraph inside a single parent `<div>`.

Step 4: Export `Welcome` as the default export of `App.jsx`, so Vite renders it.

Step 5: Save the file and check the browser — it should refresh automatically.

App.jsx:



```
App.jsx X Home.jsx X Counter.jsx X
src > App.jsx > ...
1  function App() {
2    return (
3      <div>
4        <h1>Welcome to Web Development Lab</h1>
5        <p>This is my first React component.</p>
6      </div>
7    );
8  }
9
10 export default App;
11
```

Output: the browser displays "Welcome to Web Development Lab" as a heading, with the paragraph text underneath it.

Welcome to Web Development Lab

This is my first React component.

Figure 162: Task 1: Project Setup and Your First Component

Task 2: Passing Data with Props

Objective: To pass data from a parent component to a child component using props.

Description: Build a ProfileCard component that accepts a name and a role as props, then render it three times in App with different data each time.

Steps

Step 1: Create a new file ProfileCard.jsx.

Step 2: Define a functional component ProfileCard that accepts { name, role } as its parameter.

Step 3: Return JSX that displays the name in a heading and the role in a paragraph, using {name} and {role}.

Step 4: Export ProfileCard as the default export, and import it into App.jsx.

Step 5: In App, render <ProfileCard /> three times, passing different name and role values as JSX attributes each time.

ProfileCard.jsx:

```
... App.jsx X ProfileCard.jsx X
src > ProfileCard.jsx > ...
1  function ProfileCard({ name, role }) {
2    return (
3      <div className="card">
4        <h3>{name}</h3>
5        <p>{role}</p>
6      </div>
7    );
8  }
9
10 export default ProfileCard;
11
```

App.jsx:

```
App.jsx X ProfileCard.jsx X
src > App.jsx > ...
1  import ProfileCard from "../ProfileCard";
2
3  function App() {
4    return (
5      <div>
6        <ProfileCard name="Ayesha Khan" role="Frontend Developer" />
7        <ProfileCard name="Bilal Ahmed" role="Backend Developer" />
8        <ProfileCard name="Sara Malik" role="UI/UX Designer" />
9      </div>
10   );
11 }
12
13 export default App;
14 |
```

Figure 163: Task 2: Passing Data with Props

Output: three cards are displayed on the page, each showing a different name and role — the same ProfileCard component is reused, with different data passed in each time.

Ayesha Khan

Frontend Developer

Bilal Ahmed

Backend Developer

Sara Malik

UI/UX Designer

Figure 164: Task 2: Passing Data with Props

Task 3: Managing State with useState (Interactive Counter)

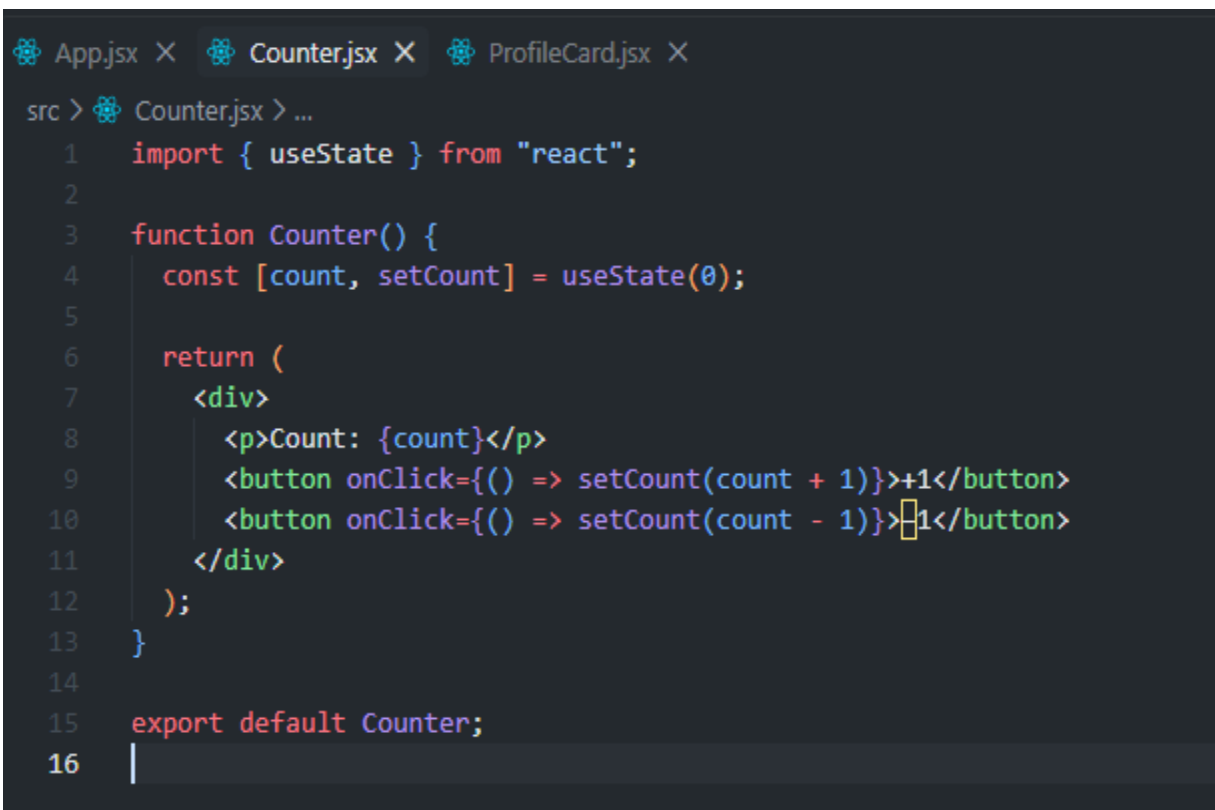
Objective: To manage local component state using useState and update the UI in response to user interaction.

Description: Build a Counter component with "+1" and "-1" buttons that increase or decrease a number displayed on screen.

Steps

- Step 1: Create a new file Counter.jsx and import useState from "react".
- Step 2: Inside the component, declare state with `const [count, setCount] = useState(0);`.
- Step 3: Display the current count inside a paragraph using `{count}`.
- Step 4: Add an increment button with an `onClick` handler that calls `setCount(count + 1)`.
- Step 5: Add a decrement button with an `onClick` handler that calls `setCount(count - 1)`, then import and render Counter inside App.

Counter.jsx:



```
App.jsx X Counter.jsx X ProfileCard.jsx X
src > Counter.jsx > ...
1  import { useState } from "react";
2
3  function Counter() {
4    const [count, setCount] = useState(0);
5
6    return (
7      <div>
8        <p>Count: {count}</p>
9        <button onClick={() => setCount(count + 1)}>+1</button>
10       <button onClick={() => setCount(count - 1)}>-1</button>
11      </div>
12    );
13  }
14
15  export default Counter;
16  |
```

Figure 165: Task 3: Managing State with useState (Interactive Counter)

Output: the page shows "Count: 0" with two buttons; clicking "+1" increases the number and clicking "-1" decreases it, with the display updating instantly each time.

Count: 7



Figure 166: Task 3: Managing State with useState (Interactive Counter)

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in a folder specified by your lab instructor.

Practice Task 1: Greeting Component

Build a Greeting component that accepts a name prop and displays "Hello, {name}!". Render it three times in App with three different names.

Use:

- A functional component that destructures name from its props parameter.
- Three separate `<Greeting name="..." />` calls with different values.

Practice Task 2: Like Button

Build a LikeButton component using useState to track whether a post has been liked. Clicking the button should toggle its text between "Like" and "Liked", and toggle a CSS class that changes its colour.

Use:

- `useState(false)` to track a boolean liked value.
- An `onClick` handler that calls `setLiked(!liked)`.
- A conditional expression (e.g. `liked ? "Liked" : "Like"`) inside the JSX to switch the button text.

Practice Task 3: Simple To-Do Input

Build a TodoInput component with a text field, an "Add" button, and a list below it. Typing a task and clicking "Add" should add it to an array stored in state and display it in the list.

Use:

- `useState("")` for the current input text, and `useState([])` for the array of todos.
 - An `onChange` handler on the input that updates the text state on every keystroke.
 - An `onClick` handler on "Add" that appends the current text to the todos array (e.g. `setTodos([...todos, text])`) and clears the input.
 - `.map()` to render each todo in the array as a `<li>`.
-

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The React application must run without errors and produce the required output and interactions.	35%	The app will be run and the output checked against the task requirements.
Correct use of JSX & components	Components must be structured correctly, return valid JSX, and be composed/nested appropriately.	20%	Code review of component structure and JSX syntax.
Correct use of props & state	Data must be passed correctly via props, and useState must be used correctly to manage and update state.	25%	Testing of prop passing and state updates in the browser.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	10%	Review of the submitted output and project folder.

Lab 09: React Fundamentals — useEffect, Conditional Rendering, and React Router

Background

Lab 08 introduced components, JSX, props, and state with `useState`. Those tools let a component hold and display data, but they don't help with two things every real app needs:

running code in response to a component appearing or its data changing (like starting a timer or listening for an event), and showing different UI depending on the current state or the current URL.

This lab covers three additions that fill those gaps. The `useEffect` hook lets a component run "side effects" — code that reaches outside of rendering, such as setting a timer, listening for a browser event, or updating the document title — at the right point in a component's lifecycle. Conditional rendering is the general pattern of using plain JavaScript logic inside JSX to decide what gets displayed.

React Router extends these ideas to the level of the whole app, letting a single-page React app show different components for different URLs without reloading the page.

By the end of this lab, students will be able to run and clean up side effects with `useEffect`, show different UI conditionally based on state, and navigate between multiple "pages" in a single-page app using React Router.

(Fetching data from an API and combining it with these hooks is covered in the next lab.)

Setting Up

This lab continues from a Vite-based React project, as set up in Lab 08. Open Visual Studio Code and go to Terminal and type:

1

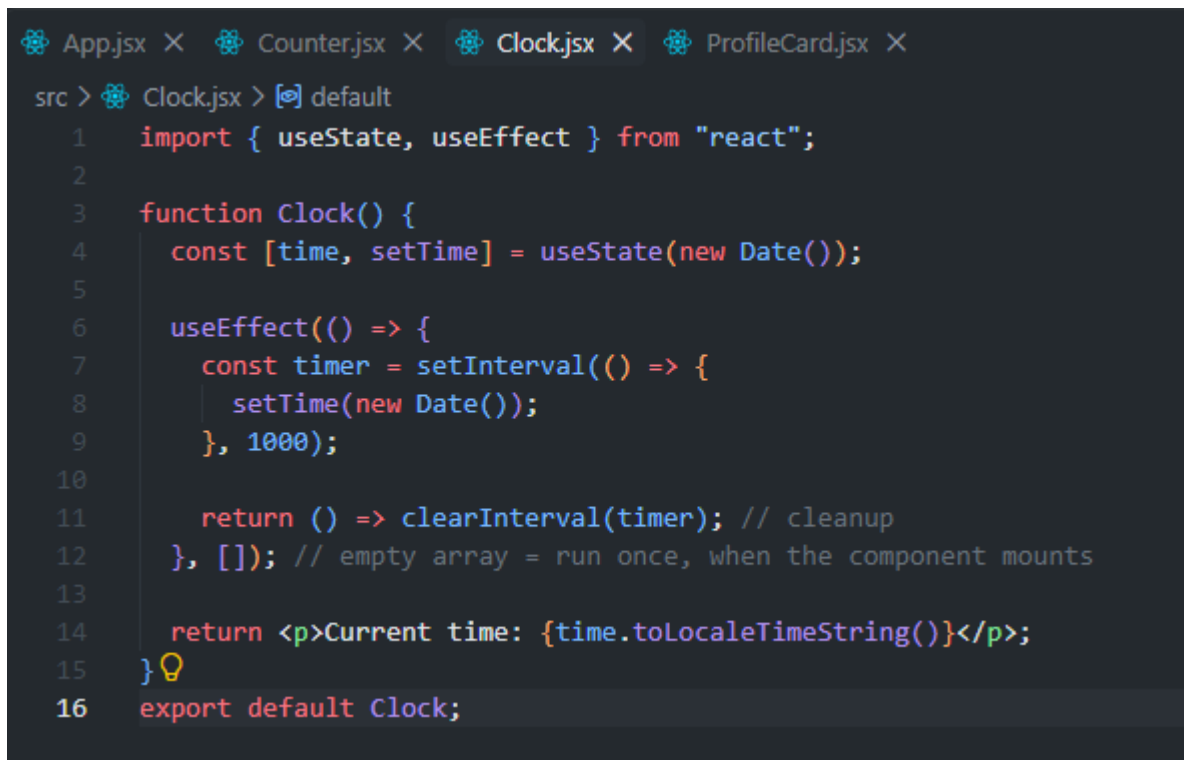
React Router is a separate package and needs to be installed on top of the base project:

➤ **npm install react-router-dom**

The useEffect Hook

useEffect lets a component run code after it renders — and optionally again whenever certain values change. This is where side effects belong: things like setting a timer, reading/writing localStorage, subscribing to a browser event, or updating document.title. These are called "side effects" because they reach outside of what the component returns as JSX.

Clock.jsx:



```
src > Clock.jsx > default
1  import { useState, useEffect } from "react";
2
3  function Clock() {
4    const [time, setTime] = useState(new Date());
5
6    useEffect(() => {
7      const timer = setInterval(() => {
8        setTime(new Date());
9      }, 1000);
10
11     return () => clearInterval(timer); // cleanup
12   }, []); // empty array = run once, when the component mounts
13
14   return <p>Current time: {time.toLocaleTimeString()}</p>;
15 }
16 export default Clock;
```

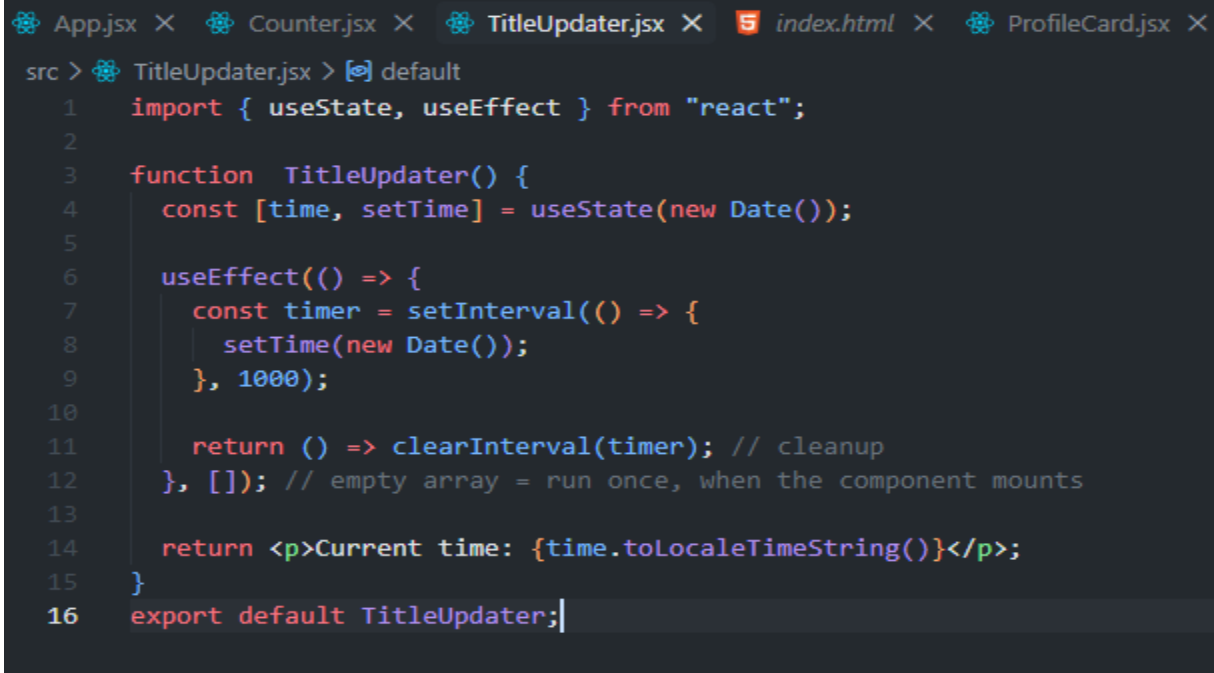
Figure 168: Lab 09: React Fundamentals — useEffect, Conditional Rendering, and React Router

useEffect takes two arguments: a function containing the side effect, and a dependency array that controls when it re-runs.

- No dependency array — the effect runs after every render.
- Empty array [] — the effect runs once, when the component first mounts.
- Array with values [count] — the effect runs on mount, and again any time count changes.
- Returning a function from inside the effect — this is the cleanup function; React calls it before the effect runs again, and when the component unmounts. It's used to cancel timers, remove event listeners, or close subscriptions so they don't pile up.

Another common use of useEffect is updating something outside of React, like the browser tab's title, whenever a piece of state changes:

TitleUpdater.jsx:

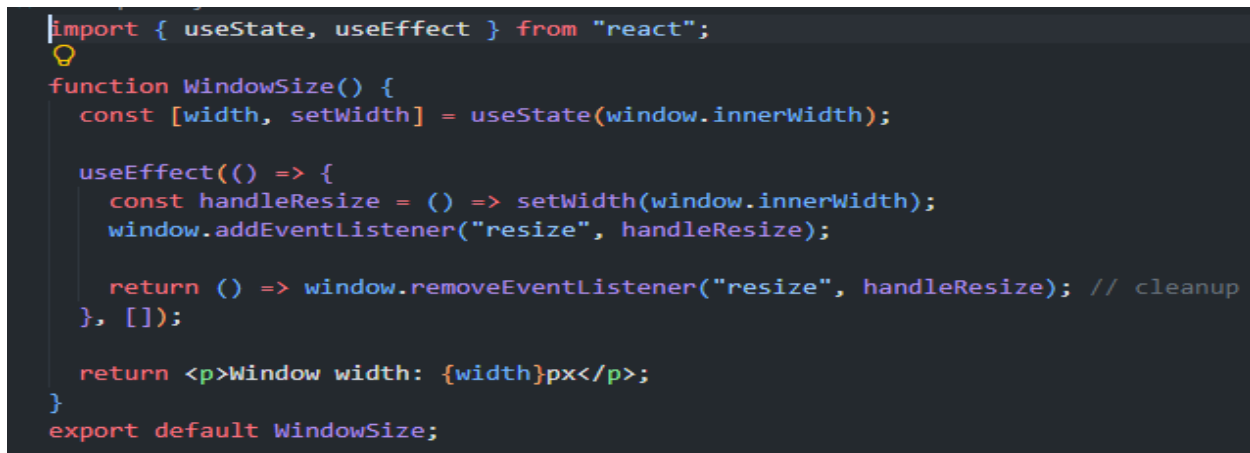


```
src > TitleUpdater.jsx > default
1  import { useState, useEffect } from "react";
2
3  function TitleUpdater() {
4    const [time, setTime] = useState(new Date());
5
6    useEffect(() => {
7      const timer = setInterval(() => {
8        setTime(new Date());
9      }, 1000);
10
11     return () => clearInterval(timer); // cleanup
12   }, []); // empty array = run once, when the component mounts
13
14   return <p>Current time: {time.toLocaleTimeString()}</p>;
15 }
16 export default TitleUpdater;
```

Figure 169: TitleUpdater.jsx:

useEffect is also used to listen for browser events that exist outside any single element, such as window resizing — and to clean the listener up afterward:

WindowSize.jsx:



```
import { useState, useEffect } from "react";
function WindowSize() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);

    return () => window.removeEventListener("resize", handleResize); // cleanup
  }, []);

  return <p>Window width: {width}px</p>;
}
export default WindowSize;
```

Figure 170: TitleUpdater.jsx:

Tip: the dependency array is the single most common source of bugs with **useEffect**. Leaving it off when it should be `[]` causes an effect to re-run on every render — for a timer or a subscription, that means creating a new one each time without ever cleaning up the last.

Conditional

Rendering

Conditional rendering means choosing what JSX to return based on a condition, using ordinary JavaScript rather than any special React syntax. There are a few common patterns:

if / else before the return statement:

Greeting.jsx:

```
> Greeting.jsx > [default]
1  function Greeting({ isLoggedIn }) {
2    if (isLoggedIn) {
3      return <h2>Welcome back!</h2>;
4    }
5    return <h2>Please log in.</h2>;
6  }
7  export default Greeting;
```

The ternary operator, useful inline within JSX:

Greeting.jsx:

```
function Greeting({ isLoggedIn }) {
  return (
    <h2>{isLoggedIn ? "Welcome back!" : "Please log in."}</h2>
  );
}

export default Greeting;
```

Figure 171: Greeting.jsx:

The && operator, useful for rendering something only when a condition is true (and nothing otherwise):

Notifications.jsx:

```
src > Greeting.jsx > default
1  function Notifications({ count }) {
2    return (
3      <div>
4        <h3>Inbox</h3>
5        {count > 0 && <p>You have {count} new messages.</p>}
6      </div>
7    );
8  }
9  export default Notifications;
```

- Conditional rendering combines naturally with `useEffect` whenever a component needs to change what it shows after some time passes or some event fires — for example, showing a "Welcome" message for a few seconds and then switching to the main content:

WelcomeBanner.jsx:

```
App.jsx × Counter.jsx × Greeting.jsx × index.html × ProfileCard.jsx ×
src > Greeting.jsx > default
1  import { useState, useEffect } from "react";
2
3  function WelcomeBanner() {
4    const [showWelcome, setShowWelcome] = useState(true);
5
6    useEffect(() => {
7      const timer = setTimeout(() => {
8        setShowWelcome(false);
9      }, 3000);
10
11     return () => clearTimeout(timer); // cleanup
12   }, []);
13
14   if (showWelcome) return <h2>Welcome! Getting things ready...</h2>;
15
16   return <h2>Here's your dashboard.</h2>;
17 }
18
19 export default WelcomeBanner;
```

Figure 172: Greeting.jsx:

Tip: this pattern — a boolean flag flipped inside `useEffect`, then checked with an early return — is the same shape you'll later use for loading states, just driven by a timer here instead of a network request.

React

A single-page application (SPA) loads one HTML page and then uses JavaScript to change what's shown as the user navigates, instead of requesting a new page from the server each time. React Router is the standard library for adding this kind of navigation to a React app: it maps URL paths to components.

Router

The three core pieces are `BrowserRouter`, `Routes`, and `Route`:

main.jsx:

```
> main.jsx
1  import { BrowserRouter } from "react-router-dom";
2  import { createRoot } from "react-dom/client";
3  import App from "./App";
4
5  createRoot(document.getElementById("root")).render(
6    <BrowserRouter>
7      <App />
8    </BrowserRouter>
9  );
```

App.jsx:

```
rc > App.jsx > ...
1  import { Routes, Route } from "react-router-dom";
2  import Home from "./Home";
3  import About from "./About";
4
5  function App() {
6    return (
7      <Routes>
8        <Route path="/" element={<Home />} />
9        <Route path="/about" element={<About />} />
10      </Routes>
11    );
12  }
13
14  export default App;
15
```

- BrowserRouter wraps the whole app once, usually in main.jsx, and enables routing.
- Routes looks at the current URL and renders the first Route whose path matches.
- Route pairs a path with the element (component) to show for that path.

Link is used instead of a plain `<a>` tag for navigation, because a normal link reloads the whole page — Link swaps components without a reload:

NavBar.jsx:

```
1 import { Link } from "react-router-dom";
2
3 function NavBar() {
4   return (
5     <nav>
6       <Link to="/">Home</Link>
7       <Link to="/about">About</Link>
8     </nav>
9   );
10 }
11 export default NavBar;
```

Dynamic segments in a path (for example, a different page per product ID) are written with a colon, and read inside the component with the `useParams` hook. Here the data comes from a small hardcoded list defined in the component itself, not from an API:

App.jsx:

```
<Route path="/products/:id" element={<ProductDetail />} />
```

ProductDetail.jsx:

```

> Greeting.jsx > [🔍] default
import { useParams } from "react-router-dom";

const products = [
  { id: "1", name: "Notebook" },
  { id: "2", name: "Desk Lamp" },
  { id: "3", name: "Water Bottle" },
];

function ProductDetail() {
  const { id } = useParams();
  const product = products.find((p) => p.id === id);

  if (!product) return <p>Product not found.</p>;

  return <h2>{product.name}</h2>;
}

export default ProductDetail;

```

Tip: useParams reads whatever value was in the URL where the route defined a :name segment — the names must match exactly between the Route path and the useParams destructuring. Notice the if (!product) check above: that's conditional rendering guarding against an id that doesn't match anything.

Home Tasks (Before Lab)

Before attending the lab session, students should:

Home Task 01

Install react-router-dom in a React project (from Lab 08 or a fresh Vite project) and confirm the app still runs with npm run dev.

Home Task 02

Read the React documentation on useEffect and write one sentence explaining what the dependency array controls.

Home Task 03

In your own words, write one sentence explaining the difference between a ternary (?:) and the && operator when used for conditional rendering.

Lab

The students will be able to:

Objectives

- Use the useEffect hook to run code on mount and in response to state changes.
- Correctly use a dependency array and a cleanup function with useEffect.

- Apply conditional rendering to show different UI based on state.
- Set up React Router with BrowserRouter, Routes, and Route.
- Navigate between components using Link without a full page reload.
- Read dynamic URL segments using useParams.

Walkthrough Task

Task 1: A Live Clock with Conditional Greeting

Objective: To use `useEffect` to run a timer, and conditional rendering to change the UI based on the current time.

Description: Build a Clock component that displays the current time, updating every second, and shows a different greeting depending on the hour.

Steps

Step 1: Create a new file `Clock.jsx` and import `useState` and `useEffect` from "react".

Step 2: Declare state for the current time with `useState(new Date())`.

Step 3: Inside a `useEffect` with an empty dependency array, use `setInterval` to update the time state every second, and return a cleanup function that calls `clearInterval`.

Step 4: Above the time display, use a ternary or if/else to show "Good morning", "Good afternoon", or "Good evening" based on `time.getHours()`.

Step 5: Import and render Clock inside `App.jsx`.

Clock.jsx:

```
import { useState, useEffect } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  const hour = time.getHours();
  let greeting = "Good evening";
  if (hour < 12) greeting = "Good morning";
  else if (hour < 18) greeting = "Good afternoon";

  return (
    <div>
      <h2>{greeting}</h2>
      <p>Current time: {time.toLocaleTimeString()}</p>
    </div>
  );
}

export default Clock;
```

Figure 173: Task 1: A Live Clock with Conditional Greeting

Output: the page shows a greeting that matches the time of day, and a clock that updates every second without any manual refresh.

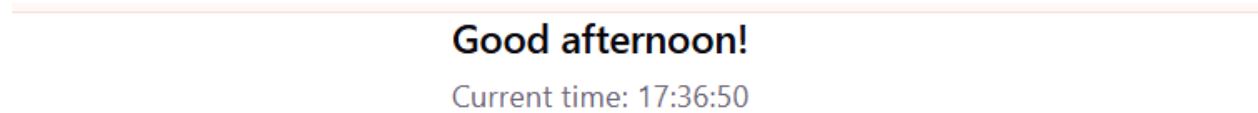


Figure 174: Task 1: A Live Clock with Conditional Greeting

Task 2: Start/Stop Control for the Timer

Objective: To control when a `useEffect` runs using a dependency that comes from state, and to see cleanup run mid-lifecycle rather than only on unmount.

Description: Extend Clock with a button that starts and stops the ticking, using a boolean state value in the effect's dependency array.

Steps

Step 1: Add a new state value `isRunning` with `useState(true)`.

Step 2: Add `isRunning` to the effect's dependency array, so the effect re-runs whenever it changes.

Step 3: Inside the effect, only start the interval if `isRunning` is true; still return the cleanup function unconditionally.

Step 4: Add a button that toggles `isRunning` with `setIsRunning(!isRunning)`, labeled "Stop" or "Start" depending on the current value.

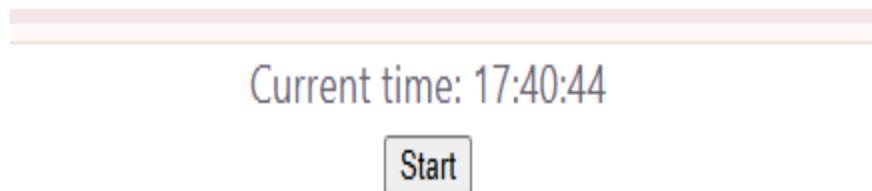
Step 5: Confirm the clock freezes when stopped and resumes from the current time when started again.

Clock.jsx:

```
c > Clock.jsx > ...
1  import { useState, useEffect } from "react";
2
3  function Clock() {
4    const [time, setTime] = useState(new Date());
5    const [isRunning, setIsRunning] = useState(true);
6
7    useEffect(() => {
8      if (!isRunning) return;
9
10     const timer = setInterval(() => {
11       setTime(new Date());
12     }, 1000);
13
14     return () => clearInterval(timer);
15   }, [isRunning]);
16
17   return (
18     <div>
19       <p>Current time: {time.toLocaleTimeString()}</p>
20       <button onClick={() => setIsRunning(!isRunning)}>
21         {isRunning ? "Stop" : "Start"}
22       </button>
23     </div>
24   );
25 }
26
27 export default Clock;
28
```

Figure 175: Task 1: A Live Clock with Conditional Greeting

Output: clicking "Stop" freezes the displayed time and changes the button to "Start"; clicking "Start" resumes ticking from the current time.



Task 3: Multi-Page Navigation with React Router

Objective: To set up client-side routing so different components render for different URLs.
Description: Build a small app with Home, About, and Contact pages, and a navigation bar built with Link.

Steps

- **Step 1:** Install react-router-dom if it isn't already installed.
- **Step 2:** In main.jsx, wrap the App component in BrowserRouter.
- **Step 3:** Create three components — Home.jsx, About.jsx, and Contact.jsx — each returning a simple heading.
- **Step 4:** In App.jsx, use Routes and Route to map "/", "/about", and "/contact" to the three components.
- **Step 5:** Add a NavBar component above the Routes, using Link for each page, and render it inside App.

main.jsx:

```
src > main.jsx
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import { BrowserRouter } from "react-router-dom";
4  import App from "./App";
5
6  createRoot(document.getElementById("root")).render(
7    <StrictMode>
8      <BrowserRouter>
9        <App />
10     </BrowserRouter>
11   </StrictMode>
12 );
```

App.jsx:

```
App.jsx > ...
import { Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}

export default App;
```

Figure 176: Task 1: A Live Clock with Conditional Greeting

Output: clicking each link in the navigation bar swaps the page content between Home, About, and Contact without a full page reload, and the URL in the address bar updates accordingly.

Practice Tasks

This section provides information about the practice tasks that must be performed during the lab session. Design solutions for the problems discussed in each task and place the solution code in a folder specified by your lab instructor.

Practice Task 1: Live Search Filter (Static List)

Build a `SearchableList` component with a hardcoded array of at least six item names defined inside the component. Add a text input; as the user types, filter the displayed list to only items whose name includes the typed text.

Use:

- `useState("")` for the search text, updated on every keystroke with an `onChange` handler.
- `.filter()` on the hardcoded array before `.map()`, to show only matching items.
- Conditional rendering to show a "No matches found" message when the filtered list is empty.

Practice Task 2: Toggleable Details Panel

Build a `ProductCard` component that shows a product name and a "Show details" / "Hide details" button. Clicking the button toggles whether a details paragraph is shown, using a boolean state value and conditional rendering with `&&`.

Use:

- `useState(false)` to track whether details are visible.
- An `onClick` handler that calls `setShowDetails(!showDetails)`.
- The `&&` operator to conditionally render the details paragraph.

Practice Task 3: Simple Routed Profile Pages (Static Data)

Build a small app with a list of at least four users, hardcoded as an array in a shared file, shown on the home page. Clicking a user's name (a `Link`) should navigate to `/users/:id` and show that user's details using `useParams`.

Use:

- A Route defined as `path="/users/:id"` pointing to a `UserProfile` component.
 - `Link` components on the home page, with `to={`/users/${user.id}`}` for each user.
 - `useParams` inside `UserProfile` to read the `id` from the URL, find the matching user in the hardcoded array, and display their data (with a conditional fallback if no match is found).
-

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The React application must run without errors and produce the required output and interactions.	35%	The app will be run and the output checked against the task requirements.
Correct use of useEffect	Effects must run at the correct time, use an appropriate dependency array, and clean up where needed.	20%	Code review of effect logic and dependency arrays.
Correct use of conditional rendering & routing	UI states must be handled correctly, and routes/links must navigate to the correct components.	25%	Testing of conditional UI and route navigation in the browser.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	10%	Review of the submitted output and project folder.

Lab 10: Async JS & Fetch API — Promises, async/await, and the Fetch API

Background

Every app you've built so far has worked with data that already exists in the browser — hardcoded arrays and local state. Real applications need to talk to servers: loading a list of products, submitting a form to a backend, or checking if a username is available. These operations take time and cannot block the rest of the page while they happen, so JavaScript handles them asynchronously.

This lab covers three related ideas: Promises, the object JavaScript uses to represent a value that isn't ready yet; `async/await`, the modern syntax for writing asynchronous code that reads like ordinary synchronous code; and the Fetch API, the browser's built-in tool for making HTTP requests. The lab then shows how to combine all three with the `useEffect` hook from Lab 09 to load data into a React component when it first appears on screen.

Promises

A Promise represents a value that will be available later — either successfully (resolved) or with an error (rejected). Many browser APIs, including `fetch`, return a Promise instead of the final value directly.

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Done!");
  }, 1000);
});
promise.then(result => {
  console.log(result); // "Done!" – logged after 1 second
});
```

A Promise is always in one of three states: **pending** (still working), **fulfilled** (succeeded, `.then()` runs), or **rejected** (failed, `.catch()` runs).

```
promise.then(result => console.log("Success:", result))
  .catch(error => console.log("Error:", error));
```

async/await

async/await is syntax built on top of Promises that lets asynchronous code be written without chaining `.then()` calls. Any function marked `async` automatically returns a Promise, and `await` pauses execution inside that function until a Promise resolves.

```
// Using .then()
function loadData() {
  fetch("/api/data")
    .then((response) => response.json())
    .then((data) => console.log(data));
}

// The same logic with async/await
async function loaddata() {
  const response = await fetch("/api/data");
  const data = await response.json();
  console.log(data);
}
```

Errors are handled with a regular `try/catch` block, instead of `.catch()`:

```
async function loadData() {
  try {
    const response = await fetch("/api/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Something went wrong:", error);
  }
}
```

Tip: `await` can only be used inside a function marked `async`. Using it at the top level of a regular function (without `async`) is a syntax error.

The Fetch API

`fetch()` is the browser's built-in function for making HTTP requests. It takes a URL and returns a Promise that resolves to a Response object.

```
async function getUsers() {  
  const response = await fetch("https://jsonplaceholder.typicode.com/users");  
  const users = await response.json(); // parses the JSON body – also returns a Promise  
  console.log(users);  
}
```

Figure 177: The Fetch API

Note that `response.json()` itself returns a Promise, so it needs its own `await`. `fetch()` only rejects on a network failure — a 404 or 500 response is still a "successful" fetch, so the status should be checked manually:

```
async function getUser(id) {  
  const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);  
  if (!response.ok) {  
    throw new Error("User not found");  
  }  
  return response.json();  
}
```

Figure 178: The Fetch API

Sending data with fetch (POST request):

```
async function createPost(title) {  
  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ title, body: "Some content", userId: 1 }),  
  });  
  return response.json();  
}
```

Figure 179: The Fetch API

Fetching Data in React (useEffect + async/await)

Since a component's render function cannot be async directly, the async function is defined separately and called from inside `useEffect`:

UserList.jsx

```
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadUsers() {
      try {
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!response.ok) throw new Error("Failed to load users");
        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadUsers();
  }, []); // empty array – fetch once, when the component mounts

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

export default UserList;
```

Figure 180: Fetching Data in React (useEffect + async/await)

This loading / error / data pattern — three pieces of state driving three different pieces of conditional rendering — is the standard shape for any component that loads data from a server.

Home Tasks (Before Lab)

Home Task 01

Write a short async function that awaits `fetch("https://jsonplaceholder.typicode.com/todos/1")`, logs the parsed JSON to the console, and wraps the whole thing in a try/catch.

Home Task 02

In your own words, write one sentence explaining the difference between `.then()` chaining and `async/await` for the same operation.

Home Task 03

Review the three states of a Promise (pending, fulfilled, rejected) and write one sentence describing what triggers each one.

Lab Objectives

The students will be able to:

- Explain what a Promise is and its three possible states.
- Write asynchronous code using `async/await` and handle errors with try/catch.
- Use the Fetch API to make GET and POST requests.
- Check a fetch response's status before using its data.
- Combine fetch with `useEffect` to load data into a React component on mount.
- Manage loading and error states while data is being fetched.

Walkthrough Task

Task 1: Fetching and Displaying a List

Objective: To fetch data from a public API using `async/await` and display it in a component.

Description: Build a `PostList` component that fetches a list of posts from a public API and displays each post's title.

Steps

- Step 1: Create a new file `PostList.jsx` and import `useState` and `useEffect` from "react".
- Step 2: Declare state for posts (starting as an empty array).
- Step 3: Inside `useEffect` with an empty dependency array, define and call an async function that fetches from `https://jsonplaceholder.typicode.com/posts`.
- Step 4: Use `await response.json()` to parse the response, then limit it to the first 5 posts with `.slice(0, 5)` before calling `setPosts`.

- Step 5: Render the posts with `.map()`, using each post's id as the key.

PostList.jsx:

```
import { useState, useEffect } from "react";

function PostList() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    async function loadPosts() {
      const response = await fetch("https://dummyjson.com/posts?limit=5");
      const data = await response.json();
      setPosts(data.posts);
    }
    loadPosts();
  }, []);

  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}

export default PostList;
```

Figure 181: Task 1: Fetching and Displaying a List

Output: Fetching `https://dummyjson.com/posts?limit=5` and rendering each post's title inside a `<li>` produces a bulleted list of the first 5 post titles

- His mother had always taught him
- He was an expert but not in a discipline
- Dave watched as the forest burned up on the hill.
- All he wanted was a candy bar.
- Hopes and dreams were dashed that day.

Task 2: Loading and Error States

Objective: To manage the full lifecycle of a fetch request: loading, success, and failure.

Description: Extend PostList to show a loading message while the request is in progress, and an error message if it fails.

Steps

- Step 1: Add two more state values: loading (starting true) and error (starting null).
- Step 2: Wrap the fetch logic in a try/catch/finally block.
- Step 3: In the catch block, store the error message with setError.
- Step 4: In the finally block, set loading to false — this runs whether the request succeeded or failed.
- Step 5: Before the main return, add early returns for the loading and error states.

PostList.jsx:

```
export default PostList;

import { useState, useEffect } from "react";

function PostList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

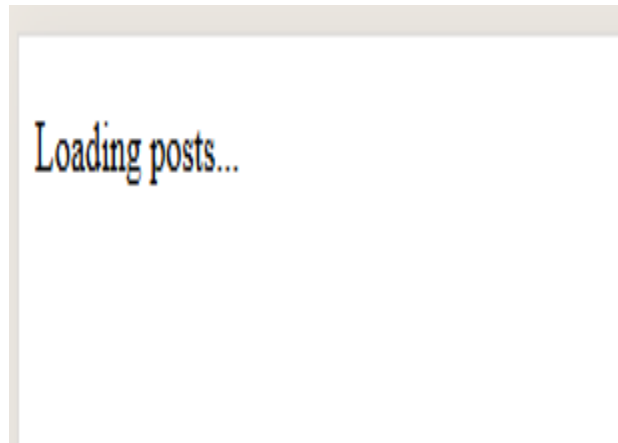
  useEffect(() => {
    async function loadPosts() {
      try {
        const response = await fetch("https://dummyjson.com/posts?limit=5");
        if (!response.ok) throw new Error("Failed to load posts");
        const data = await response.json();
        setPosts(data.posts);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadPosts();
  }, []);

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Figure 182: Task 2: Loading and Error States

Output: "Loading posts..." briefly appears first, then is replaced by the list of titles; if the network request is blocked (e.g. offline), an error message is shown instead.



Task 3: Fetching Details on Click (Dynamic Fetch)

Objective: To trigger a new fetch request in response to a user action, rather than only on mount.

Description: Extend the app so clicking a post in the list fetches and displays that single post's full body text below the list.

Steps

- Step 1: Add state for `selectedPost`, starting as `null`.
- Step 2: Write a separate async function `loadPostDetail(id)` that fetches `https://jsonplaceholder.typicode.com/posts/${id}` and calls `setSelectedPost` with the result.
- Step 3: Add an `onClick` handler on each list item that calls `loadPostDetail(post.id)`.
- Step 4: Below the list, conditionally render the selected post's body using `selectedPost` &&.
- Step 5: Confirm that clicking a different post replaces the previously shown detail.

PostList.jsx:

```
import { useState, useEffect } from "react";

function PostList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [selectedPost, setSelectedPost] = useState(null);

  useEffect(() => {
    async function loadPosts() {
      try {
        const response = await fetch("https://dummyjson.com/posts?limit=5");
        if (!response.ok) throw new Error("Failed to load posts");
        const data = await response.json();
        setPosts(data.posts);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadPosts();
  }, []);

  async function loadPostDetail(id) {
    const response = await fetch(`https://dummyjson.com/posts/${id}`);
    const data = await response.json();
    setSelectedPost(data);
  }

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div>
      <ul>
        {posts.map((post) => (
          <li key={post.id} onClick={() => loadPostDetail(post.id)}>
            {post.title}
          </li>
        ))}
      </ul>

      {selectedPost && (
        <div>
          <h3>{selectedPost.title}</h3>
          <p>{selectedPost.body}</p>
        </div>
      )}
    </div>
  );
}

export default PostList;
```

Figure 183: Task 3: Fetching Details on Click (Dynamic Fetch)

Output: clicking any post title in the list loads and displays that post's full text underneath the list, without reloading the whole component.

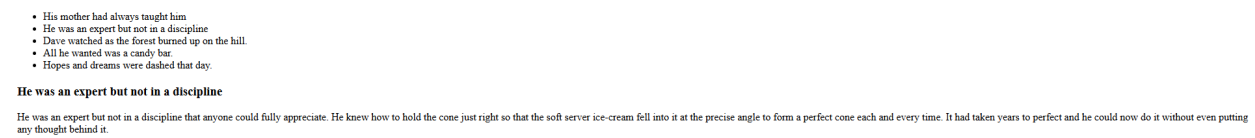


Figure 184: Task 3: Fetching Details on Click (Dynamic Fetch)

Practice Tasks

Practice Task 1: User Directory

Build a `UserDirectory` component that fetches `https://jsonplaceholder.typicode.com/users` and displays each user's name and email in a list, with loading and error states handled.

Use:

- `useState` for users, loading, and error.
- `useEffect` with an empty dependency array to fetch once on mount.
- Conditional rendering for the loading and error states before the main list.

Practice Task 2: Search-as-you-fetch

Build a `ProductSearch` component with a text input. On every keystroke, fetch `https://jsonplaceholder.typicode.com/photos?title=${searchText}` and display the matching titles (limit the results shown to the first 10).

Use:

- `useState` for the search text and the results array.
- An async function triggered from the input's `onChange` handler (not from `useEffect`, since it should re-run on every keystroke, not just on mount).
- `.slice(0, 10)` on the results before rendering.

Practice Task 3: Submitting a Form with POST

Build a `NewPostForm` component with a title input and a "Submit" button. On submit, send a POST request to `https://jsonplaceholder.typicode.com/posts` with the title, and display a success message with the returned post's id once the request completes.

Use:

- `event.preventDefault()` inside the submit handler (from Lab 07).
- An async function that sends the POST request with method: "POST", a JSON Content-Type header, and a `JSON.stringify()`'d body.
- A submitted boolean state to conditionally show the success message.

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The application must run without errors and correctly fetch, display, or send the required data.	35%	The app will be run and the output checked against the task requirements.
Correct use of async/await & Promises	Asynchronous code must be written correctly, with errors handled using try/catch.	25%	Code review of async function structure and error handling.
Correct use of Fetch + useEffect	Fetch requests must be triggered at the correct point (mount vs. user action) and combined correctly with React state.	20%	Testing of data loading, loading states, and error states in the browser.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	10%	Review of the submitted output and project folder.

Lab 11: Node.js Basics — Node Runtime, npm/package.json, and a Simple HTTP Server

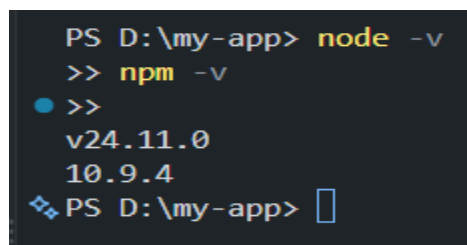
Background

Every lab so far has run entirely inside the browser. Node.js is a JavaScript runtime that runs outside the browser — on a server, or on your own machine — which is what lets JavaScript be used to build the backend half of a web application: the part that stores data, handles requests, and talks to a database.

This lab introduces the Node.js runtime, the npm package manager and the package.json file that describes a Node project, and Node's built-in http module for building a basic web server. This is the foundation the next three labs (Express, Sequelize, and full-stack integration) build directly on top of.

Setting Up

Download and install Node.js (LTS version) from nodejs.org. Verify the installation from a terminal:



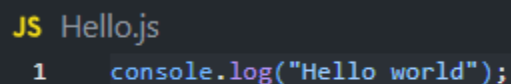
```
PS D:\my-app> node -v
>> npm -v
v24.11.0
10.9.4
PS D:\my-app>
```

Both commands should print a version number. node is the JavaScript runtime itself; npm (Node Package Manager) is installed alongside it and is used to install and manage third-party packages.

Running JavaScript with Node

Any .js file can be executed directly with Node, outside of a browser:

Hello.js:



```
JS Hello.js
1 console.log("Hello world");
```

Node also gives access to things a browser doesn't, such as the file system and the current process's arguments — but it does not have document or window, since there is no page to render.

npm and package.json

package.json is a file that describes a Node project: its name, its dependencies, and scripts that can be run with npm run. It's created with:

npm init -y

This produces a file similar to:

Package.json:

```
{
  "name": "my-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "eslint": "^10.10.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.6",
    "globals": "^17.12.0",
    "vite": "^8.3.0"
  }
}
```

Figure 185: Package.json:

Installing packages: `npm install <package-name>` downloads a package into a `node_modules` folder and adds it to dependencies in `package.json`:

```
PS D:\my-app> npm install express
```

Custom scripts: any entry under "scripts" can be run with `npm run <name>` (the special names `start` and `test` can be run without `run`):

Package.json:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "preview": "vite preview"  
},
```

Such as :

`npm run dev`

Tip: `node_modules` should never be committed to version control — it's regenerated on any machine by running `npm install`, which reads `package.json` and downloads everything listed there.

Modules: require and import

Node code is split across files using modules. Node supports two module systems:

CommonJS (the older, still very common style — uses `require`):

Math.js:

```
// math.js  
function add(a, b) {  
  return a + b;  
}  
module.exports = { add };  
js  
// index.js  
const { add } = require("./math");  
console.log(add(2, 3)); // 5  
|
```

ES Modules (the modern style — uses import/export, same syntax as React):

Math.js

```
// math.js
export function add(a, b) {
  return a + b;
}
```

Index.js

```
src > ⚙ Clock.jsx
1 // index.js
2 import { add } from "./math.js";
3 console.log(add(2, 3));
4 |
```

To use ES Modules, add "type": "module" to package.json. This lab uses CommonJS (require) since it's still the default and most examples online use it, but both are valid.

Building a Simple HTTP Server

Node's built-in http module can create a web server without installing anything:

Server.js:

```
// server.js
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from my Node server!");
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:\${PORT}`);
});
```

Visiting <http://localhost:3000> in a browser (or refreshing it) shows the response text. Every request — whatever the URL — currently gets the same response; the next section checks the URL to respond differently.

Responding differently based on the URL:

Server.js

```
import http from "http";

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Welcome to the home page");
  } else if (req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("This is the about page");
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page not found");
  }
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Figure 186: Building a Simple HTTP Server

Responding with JSON (the format almost every real API uses):

Server.js:

```
const server = http.createServer((req, res) => {
  if (req.url === "/api/users") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify([
      { id: 1, name: "Ali" },
      { id: 2, name: "Sara" }
    ]));
  }
});
```

Figure 187: Building a Simple HTTP Server

Tip: `res.writeHead()` must be called before `res.end()`, and the `Content-Type` header should match what's actually being sent — a browser or `fetch()` call uses that header to decide how to interpret the response.

Home Tasks (Before Lab)

Home Task 01

Install Node.js, confirm it with `node -v` and `npm -v`, then create and run a `hello.js` file that logs a message to the console.

Home Task 02

Run `npm init -y` in a new folder and open the generated `package.json`. Write one sentence explaining what the "scripts" section is for.

Home Task 03

In your own words, write one sentence explaining the difference between `require()` and `import` in Node.js.

Lab Objectives

The students will be able to:

- Install Node.js and run JavaScript files outside the browser.
- Create and understand a `package.json` file.
- Install and use third-party packages with `npm`.
- Split code across files using modules (`require/module.exports`).
- Build a basic HTTP server using Node's built-in `http` module.
- Respond to different URLs with different content, including JSON.

Walkthrough Task

Task 1: A Basic HTTP Server

Objective: To create and run a minimal HTTP server using Node's built-in `http` module.

Description: Build a server that responds with a plain text greeting on every request, and confirm it in the browser.

Steps

- Step 1: Create a new folder and run `npm init -y` inside it.
- Step 2: Open `package.json` and add `"type": "module"`, inside it, so Node treats `.js` files as ES Modules.
- Step 3: Create `server.js` and import the built-in `http` module using `import http from "http";`.
- Step 4: Call `http.createServer()` with a callback function that takes `req` and `res`.

- Step 5: Inside the callback, call `res.writeHead(200, ...)` and `res.end(...)` with a greeting message.
- Step 6: Call `server.listen(3000, ...)` and run the file with `node server.js`.

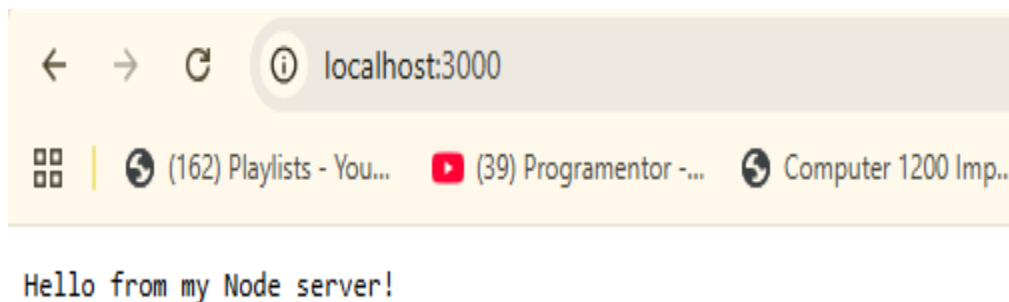
Server.js

```
// server.js
import http from "http";

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from my Node server!");
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Output: running `node server.js` prints "Server running at <http://localhost:3000>" in the terminal, and visiting that URL in a browser shows "Hello from my Node server!".



Task 2: Routing by URL

Objective: To respond differently depending on the requested URL.

Description: Extend the server to serve a home page, an about page, and a 404 message for anything else.

Steps

- Step 1: Inside the request callback, check `req.url` with an `if/else if/else` chain.
- Step 2: For `req.url === "/"`, respond with a home page message.
- Step 3: For `req.url === "/about"`, respond with an about page message.
- Step 4: For any other URL, respond with status code 404 and a "Page not found" message.
- Step 5: Restart the server and test all three cases by visiting `/`, `/about`, and `/xyz` in the browser.

Server.js

```
// server.js
import http from "http";

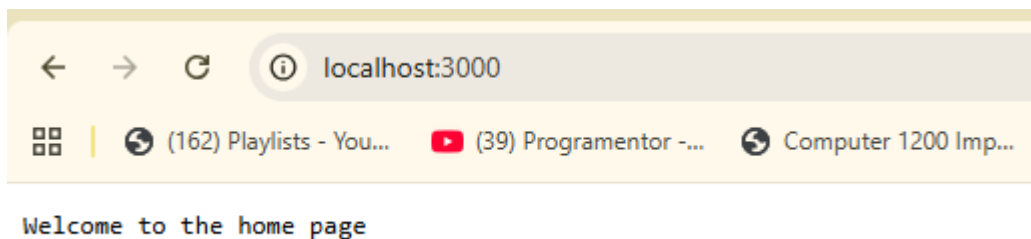
const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Welcome to the home page");
  } else if (req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("This is the about page");
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page not found");
  }
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

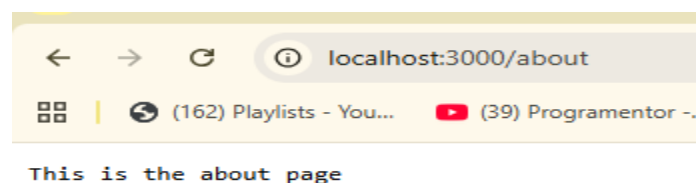
Figure 188: Task 2: Routing by URL

Output: / shows the home message, /about shows the about message, and any other path (e.g. /contact) shows "Page not found" with a 404 status, visible in the browser's Network tab.

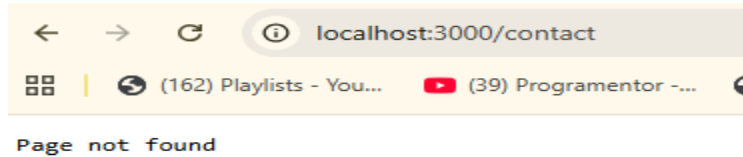
/:



/about:



/contact:



Task 3: Serving JSON Data

Objective: To respond with structured JSON data instead of plain text, matching the format a real API uses.

Description: Add a /api/users route that returns a hardcoded array of user objects as JSON.

Steps

- Step 1: Define an array of at least three user objects (each with id and name) above the server.
- Step 2: Add a new branch to the if/else chain for req.url === "/api/users".
- Step 3: Set the response header's Content-Type to "application/json".
- Step 4: Convert the array to a JSON string with JSON.stringify() and pass it to res.end().
- Step 5: Test by visiting /api/users in the browser — the raw JSON should be displayed.

Server.js

```
// server.js
import http from "http";

const users = [
  { id: 1, name: "Ali" },
  { id: 2, name: "Sara" },
  { id: 3, name: "Bilal" },
];

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Welcome to the home page");
  } else if (req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("This is the about page");
  } else if (req.url === "/api/users") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify(users));
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page not found");
  }
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Figure 189: Task 3: Serving JSON Data

Output: visiting `http://localhost:3000/api/users` displays the raw JSON array of users in the browser.

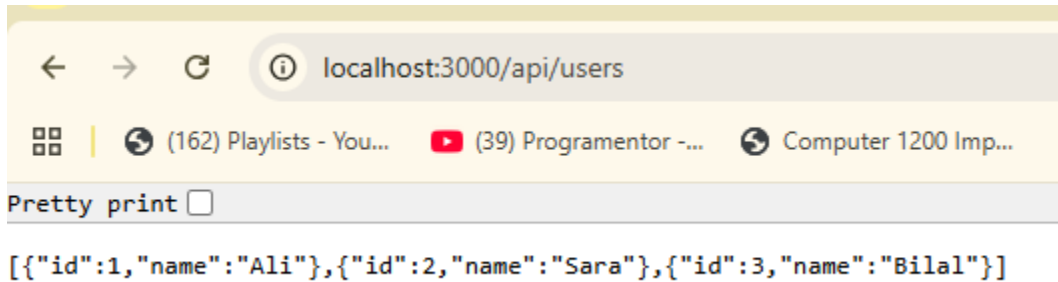


Figure 190: Task 3: Serving JSON Data

Practice Tasks

Practice Task 1: Multi-Route Server

Build a server with at least four routes: `/`, `/about`, `/contact`, and a catch-all 404. Each route should return a distinct plain-text message.

Use:

- `http.createServer()` with an if/else if/else chain based on `req.url`.
- The correct status code (200 for known routes, 404 for anything else).

Practice Task 2: JSON Products API

Build a server with a hardcoded array of at least five product objects (`id`, `name`, `price`). Add a `/api/products` route that returns the whole array as JSON.

Use:

- `JSON.stringify()` to convert the array before sending it.
- The `"application/json"` Content-Type header.

Practice Task 3: Single Product by ID

Extend the products server so a URL like `/api/products/2` returns just that one product as JSON, or a 404 JSON error message if no product with that `id` exists.

Use:

- `req.url.split("/")` to pull the `id` segment out of the URL string.
 - `.find()` on the products array to locate the matching product.
 - A conditional check for "not found" that still responds with valid JSON (e.g. `{ "error": "Product not found" }`).
-

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	The server must start without errors and respond correctly on every required route.	35%	The server will be run and each route tested in the browser.
Correct use of the http module	Requests and responses must be handled correctly, with appropriate status codes and headers.	25%	Code review of request handling logic.
Correct use of npm/package.json	The project must have a valid package.json and use npm scripts appropriately.	15%	Review of package.json and npm script usage.
Code quality	Code should be clean, consistently indented, logically organized, and free of unnecessary code.	10%	Review of code formatting and structure.
Presentation	The output should be clearly demonstrated and the submitted work should be properly organized and labelled with the student's name.	15%	Review of the submitted output and project

Lab 12: Express.js REST API — Server Setup, Routing, and Basic CRUD Routes

Background

Lab 11 built an HTTP server using Node's built-in http module, and routing by hand quickly became repetitive. Express is a lightweight framework on top of Node's http module that makes defining routes, reading request data, and sending responses much simpler.

This lab covers setting up an Express server, routing for each HTTP method, and building a full CRUD API for a single resource, stored in memory. Connecting it to a real database is Lab 13.

Setting Up

npm install express

Add "type": "module" to package.json so import/export can be used instead of require:

package.json

```
{
  "name": "my-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
}
```

Server.js

```
import express from "express";
const app = express();

app.get("/", (req, res) => res.send("Hello from Express!"));

app.listen(3000, () => console.log("Server running at http://localhost:3000));
```

Figure 191: Setting Up

Routing, JSON, Params, and the Request Body

Server.js:

```
// One method per HTTP verb: app.get, app.post, app.put, app.delete
app.get("/about", (req, res) => res.send("About page"));

// res.json() sends an object as JSON, with the correct header
app.get("/api/users", (req, res) => {
  res.json([{ id: 1, name: "Ali" }]);
});

// A colon defines a route parameter, read from req.params
app.get("/api/users/:id", (req, res) => {
  res.send(`You asked for user ${req.params.id}`);
});

// Middleware to parse a JSON request body - add this once, before any route that reads req.body
app.use(express.json());

app.post("/api/users", (req, res) => {
  console.log(req.body); // the parsed JSON the client sent
  res.send("Received!");
});
```

Figure 192: Routing, JSON, Params, and the Request Body

Tip: `app.use(express.json())` must be added before the routes that need `req.body` — it's middleware, meaning it runs on every request before your route handler does.

Building a CRUD API

Server.js

```
// server.js
import express from "express";
const app = express();
app.use(express.json());

let books = [
  { id: 1, title: "1984", author: "George Orwell" },
  { id: 2, title: "Dune", author: "Frank Herbert" },
];
let nextId = 3;

app.get("/api/books", (req, res) => res.json(books));

app.get("/api/books/:id", (req, res) => {
  const book = books.find((b) => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  res.json(book);
});

app.post("/api/books", (req, res) => {
  const newBook = { id: nextId++, title: req.body.title, author: req.body.author };
  books.push(newBook);
  res.status(201).json(newBook);
});

app.put("/api/books/:id", (req, res) => {
  const book = books.find((b) => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  book.title = req.body.title;
  book.author = req.body.author;
  res.json(book);
});

app.delete("/api/books/:id", (req, res) => {
  books = books.filter((b) => b.id !== Number(req.params.id));
  res.status(204).end();
});

app.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

Figure 193: Building a CRUD API

Tip: `res.status(404)` (or 201, 204) explicitly sets the status code — this is how Postman or a frontend's `fetch()` call knows whether a request succeeded.

Home Tasks (Before Lab)

Home Task 01 — Install Express, add `"type": "module"` to `package.json`, and run the minimal "Hello from Express!" server.

Home Task 02 — Match each HTTP method (GET, POST, PUT, DELETE) to the CRUD operation it represents, in one sentence each.

Home Task 03 — In one sentence, explain what `app.use(express.json())` does and why it must come before routes that use `req.body`.

Lab Objectives

The students will be able to:

- Set up a basic Express server using ES Modules.
 - Define routes for GET, POST, PUT, and DELETE requests.
 - Use route parameters to target a specific resource by id.
 - Read JSON data sent in a request body.
 - Build a complete CRUD API for a resource stored in memory.
 - Use appropriate HTTP status codes in responses.
-

Walkthrough Task

Task 1: GET Routes and Route Parameters

Objective: To serve a list and a single item, using a route parameter.

Description: Build a `/api/books` route that lists all books, and `/api/books/:id` that returns one, with a 404 for no match.

Steps

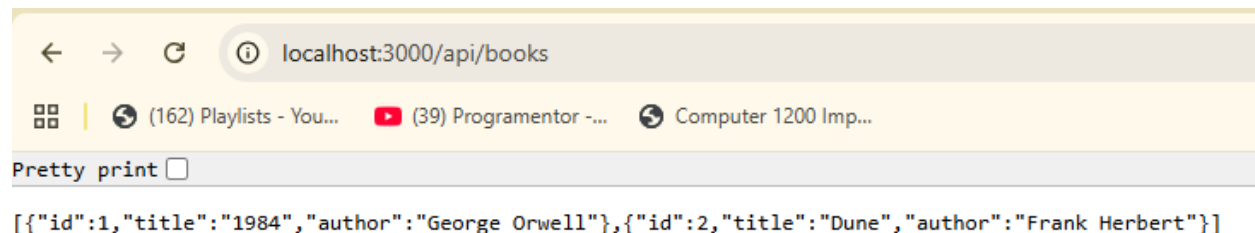
- Step 1: Create `server.js`, add `"type": "module"` to `package.json`, and import Express.
- Step 2: Define an array of at least two books above the routes.
- Step 3: Add `app.get("/api/books", ...)` returning `res.json(books)`.
- Step 4: Add `app.get("/api/books/:id", ...)`, using `.find()` with `Number(req.params.id)`.
- Step 5: Return `res.status(404).json({ error: "...", })` if no book matches.

Server.js:

```
server.js / ...
1 import express from "express";
2 const app = express();
3
4 const books = [
5   { id: 1, title: "1984", author: "George Orwell" },
6   { id: 2, title: "Dune", author: "Frank Herbert" },
7 ];
8
9 app.get("/api/books", (req, res) => res.json(books));
10
11 app.get("/api/books/:id", (req, res) => {
12   const book = books.find((b) => b.id === Number(req.params.id));
13   if (!book) return res.status(404).json({ error: "Book not found" });
14   res.json(book);
15 });
16
17 app.listen(3000, () => console.log("Server running at http://localhost:3000"));
18
```

Figure 194: Task 1: GET Routes and Route Parameters

Output: /api/books lists both books; /api/books/1 returns just that one; /api/books/99 returns a 404 JSON error.



The screenshot shows a web browser window with the address bar displaying `localhost:3000/api/books`. Below the address bar, there are several tabs: "(162) Playlists - You...", "(39) Programator - ...", and "Computer 1200 Imp...". A "Pretty print" checkbox is visible. The main content area displays the JSON response: `[{"id":1,"title":"1984","author":"George Orwell"}, {"id":2,"title":"Dune","author":"Frank Herbert"}]`.

Figure 195: Task 1: GET Routes and Route Parameters

Task 2: Completing CRUD (POST, PUT, DELETE)

Objective: To let books be created, updated, and deleted through the API.

Description: Extend Task 1's server with the remaining three routes.

Steps

- Step 1: Add `app.use(express.json())` before the routes, and a let `nextId` counter.
- Step 2: Add `app.post("/api/books", ...)`, building a new book from `req.body`, pushing it, and responding with status 201.

- Step 3: Add `app.put("/api/books/:id", ...)`, finding the book, overwriting title/author, and responding with the updated book.
- Step 4: Add `app.delete("/api/books/:id", ...)`, filtering the book out and responding with status 204.
- Step 5: Test all three with Postman (browsers can't send POST/PUT/DELETE just by visiting a URL).

Server.js:

```
app.use(express.json());
let nextId = 3;
app.post("/api/books", (req, res) => {
  const newBook = { id: nextId++, title: req.body.title, author: req.body.author };
  books.push(newBook);
  res.status(201).json(newBook);
});
app.put("/api/books/:id", (req, res) => {
  const book = books.find((b) => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  book.title = req.body.title;
  book.author = req.body.author;
  res.json(book);
});
app.delete("/api/books/:id", (req, res) => {
  books = books.filter((b) => b.id !== Number(req.params.id));
  res.status(204).end();
});
```

Output: POST adds a book and returns it with status 201; PUT updates an existing one; DELETE removes it, and a following GET no longer shows it.

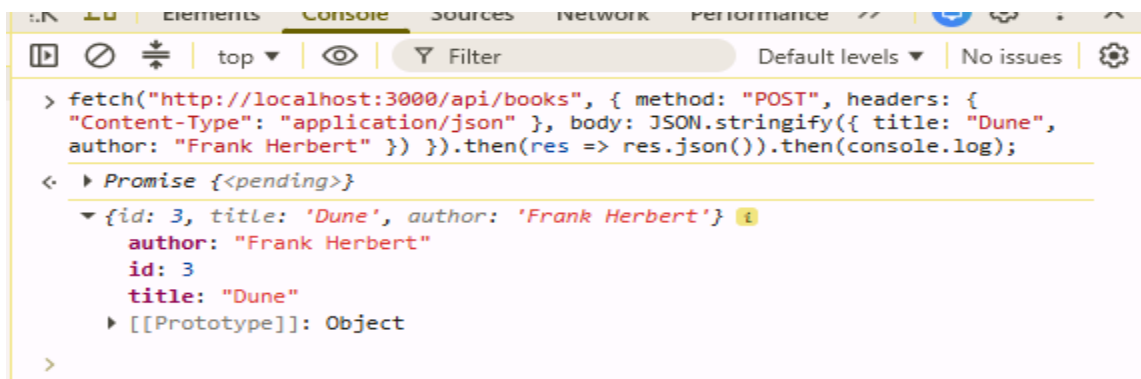


Figure 196: Task 2: Completing CRUD (POST, PUT, DELETE)

```

> fetch("http://localhost:3000/api/books/1", { method: "PUT", headers: {
  "Content-Type": "application/json" }, body: JSON.stringify({ title: "1984
(Updated)", author: "George Orwell" }) }).then(res =>
  res.json()).then(console.log);

```



```

< ▶ Promise {<pending>}
  ▼ {id: 1, title: '1984 (Updated)', author: 'George Orwell'} ⓘ
    author: "George Orwell"
    id: 1
    title: "1984 (Updated)"
    ▶ [[Prototype]]: Object

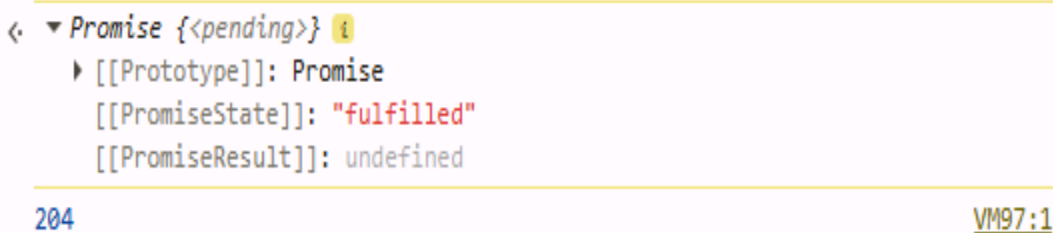
```

Figure 197: Task 2: Completing CRUD (POST, PUT, DELETE)

```

> fetch("http://localhost:3000/api/books/2", { method: "DELETE" }).then(res =>
  console.log(res.status));

```



```

< ▼ Promise {<pending>} ⓘ
  ▶ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: undefined

```

204 VM97:1

Figure 198: Task 2: Completing CRUD (POST, PUT, DELETE)

Practice Tasks

Practice Task 1: Products CRUD API

Build a full CRUD API for a products resource (id, name, price) in an in-memory array — GET all, GET one, POST, PUT, DELETE.

Use `express.json()` middleware, route parameters, and correct status codes (200, 201, 204, 404).

Practice Task 2: Input Validation

Add a check to the POST/PUT routes: if name or price is missing from `req.body`, respond with status 400 and an error message before touching the array.

Practice Task 3: Filtering with Query Parameters

Add GET `/api/products?minPrice=100` that returns only products at or above that price, using `req.query.minPrice` (converted with `Number()`) and `.filter()`, only applying the filter if the query param was given.

Assessment Criteria

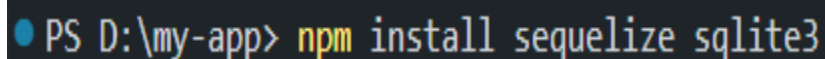
Criterion	Description	Weighting	Evaluation Method
Correctness	The API must run without errors and each CRUD route must behave correctly.	35%	Each route tested (e.g. with Postman) against requirements.
Correct use of Express routing	Routes, route params, and query params used correctly.	25%	Code review of route definitions and parameter handling.
Correct use of request/response handling	Bodies read correctly; responses use correct status codes and JSON.	20%	Testing of bodies, status codes, and response format.
Code quality	Clean, consistently indented, organized code.	10%	Review of formatting and structure.
Presentation	Output clearly demonstrated, work organized and labelled.	10%	Review of submitted output and folder.

Lab 13: SQL & Sequelize — SQL CRUD Operations and Connecting Express to SQL Using Sequelize

Background

Lab 12's API loses all its data on every restart, since it's stored in a plain array. Real applications store data in a database. This lab covers basic SQL — the standard language for relational databases — and Sequelize, an ORM that lets Express talk to a SQL database with JavaScript objects and methods instead of raw SQL.

Setting Up



```
PS D:\my-app> npm install sequelize sqlite3
```

This lab uses SQLite — a file-based database needing no separate server. With "type": "module" already set (Lab 12), relative imports must include their file extension: import sequelize from `"./db.js"`, not `"./db"`.

SQL Basic

```
CREATE TABLE Books (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  author TEXT NOT NULL  
);
```

```
INSERT INTO Books (title, author) VALUES ('1984', 'George Orwell'); -- Create  
SELECT * FROM Books; -- Read (all)  
SELECT * FROM Books WHERE id = 1; -- Read (one)  
UPDATE Books SET title = 'Nineteen Eighty-Four' WHERE id = 1; -- Update  
DELETE FROM Books WHERE id = 1; -- Delete
```

Tip: a WHERE clause is what limits UPDATE/DELETE to one row — leaving it off changes every row in the table.

Connecting Sequelize and Defining a Model

db.js:

```
1 // db.js
2 import { Sequelize } from "sequelize";
3
4 const sequelize = new Sequelize({ dialect: "sqlite", storage: "./database.sqlite" });
5 export default sequelize;
```

Figure 199: Connecting Sequelize and Defining a Model

models/book.js:

```
import { DataTypes } from "sequelize";
import sequelize from "../db.js";

const Book = sequelize.define("Book", {
  title: { type: DataTypes.STRING, allowNull: false },
  author: { type: DataTypes.STRING, allowNull: false },
});

export default Book;
```

`sequelize.sync()` creates the actual table in the database file if it doesn't exist yet — call it once, in `server.js`.

CRUD with Sequelize

Server.js:

```
import express from "express";
import { Sequelize, DataTypes } from "sequelize";
const app = express();
app.use(express.json());
// Connect to a local SQLite database file (creates it automatically if it doesn't exist)
const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: "./database.sqlite",
});
// Define the Book model — maps to a "Books" table
const Book = sequelize.define("Book", {
  title: { type: DataTypes.STRING, allowNull: false },
  author: { type: DataTypes.STRING, allowNull: false },
});
// Create the table if it doesn't exist yet
await sequelize.sync();
app.get("/api/books", async (req, res) => {
  const books = await Book.findAll();
  res.json(books);
});
app.post("/api/books", async (req, res) => {
  const newBook = await Book.create({ title: req.body.title, author: req.body.author });
  res.status(201).json(newBook);
});
app.put("/api/books/:id", async (req, res) => {
  const book = await Book.findByPk(req.params.id);
  if (!book) return res.status(404).json({ error: "Book not found" });
  book.title = req.body.title;
  book.author = req.body.author;
  await book.save();
  res.json(book);
});
app.delete("/api/books/:id", async (req, res) => {
  const book = await Book.findByPk(req.params.id);
  if (!book) return res.status(404).json({ error: "Book not found" });
  await book.destroy();
  res.status(204).end();
});
app.listen(3000, () => console.log("Server running on port 3000"));
```

Figure 200: CRUD with Sequelize

Every Sequelize call that touches the database returns a Promise, so each needs await — same as fetch() in Lab 10. Every Express route that uses await must itself be marked async.

Home Tasks (Before Lab)

Home Task 01 — Write the four SQL statements (INSERT, SELECT, UPDATE, DELETE) for a Students table with id, name, grade.

Home Task 02 — Install sequelize and sqlite3, and confirm sequelize.sync() creates a database.sqlite file without errors.

Home Task 03 — In one sentence, explain what an ORM does and why Book.findAll() is used instead of writing SELECT * FROM Books directly.

Lab Objectives

The students will be able to:

- Write basic SQL statements for creating, reading, updating, and deleting rows.
 - Connect an Express project to a SQLite database using Sequelize.
 - Define a Sequelize model with ES Module imports/exports.
 - Perform CRUD operations using Sequelize methods.
 - Rebuild an Express CRUD API so data is persisted in a real database.
-

Walkthrough Task

Task 1: Connect, Define a Model, and Read/Create Books

Objective: To connect to SQLite, define a model, and wire up GET/POST routes.

Description: Set up Sequelize, define a Book model, and rebuild the list + create routes from Lab 12 to use the database.

Steps

- Step 1: Install sequelize/sqlite3; create db.js exporting a Sequelize instance (dialect: "sqlite").
- Step 2: Create models/Book.js, defining title and author as required DataTypes.STRING.
- Step 3: In server.js, import the model, call sequelize.sync(), and start app.listen inside its .then().
- Step 4: Add app.get("/api/books", async (req, res) => { ... }) using await Book.findAll().
- Step 5: Add app.post("/api/books", async (req, res) => { ... }) using await Book.create({ ... }), responding with status 201.

db.js:

```
// db.js
import { Sequelize } from "sequelize";
const sequelize = new Sequelize({ dialect: "sqlite", storage: "./database.sqlite" });
export default sequelize;
```

Figure 201: Task 1: Connect, Define a Model, and Read/Create Books

models/book.js:

```
// models/Book.js
import { DataTypes } from "sequelize";
import sequelize from "../db.js";

const Book = sequelize.define("Book", {
  title: { type: DataTypes.STRING, allowNull: false },
  author: { type: DataTypes.STRING, allowNull: false },
});
export default Book;
```

server.js:

```
// server.js
import express from "express";
import sequelize from "../db.js";
import Book from "../models/book.js";

const app = express();
app.use(express.json());

app.get("/api/books", async (req, res) => {
  res.json(await Book.findAll());
});

app.post("/api/books", async (req, res) => {
  const newBook = await Book.create({ title: req.body.title, author: req.body.author });
  res.status(201).json(newBook);
});

sequelize.sync().then(() => {
  app.listen(3000, () => console.log("Server running at http://localhost:3000"));
});
```

Figure 202: Task 1: Connect, Define a Model, and Read/Create Books

Output: a database.sqlite file is created; POST adds a row that survives a server restart; GET returns it.

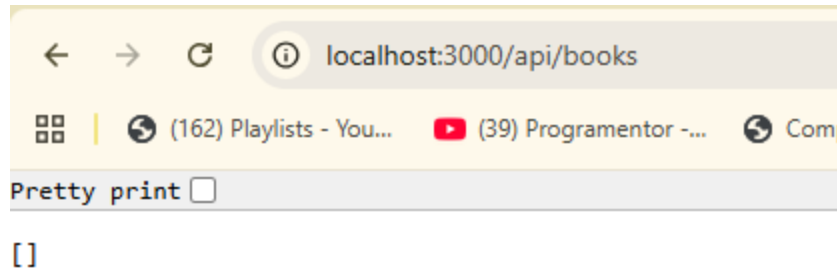


Figure 203: Task 1: Connect, Define a Model, and Read/Create Books

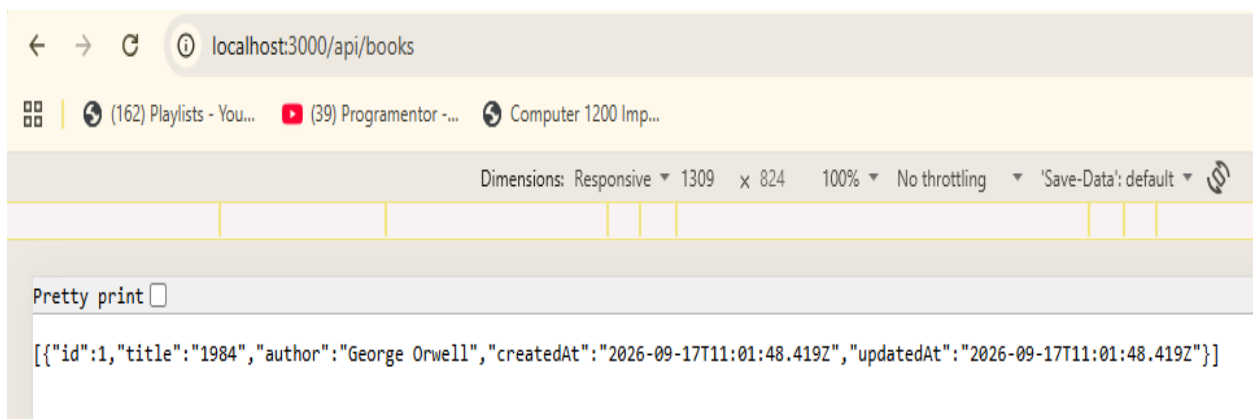


Figure 204: Task 1: Connect, Define a Model, and Read/Create Books

Task 2: Updating and Deleting Books

Objective: To complete the CRUD set with Update and Delete.

Description: Add PUT and DELETE routes to Task 1's server.

Steps

- **Step 1:** Add `app.put("/api/books/:id", async (req, res) => { ... })`.
- **Step 2:** Use `await Book.findById(req.params.id)`; respond 404 if null.
- **Step 3:** Otherwise set title/author from `req.body` and call `await book.save()`.
- **Step 4:** Add `app.delete("/api/books/:id", async (req, res) => { ... })`, same `findById` + 404 check, then `await book.destroy()`, responding 204.
- **Step 5:** Test both using the browser's DevTools Console (Inspect → Console) with `fetch()` calls for PUT and DELETE, then confirm the change by reloading GET `/api/books` in a browser tab afterward.

```
// Test PUT
fetch("http://localhost:3000/api/books/1", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Nineteen Eighty-Four", author: "George Orwell" })
}).then(res => res.json()).then(console.log);

// Test DELETE
fetch("http://localhost:3000/api/books/1", { method: "DELETE" })
  .then(res => console.log(res.status));
```

Figure 205: Task 2: Updating and Deleting Books

Server.js:

```
// server.js
import express from "express";
import sequelize from "../db.js";
import Book from "../models/book.js";

const app = express();
app.use(express.json());

app.get("/api/books", async (req, res) => {
  res.json(await Book.findAll());
});

app.post("/api/books", async (req, res) => {
  const newBook = await Book.create({ title: req.body.title, author: req.body.author });
  res.status(201).json(newBook);
});

app.put("/api/books/:id", async (req, res) => {
  const book = await Book.findByPk(req.params.id);
  if (!book) return res.status(404).json({ error: "Book not found" });
  book.title = req.body.title;
  book.author = req.body.author;
  await book.save();
  res.json(book);
});

app.delete("/api/books/:id", async (req, res) => {
  const book = await Book.findByPk(req.params.id);
  if (!book) return res.status(404).json({ error: "Book not found" });
  await book.destroy();
  res.status(204).end();
});

sequelize.sync().then(() => {
  app.listen(3000, () => console.log("Server running at http://localhost:3000"));
});
```

Figure 206: Task 2: Updating and Deleting Books

Output: PUT updates the row in the database; DELETE removes it, and a following GET no longer includes it.

Steps for students to see the required output:

1 1. Restart the server

In the terminal, stop any previously running server with Ctrl+C, then run: `node server.js` — confirm it prints "Server running at http://localhost:3000".

2 2. Open GET /api/books and note a real id

Type `http://localhost:3000/api/books` into the browser's address bar and press Enter. Note the id of at least one existing book (add one with a POST `fetch()` call first if the list is empty).

3 3. Open DevTools Console from that same page

Right-click directly on the JSON text on that page → Inspect → Console tab.

4 4. Run the PUT fetch() call

`fetch("http://localhost:3000/api/books/1", { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ title: "Nineteen Eighty-Four", author: "George Orwell" }) }).then(res => res.json()).then(console.log);` Replace 1 with the real id from Step 2. The logged result should show the updated title.

5 5. Reload GET /api/books to confirm the update

Reload `http://localhost:3000/api/books` — the book's title should now show "Nineteen Eighty-Four" instead of the original title, confirming the row was updated in the database.

6 6. Run the DELETE fetch() call

`fetch("http://localhost:3000/api/books/1", { method: "DELETE" }).then(res => console.log(res.status));` Use the same real id. This should log 204.

7 7. Reload GET /api/books to confirm the delete

Reload `http://localhost:3000/api/books` one more time — that book should no longer appear in the list at all, confirming DELETE removed the row from the database.

Figure 207: Task 2: Updating and Deleting Books

Practice Tasks

Practice Task 1: Products Model and CRUD API

Define a Product model (name, price) and rebuild the Products CRUD API from Lab 12 using `findAll`, `create`, `findByPk`, `save`, `destroy`.

Practice Task 2: Filtering with WHERE

Add GET `/api/products?minPrice=100` using Sequelize's `where` with the `Op` operators (`Product.findAll({ where: { price: { [Op.gte]: minPrice } } })`), still returning everything if no `minPrice` is given.

Practice Task 3: A Second Related Model (Categories)

Add a Category model (name) and a `categoryId` field on Product. Add GET `/api/categories/:id/products` using `Product.findAll({ where: { categoryId: req.params.id } })`.

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	API runs without errors; data persists through the database across all routes.	35%	Tested with Postman, including surviving a restart.
Correct use of Sequelize models	Models defined correctly; database syncs correctly.	20%	Code review of model definitions and sync logic.
Correct use of Sequelize CRUD methods	<code>findAll/create/findByPk/save/destroy</code> used and awaited correctly.	25%	Code review and testing of each operation.
Code quality	Clean, consistently indented, organized code.	10%	Review of formatting and structure.
Presentation	Output clearly demonstrated, work organized and labelled.	10%	Review of submitted output and folder.

Lab 14: Full-Stack Integration & Deployment — Connecting Frontend, Backend, and Database

Background

Every previous lab worked on one layer in isolation: React (Labs 08–10), Express + Sequelize (Labs 11–13). This lab connects all three into one working app, and covers deploying it beyond localhost.

Setting Up

In the backend project:

npm install cors

Both the React app (port 5173) and the Express API (port 3000) need to be running in different terminals at the same time for this lab.

CORS

Browsers block JavaScript on one origin (localhost:5173) from calling a different origin (localhost:3000) by default — a security feature. The cors middleware tells Express to allow it:

Server.js

```
// server.js
import cors from "cors";
import express from "express";

app.use(cors());
const app = express();
app.use(express.json());
```

Tip: in production, restrict this — `cors({ origin: "https://my-app.vercel.app" })` — instead of allowing every origin.

Calling the Backend from React, and Environment Variables

BookList.jsx:

```
BookList.jsx > ...
import { useState, useEffect } from "react";
const API_URL = import.meta.env.VITE_API_URL;

function BookList() {
  const [books, setBooks] = useState([]);

  useEffect(() => {
    async function loadBooks() {
      const response = await fetch(`${API_URL}/api/books`);
      setBooks(await response.json());
    }
    loadBooks();
  }, []);

  return (
    <ul>
      {books.map((book) => <li key={book.id}>{book.title} - {book.author}</li>)}
    </ul>
  );
}
export default BookList;
```

Figure 208: Calling the Backend from React, and Environment Variables

Make The below File(.env):

.env (Vite requires the VITE_ prefix)

```
.env
1 VITE_API_URL=http://localhost:3000
```

Environment variables let the API's URL change between local development and a deployed backend without editing code. Restart the Vite dev server after changing .env.

Deployment

- **Backend:** push to GitHub, deploy on a platform like Render or Railway — it runs npm install then the start script ("start": "node server.js").

- **Frontend:** push to GitHub, deploy on a platform like Vercel or Netlify — it runs `npm run build` and serves the resulting static dist folder.
- **Connecting them:** set `VITE_API_URL` (in the hosting platform's dashboard, not a committed `.env`) to the deployed backend's URL, and update `cors()` to allow the deployed frontend's URL.

Tip: SQLite's local file usually doesn't persist on cloud platforms without permanent disk storage — a real deployment typically switches to a hosted database (e.g. PostgreSQL), which only requires changing dialect and connection details in `db.js`.

Home Tasks (Before Lab)

Home Task 01 — Run the Lab 13 API and the Lab 08–10 React app at the same time, on their two different ports.

Home Task 02 — Install `cors` in the backend and add `app.use(cors());`; confirm the server still starts.

Home Task 03 — In one sentence, explain why a browser blocks a request from `localhost:5173` to `localhost:3000` without CORS enabled.

Lab Objectives

The students will be able to:

- Connect a React frontend to an Express backend using `fetch`.
- Enable and understand CORS.
- Use environment variables to configure the API's base URL.
- Build a complete flow: React UI → Express route → Sequelize model → database.
- Deploy a React frontend and Express backend outside localhost.

Walkthrough Task

Task 1: Fetching and Displaying Books in React

Objective: To fetch and display data from the Express + Sequelize backend inside React.

Description: Build `BookList`, fetching from the running Books API.

Steps

- Step 1: Ensure the Lab 13 API is running, with cors() enabled.
- Step 2: Create .env with VITE_API_URL=http://localhost:3000.
- Step 3: In BookList.jsx, read import.meta.env.VITE_API_URL into a constant.
- Step 4: Inside useEffect, fetch `\${API\_URL}/api/books` and store the result with useState.
- Step 5: Render the list with .map(), and import BookList into App.jsx.

BookList.jsx:

```

BookList.jsx > [🔗] default
import { useState, useEffect } from "react";
const API_URL = import.meta.env.VITE_API_URL;

function BookList() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function loadBooks() {
      const response = await fetch(`${API_URL}/api/books`);
      setBooks(await response.json());
      setLoading(false);
    }
    loadBooks();
  }, []);

  if (loading) return <p style={styles.status}>Loading books...</p>;
  if (books.length === 0) return <p style={styles.status}>No books yet.</p>;

  return (
    <div style={styles.grid}>
      {books.map((book) => (
        <div key={book.id} style={styles.card}>
          <h3 style={styles.title}>{book.title}</h3>
          <p style={styles.author}>by {book.author}</p>
        </div>
      ))}
    </div>
  );
}

const styles = {
  grid: {
    display: "grid",
    gridTemplateColumns: "repeat(auto-fill, minmax(220px, 1fr))",
    gap: "16px",
    padding: "24px",
    maxWidth: "1000px",
    margin: "0 auto",
  },
  card: {
    background: "■ #ffffff",
    borderRadius: "10px",
    padding: "18px",
    boxShadow: "0 2px 8px ■ rgba(0,0,0,0.08)",
    border: "1px solid ■ #eee",
    transition: "transform 0.15s ease, box-shadow 0.15s ease",
  },
};

```

Figure 209: Task 1: Fetching and Displaying Books in React

```

    },
    title: {
      margin: "0 0 6px",
      fontSize: "1.05rem",
      color: "■ #1a1a1a",
    },
    author: {
      margin: 0,
      fontSize: "0.9rem",
      color: "■ #666",
    },
    status: {
      textAlign: "center",
      color: "■ #888",
      padding: "40px",
      fontSize: "1rem",
    },
  },
};

export default BookList;

```

Figure 210: Task 1: Fetching and Displaying Books in React

Output: the React app displays the books currently in the SQLite database, fetched live from Express.

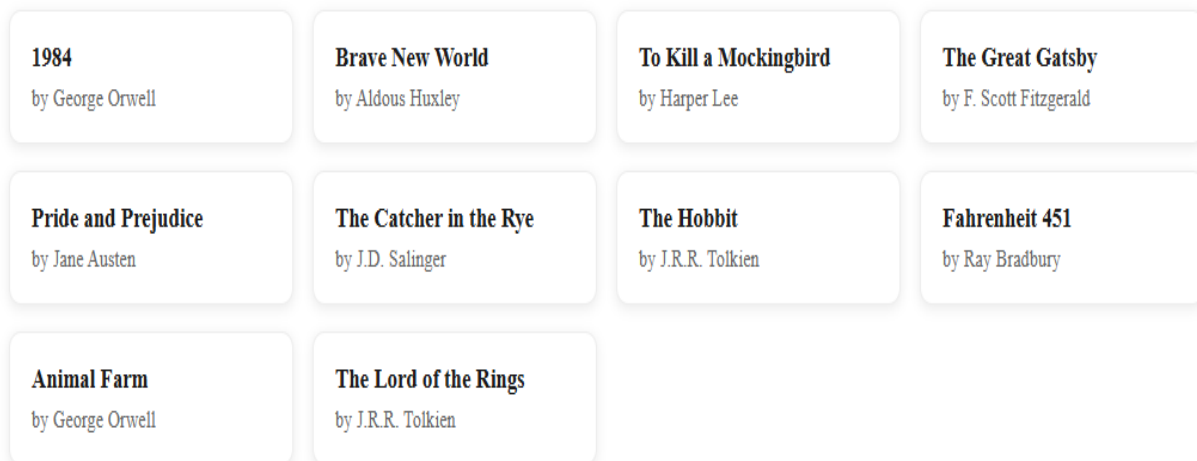


Figure 211: Task 1: Fetching and Displaying Books in React

Hint: Run the command **node server.js** in one terminal and don't close it (keep that running) and open another terminal in the same project and run the command **npm run dev**, then press **O+Enter** to see the output in the browser.

Task 2: Adding a Book from a Form

Objective: To send data from React to the backend and update the UI immediately.

Description: Add a form above the list; submitting it POSTs a new book and refreshes the list without a reload.

Steps

- Step 1: Add title/author inputs controlled with useState.
- Step 2: Write an async handleSubmit(event) that calls event.preventDefault(), then POSTs to `${API_URL}/api/books`.
- Step 3: Append the returned book to the books state so the UI updates immediately.
- Step 4: Clear both inputs after a successful submit.
- Step 5: Attach handleSubmit to the form's onSubmit.

BookList.jsx:

```
🔗 BookList.jsx > [🔗] default
import { useState, useEffect } from "react";
const API_URL = import.meta.env.VITE_API_URL;

function BookList() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");

  useEffect(() => {
    async function loadBooks() {
      const response = await fetch(`${API_URL}/api/books`);
      setBooks(await response.json());
      setLoading(false);
    }
    loadBooks();
  }, []);

  async function handleSubmit(event) {
    event.preventDefault();
    const response = await fetch(`${API_URL}/api/books`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title, author }),
    });
    setBooks([...books, await response.json()]);
    setTitle("");
    setAuthor("");
  }

  return (
    <div style={styles.page}>
      <form onSubmit={handleSubmit} style={styles.form}>
        <input
          value={title}
          onChange={(e) => setTitle(e.target.value)}
          placeholder="Title"
          style={styles.input}
        />
        <input
          value={author}
          onChange={(e) => setAuthor(e.target.value)}
          placeholder="Author"
          style={styles.input}
        />
        <button type="submit" style={styles.button}>
          Add Book
        </button>
      </form>
    </div>
  );
}
```

Figure 212: Task 2: Adding a Book from a Form

```

    </button>
  </form>

  {loading ? (
    <p style={styles.status}>Loading books...</p>
  ) : books.length === 0 ? (
    <p style={styles.status}>No books yet.</p>
  ) : (
    <div style={styles.grid}>
      {books.map((book) => (
        <div key={book.id} style={styles.card}>
          <h3 style={styles.title}>{book.title}</h3>
          <p style={styles.author}>by {book.author}</p>
        </div>
      ))}
    </div>
  )}
</div>
);
}

const styles = {
  page: {
    maxWidth: "1000px",
    margin: "0 auto",
    padding: "24px",
  },
  form: {
    display: "flex",
    flexWrap: "wrap",
    gap: "10px",
    marginBottom: "28px",
    padding: "16px",
    background: "■ #f7f7f9",
    borderRadius: "10px",
    border: "1px solid ■ #eee",
  },
  input: {
    flex: "1",
    minWidth: "160px",
    padding: "10px 12px",
    fontSize: "0.95rem",
    border: "1px solid ■ #ccc",
    borderRadius: "6px",
    outline: "none",
  },
};

```

Figure 213: Task 2: Adding a Book from a Form

```

    },
    button: {
      padding: "10px 20px",
      fontSize: "0.95rem",
      fontWeight: 600,
      color: "■ #fff",
      background: "■ #3b82f6",
      border: "none",
      borderRadius: "6px",
      cursor: "pointer",
    },
    grid: {
      display: "grid",
      gridTemplateColumns: "repeat(auto-fill, minmax(220px, 1fr))",
      gap: "16px",
    },
    card: {
      background: "■ #ffffff",
      borderRadius: "10px",
      padding: "18px",
      boxShadow: "0 2px 8px ■ rgba(0,0,0,0.08)",
      border: "1px solid ■ #eee",
    },
    title: {
      margin: "0 0 6px",
      fontSize: "1.05rem",
      color: "■ #1a1a1a",
    },
    author: {
      margin: 0,
      fontSize: "0.9rem",
      color: "■ #666",
    },
    status: {
      textAlign: "center",
      color: "■ #888",
      padding: "40px",
      fontSize: "1rem",
    },
  },
};

export default BookList;

```

Figure 214: Task 2: Adding a Book from a Form

Output: submitting the form saves the book via Sequelize and it appears in the list immediately, with no page reload.

1984
 by George Orwell

Brave New World
 by Aldous Huxley

To Kill a Mockingbird
 by Harper Lee

The Great Gatsby
 by F. Scott Fitzgerald

Pride and Prejudice
 by Jane Austen

The Catcher in the Rye
 by J.D. Salinger

The Hobbit
 by J.R.R. Tolkien

Fahrenheit 451
 by Ray Bradbury

Animal Farm
 by George Orwell

The Lord of the Rings
 by J.R.R. Tolkien

Figure 215: Task 2: Adding a Book from a Form

1984
 by George Orwell

Brave New World
 by Aldous Huxley

To Kill a Mockingbird
 by Harper Lee

The Great Gatsby
 by F. Scott Fitzgerald

Pride and Prejudice
 by Jane Austen

The Catcher in the Rye
 by J.D. Salinger

The Hobbit
 by J.R.R. Tolkien

Fahrenheit 451
 by Ray Bradbury

Animal Farm
 by George Orwell

The Lord of the Rings
 by J.R.R. Tolkien

The Great Victory
 by Jane Austen

Figure 216: Task 2: Adding a Book from a Form

Practice Tasks

Practice Task 1: Full CRUD in the React UI

Add a "Delete" button (calling the DELETE route) and an "Edit" mode (calling PUT) to each book in the list, updating state after each successful call.

Practice Task 2: Loading and Error States End-to-End

Add loading and error handling to every fetch call (load, add, edit, delete), including a visible message if the backend is unreachable (test by stopping the Express server).

Practice Task 3: Deploy the Full Stack

Deploy the backend (Render/Railway) and frontend (Vercel/Netlify). Restrict cors() to the deployed frontend's URL, set VITE_API_URL on the platform dashboard to the deployed backend, and confirm full CRUD works against the live app.

Assessment Criteria

Criterion	Description	Weighting	Evaluation Method
Correctness	Full-stack app runs without errors; data flows correctly React ↔ Express ↔ database.	35%	Each CRUD operation tested end-to-end through the UI.
Correct use of frontend-backend integration	fetch, CORS, and environment variables set up and used correctly.	25%	Code review of API calls, CORS setup, env variable usage.
Correct handling of loading/error states	Loading and error states handled gracefully across all operations.	15%	Testing under normal and failure conditions.
Deployment	Both frontend and backend successfully deployed and reachable outside localhost.	15%	Deployed URLs opened and tested directly.
Presentation	Output clearly demonstrated, work organized and labelled.	10%	Review of submitted output, folders, and deployed links.